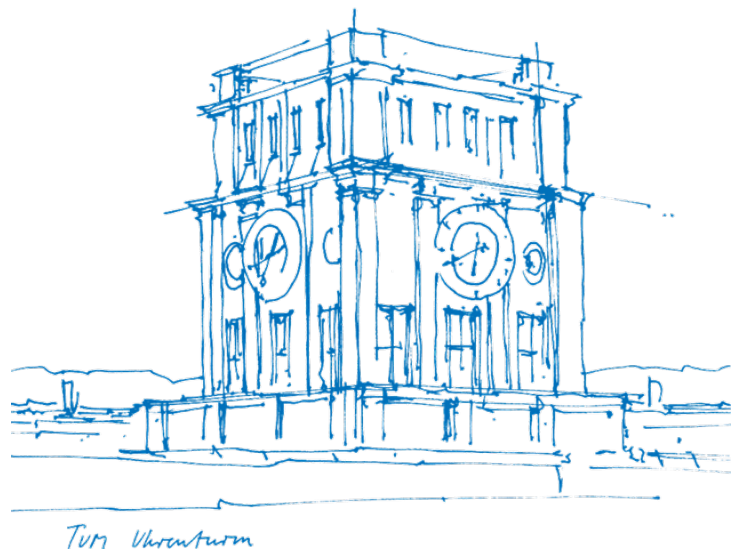


An Agentic LLM System for Executive Decision Support over Hierarchical KPI Outlier Data

Master's Thesis in Informatics

Hakan Duran



An Agentic LLM System for Executive Decision Support over Hierarchical KPI Outlier Data

Master's Thesis in Informatics

Hakan Duran

An Agentic LLM System for Executive Decision Support over Hierarchical KPI Outlier Data

Master's Thesis in Informatics

Hakan Duran

Thesis for the attainment of the academic degree

Master of Science (M.Sc.)

at the TUM School of Computation, Information and Technology of the Technical University of Munich.

Examiner:

Prof. Dr. Mario Trapp

Supervisors:

Dr. Marco Ratushny

Dr. Steffen Stiefvater

Start Date:

15.03.2026

Submitted:

Munich, 15.09.2026

I hereby declare that this thesis is entirely the result of my own work except where otherwise indicated. I have only used the resources given in the list of references.

Munich, 15.09.2026

Hakan Duran

Acknowledgements

I thank Dr. Marco Ratushny and Dr. Steffen Stiefvater for supervising this thesis. Their guidance, their careful feedback on every chapter, and their patience over the months of this work made this thesis possible.

I thank Prof. Dr. Mario Trapp for accepting me at his chair, for the opportunity to write this thesis there, and for acting as its examiner.

I thank Infineon Technologies for the opportunity to carry out this thesis in an industrial setting, and for the trust and support that made it possible.

Transparency in the Use of AI Tools

During my thesis work, I used GitHub Copilot with various language models and GPT IFX, Infineon's internal chat assistant, for design discussions, code planning and implementation, test development, and error investigation. All tasks were carried out under my guidance. The initial thesis outline was developed with these tools, following the objectives and structure I defined. Copilot also supported me in extracting evaluation evidence, checking answers against reference queries, preparing tables, refining diagrams, drafting and reviewing the abstract, and translating it from English into German.

The initial thesis draft was written by me. It was then revised under my supervision with support from Claude Code and Codex, including rephrasing existing text and drafting additional passages. In my personal environment, these tools were also used to identify gaps, check consistency, and improve clarity. Claude Code was also used to find additional literature, check references using publicly available sources, and support me in formatting the thesis using the TUM LaTeX template.

I remain responsible for the final text, source selection, interpretation of the evidence, and conclusions. AI-generated suggestions do not replace the cited sources or experimental records.

Abstract

Hierarchical analyses can identify many overlapping deviations in enterprise data without providing a concise report for top-level management. Follow-up questions also require business terms to be translated into appropriate analytical requests.

This thesis designs, implements, and evaluates an agentic system based on large language models. It connects weekly executive summaries with business-language questions over hierarchical key performance indicators and underlying operational data. An expert-reviewed semantic context supplies shared definitions. Tools calculate measurements and enforce constraints, while agents choose permitted reporting levels and decide which findings lead and which provide supporting detail. The conversational agent uses progressive tool disclosure to select predefined analyses. The orchestrator routes requests to guarded, read-only Text-to-SQL when no tool fits or the tool step cannot complete the request.

Completed live-agent reports passed the applied numerical and reporting checks. Agent decisions changed collected findings in the drill experiments and report composition in the summary comparisons. Shorter reports did not consistently leave less movement in the remainder. The conversational agent supported business-language questions and follow-up analysis of published summaries. On the evaluated questions, the tool-first configuration produced more fully correct answers and used fewer input tokens than direct Text-to-SQL, although it required more output tokens and model calls. Incomplete answers, unsupported explanations, and clarification failures remained. The findings indicate that agents can contribute analytical and reporting choices while tools perform the calculations. Whether the resulting reports are more useful to management requires further assessment.

Zusammenfassung

Hierarchische Analysen können zahlreiche überlappende Abweichungen in Unternehmensdaten aufzeigen, liefern daraus jedoch noch keinen kompakten Bericht für die oberste Führungsebene. Für weiterführende Fragen müssen zudem betriebliche Begriffe in passende Analyseanfragen übersetzt werden.

Diese Arbeit entwirft, implementiert und evaluiert ein agentisches System auf Basis großer Sprachmodelle. Es verbindet wöchentliche Managementberichte mit der Beantwortung fachlich formulierter Fragen zu hierarchischen Unternehmenskennzahlen und den zugrunde liegenden operativen Daten. Ein durch Experten geprüfter semantischer Kontext stellt gemeinsame Definitionen bereit. Werkzeuge übernehmen die Berechnungen und setzen feste Grenzen durch. Innerhalb dieser Grenzen wählen Agenten Berichtsebenen aus und entscheiden, welche Befunde im Vordergrund stehen und welche als ergänzende Details dienen. Der dialogbasierte Agent nutzt die schrittweise Bereitstellung von Analysewerkzeugen, um vordefinierte Analysen auszuwählen. Wenn kein Werkzeug passt oder der Werkzeugschritt die Anfrage nicht abschließen kann, leitet der Orchestrator sie an Text-to-SQL mit Schutzprüfungen für ausschließlich lesende Datenbankabfragen weiter.

Die abgeschlossenen Berichte mit Agentenentscheidungen bestanden die angewandten numerischen Prüfungen und Berichtskontrollen. Agentenentscheidungen veränderten die gesammelten Befunde in den Drill-down-Experimenten und die Zusammenstellung der Berichte in den Zusammenfassungsvergleichen. Bei kürzeren Berichten waren die verbleibenden Bewegungen nicht durchgehend geringer. Der dialogbasierte Agent unterstützte fachlich formulierte Fragen und weiterführende Analysen veröffentlichter Zusammenfassungen. Bei den untersuchten Fragen lieferte die werkzeuggesteuerte Konfiguration mehr vollständig korrekte Antworten und benötigte weniger Eingabetokens als direktes Text-to-SQL, allerdings mehr Ausgabetokens und Modellaufrufe. Unvollständige Antworten, nicht belegte Erklärungen und Fehler bei erforderlichen Rückfragen blieben bestehen. Die Ergebnisse deuten darauf hin, dass Agenten Analyse- und Berichtsentscheidungen beitragen können, während Werkzeuge die Berechnungen übernehmen. Ob die daraus entstehenden Berichte für das Management nützlicher sind, muss weiter untersucht werden.

Contents

Acknowledgements	v
Transparency in the Use of AI Tools	vi
Abstract	vii
Zusammenfassung	viii
Acronyms and Abbreviations	xii
1 Introduction	1
1.1 Problem	1
1.2 Objectives	2
1.3 Outline	3
2 Theoretical Background	4
2.1 KPI Data and Multidimensional Aggregation	4
2.2 Large Language Models and Agentic Systems	4
2.3 Tool Interfaces	5
2.3.1 Function Calling	5
2.3.2 The Model Context Protocol	6
2.3.3 Tool Exposure and Progressive Disclosure	6
2.4 Agent Loop Patterns	6
2.5 Memory in Agentic Systems	7
2.5.1 Working Memory	7
2.5.2 Semantic Memory	7
2.5.3 Experiential Memory	7
2.5.4 Long-Term Memory	7
2.5.5 Multi-Agent Memory	8
2.6 Checking LLM Answers	8
2.7 Text-to-SQL and the Semantic Layer	8
2.7.1 The Text-to-SQL Task	8
2.7.2 Benchmarks	8
2.7.3 The Semantic Layer	8
2.7.4 Ambiguity	9
2.8 Retrieval and Value Grounding	9
2.9 Grouping and Ranking Findings	9
3 Related Work	11
3.1 Automatic Semantic Context Construction	11
3.1.1 Automatic Database Description Generation	11
3.1.2 ktx	11
3.2 Explaining and Summarizing KPI Deviations	12
3.3 LLM Agents for Data Analysis	13
3.4 Text-to-SQL Systems	14
3.4.1 XiYan-SQL	14

3.4.2	ktx Runtime	15
3.4.3	SOMA-SQL and Ambiguity	15
3.5	Summary of Research Gaps and Positioning	16
4	Methodology	17
4.1	Data Foundation and Semantic Context	17
4.1.1	Two-Tier Data Model	17
4.1.2	The Semantic Context	18
4.1.3	Semantic-Context Construction	20
4.2	Executive-Summary Pipeline	21
4.2.1	Problem Setting	22
4.2.2	Filtering and Concentration-Based Drill-Down	23
4.2.3	Same-Effect Clustering	26
4.2.4	Impact Scoring and Story Selection	28
4.2.5	Story Localization	30
4.2.6	Composition and Output	31
4.3	Context-Grounded Conversational Agent	32
4.3.1	Problem Setting	33
4.3.2	The Escalation Ladder	33
4.3.3	Predefined Analytical Tools	34
4.3.4	Text-to-SQL Methods	35
4.3.5	Ambiguity and Clarification	36
4.3.6	Execution Checks and Outcome Handling	37
4.3.7	Memory and Learning	37
5	Implementation	39
5.1	System Overview and Environment	39
5.2	Semantic Context and Builder	41
5.2.1	The Semantic Context	41
5.2.2	Semantic-Context Construction	43
5.3	Executive-Summary Pipeline	45
5.3.1	Filtering and Concentration-Based Drill-Down	45
5.3.2	Same-Effect Clustering	50
5.3.3	Impact Scoring and Story Selection	53
5.3.4	Story Localization	55
5.3.5	Composition and Output	58
5.4	Conversational Agent	59
5.4.1	The Ladder Orchestrator	59
5.4.2	The Predefined Tool Step	62
5.4.3	The Open Analysis Step	63
5.4.4	Guards and Typed Outcomes	64
5.4.5	Memory Stores	64
6	Results	66
6.1	Data and Execution Conditions	66
6.2	Semantic-Context Construction	66
6.2.1	Establishing Reporting Weeks and Fiscal Quarters	67
6.2.2	Resolving Ambiguity During Construction	67
6.3	Executive-Summary Pipeline	68
6.3.1	Filtering and Concentration-Based Drill-Down	69
6.3.2	Same-Effect Clustering	69
6.3.3	Impact Scoring and Story Selection	70

6.3.4	Remainder and Accounting	71
6.3.5	Story Localization	71
6.3.6	Composition and Reported Output	72
6.4	Context-Grounded Conversational Agent	73
6.4.1	Progressive Disclosure and a Predefined Tool	73
6.4.2	Open Analysis Through a ReAct Loop	73
6.4.3	Applying the Ambiguity Rules	74
6.4.4	Clarification and Continued Conversation	75
6.4.5	Escalation After an Attempted Tool Step	75
6.4.6	Refusal for a Question Outside the Data	75
6.4.7	Published Summaries and Follow-Up Questions	76
6.4.8	Observed Outcomes	76
7	Evaluation	77
7.1	Evaluation Scope	77
7.2	Data, Configurations, and Measures	77
7.2.1	Experimental Conditions	77
7.2.2	Development and Final Assessment	78
7.2.3	Measures	79
7.3	Evidence for the Semantic Context	79
7.4	Executive-Summary Evaluation	79
7.4.1	Numerical Consistency and Analytical Routes	80
7.4.2	Controlled Drill Decisions and Repetition	80
7.4.3	Real-Data Hierarchy Traversals	81
7.4.4	Paired Story Selection	82
7.4.5	Agent Choices in Complete Summary Runs	82
7.4.6	Readability and Management Usefulness	84
7.5	Conversational Evaluation	85
7.5.1	Questions and References	85
7.5.2	Execution Reliability	85
7.5.3	Answer Correctness	85
7.5.4	Value-Lookup Diagnostic	86
7.6	Model Usage and Method Comparison	87
7.6.1	Comparison Conditions	87
7.6.2	Tokens, Calls, and Time	87
7.6.3	Repeated Conversational Attempts	88
7.7	Research-Question Assessment and Limitations	88
8	Conclusion and Future Work	89
8.1	Conclusion	89
8.2	Future Work	90
A	Tool Inventory	92
B	Constructed Executive-Summary Examples	93
C	Large-Format Figures	97
D	Application Interface	101
E	Detailed Algorithms	108
	Bibliography	113

Acronyms and Abbreviations

AI	artificial intelligence
API	application programming interface
CEO	chief executive officer
HTML	Hypertext Markup Language
HTTP	Hypertext Transfer Protocol
JSON	JavaScript Object Notation
KPI	key performance indicator
LLM	large language model
MCP	Model Context Protocol
PDF	Portable Document Format
ReAct	reasoning and acting
SQL	Structured Query Language
YAML	YAML Ain't Markup Language

1 Introduction

The capabilities of large language models (LLMs) have advanced rapidly in recent years. GPT-4 demonstrated that a single general-purpose model can perform a wide range of complex tasks without task-specific training [1]. DeepSeek-R1 later showed that reinforcement learning can elicit strong reasoning capabilities, further extending the ability of LLMs to solve complex problems [2]. More recently, GPT-5 has continued this progression toward increasingly capable general-purpose reasoning systems [3]. This progress has also led to the development of agentic LLM systems, in which generative artificial intelligence (AI) models not only answer questions or generate text but also make decisions and call the necessary software tools to perform actions. In some cases, they can even generate or adapt tools themselves when needed [4]. Such systems also have the potential to support everyday business processes in enterprises. In particular, they can reduce workflow effort by taking over intermediate steps in tasks that are difficult to automate using traditional rule-based approaches.

Enterprise data analysis is one area where this potential can be particularly useful. Enterprises usually collect large amounts of raw operational data used to calculate their key performance indicators (KPIs) and apply different analytical methods to identify potential outliers or other relevant findings, for example through machine learning methods or mathematical rules. However, these findings are often still too technical for the top-level management that needs to use them, for example the chief executive officer (CEO), the heads of divisions, or other members of the executive board, creating a gap between the analytical results and the business decisions they support.

The goal of this thesis is to develop an agentic system that helps close this gap. The system turns flagged KPI deviations into a short weekly executive summary and also allows top-level management to ask questions about the underlying data while keeping the interaction in business language rather than technical language.

1.1 Problem

This thesis works with two main types of data. The first is a large amount of raw operational data used to calculate KPIs. The second is a hierarchical analytics layer that data engineers prepare from this raw data. The layer is built around business dimensions such as division and customer, and it stores KPI values at different levels of this hierarchy. For example, the confirmed orders of a division can be viewed as one value for the whole division and, one level deeper, as one value per customer of that division. On top of this layer, an upstream detection system compares the aggregated values with their expected levels and flags unusual deviations as outliers. In this thesis, however, the main question is not how these outliers are detected, but which of them are actually useful for top-level management.

The flagged outliers are still highly technical, and their importance for top-level management is not always clear. A single reporting week can contain many outliers, including duplicated or low-relevance findings. At the same time, locating the contributions to an outlier may require repeatedly drilling down through the KPI hierarchy.

A fixed reporting depth is difficult to apply in this setting. The same underlying movement can appear at different points of the hierarchy at the same time. For example, a deviation visible at division level may also appear under several customers or other lower-level categories. In some cases, one branch explains most of the change, while in others the change is shared across several branches. The reporting level must therefore depend on the observed distribution. It can also be difficult to understand whether multiple outliers represent different issues or the same underlying story appearing in different parts of the hierarchy.

Real enterprise data also contains many edge cases, which makes a complete set of fixed reporting rules difficult to define.

As a result, top-level management may receive a large number of technical findings without a clear indication of which ones actually matter or how they are connected.

A further concern is dependence on organization-specific meanings. Separating dimension definitions, business terms, and reporting parameters from analytical code makes adaptation more manageable. This separation can support deterministic procedures as well as agents. The reason for an agent is its proposed ability to interpret combined evidence and choose among permitted analyses, not configurability alone.

In the end, this thesis provides two concrete results on top of this data. The first is a weekly executive summary that turns the flagged outliers into a small number of stories. The top level of a KPI is easy to follow, but the explanation of a deviation sits deeper in the hierarchy. Finding it means drilling down and comparing many branches, which would be technically demanding and time-consuming to do manually. The summary does exactly this work. It selects material deviations, shows which business dimensions carry them, and records the supporting amounts. These contributions locate a movement without establishing its business cause. The second result is a conversational agent with which top-level management can ask follow-up questions about the findings and the underlying data in business language. Together, the two results make selected deviations and their analytical evidence accessible. Reported numbers are required to be traceable to computations, and the evaluation checks whether the delivered answers satisfy that requirement. With this information, management can act while a deviation is still current. A drop that a few customers carry can be raised with the responsible unit, an unexpected rise can be questioned before it is planned with, and a deviation that spreads across a whole division can be treated as a structural topic rather than a single event. Management retains responsibility for decisions. The system provides analytical findings for management to review.

1.2 Objectives

The main objective of this thesis is to design, implement, and evaluate an agentic LLM system for executive decision support over KPI outlier data. The system uses a hierarchical analytics layer together with access to the underlying raw data. It follows a tools-first approach: deterministic tools handle calculations and data access, while the agent controls the analysis and interprets the results. The system is built around two tasks that share the same data foundation, semantic context, and tool layer.

The first task produces a weekly executive summary from flagged KPI deviations. A drill agent uses tool-computed concentration evidence to choose among permitted reporting-level actions. Tools then compare the candidates on raw records and combine views of the same effect. A story-selection agent proposes which findings lead, which provide detail or context, and which remain in the disclosed remainder. Tools compute the reported amounts and check record ownership and accounting. Deterministic procedures provide the baseline and complete the run if an agent cannot finish a valid decision or plan. The summary evaluation asks what changes when agents choose among the permitted reporting actions. It examines which findings are collected, how they are grouped, and what remains outside the selected stories.

The second task is a context-grounded conversational agent for top-level management. Users ask questions in business language, and a ladder orchestrator controls how each question is handled. The system first tries a predefined analytical tool. When no tool fits or its result is insufficient, the orchestrator moves the question to an open analysis step that can build a guarded read-only query at runtime. Each answering step returns its response and outcome to the orchestrator. Explicit rules determine whether the system delivers an answer, asks for clarification, escalates once, refuses the question, or reports an incomplete attempt or technical error. The conversational system does not use a separate model to review completed answers. Answer correctness is assessed against separately queried reference results and recorded outcome requirements. The system keeps conversation state and provides a refusal outcome for questions that the available data cannot support.

A broader design principle behind both tasks is to keep the system as generic as possible. Dataset-specific knowledge is therefore separated from the main system and provided through semantic context

describing the KPIs, business dimensions, and their meaning. The core logic is designed to rely on more general signals rather than rules written for one specific dataset. Although the thesis evaluates the system on data from one organization, this separation is intended to make the same architecture easier to adapt to other organizations or business domains.

Research questions. The objectives lead to the following overall research question.

How can an agentic LLM system summarize what changed in weekly KPIs, identify where those changes are concentrated, and answer business-language questions for top-level management?

This question is examined through three subquestions.

- RQ1.** How do bounded LLM agents select reporting levels and organize findings into weekly summaries, and how do their choices differ from the corresponding deterministic procedures?
- RQ2.** How accurately can the conversational system interpret business questions and answer them using the available data?
- RQ3.** How does using predefined analytical tools affect answer accuracy and model usage compared with direct Text-to-SQL analysis?

The evaluation examines the summaries and answers produced in the studied company setting. It assesses numerical consistency, agent decisions, answer correctness, and model usage. It does not measure changes in management decisions.

Contributions. The thesis contributes a shared semantic foundation for reporting and conversation, a summary procedure that combines bounded agent choices with record ownership and accounting checks, and a tool-first conversational implementation evaluated against reference queries. The evaluation separates constraint compliance, execution reliability, whole-answer correctness, and model usage. These are system-level contributions rather than a new anomaly detector or a claim of improved management decisions.

1.3 Outline

The rest of this thesis is organized as follows. Chapter 2 introduces the concepts needed to follow the design: the data cube model and KPI hierarchies, outliers in aggregated data, agentic LLM systems and tool use, and the Text-to-SQL task with its semantic layer. Chapter 3 reviews related work on LLM agents for data analysis, Text-to-SQL systems, and automated insight generation, and positions this thesis against them. Chapter 4 presents the methodology: the shared data foundation and semantic context, the executive-summary pipeline, and the context-grounded conversational agent with its escalation ladder. Chapter 5 describes the implementation of the main components. Chapter 6 illustrates recorded behavior with constructed numerical examples that withhold company figures. Chapter 7 evaluates numerical consistency, agent decisions, conversational correctness, and model usage on the recorded experiments. Chapter 8 concludes the thesis and outlines future work.

2 Theoretical Background

This chapter introduces the concepts and techniques that the design in the later chapters builds on. It starts with the structure of the KPI data, then covers agentic LLM systems together with their tool interfaces, loop patterns, memory, and answer checking, and closes with the Text-to-SQL task, the semantic layer, retrieval and value grounding, and the grouping and ranking of findings. Each section states how the concept is used in this thesis.

2.1 KPI Data and Multidimensional Aggregation

Aggregated business data is commonly organized as a data cube. A cube describes a measure, for example a revenue value, along several dimensions such as division, customer, or product, and the same measure can be viewed at different levels of aggregation [5]. The dimensions themselves can form hierarchies, so that values roll up from finer members to coarser ones, and formal models for such multidimensional data with hierarchical dimensions were introduced early [6]. Moving from a coarser view to a finer one is called drill-down, and the opposite direction is called roll-up. Data warehouses store operational data in this form because most analytical questions ask for aggregated values rather than for single transactions [7].

The analytics layer of this thesis follows this model, and a few terms are used for it throughout the thesis. An analytic route is an ordered drill path through the dimensions, for example from division down to customer. A node is one KPI viewed on one route with concrete member values, such as the confirmed orders of one division and one of its customers. The node one level above is its parent, and the nodes one level below are its children. The share of a child expresses how much of the parent's value, or of the parent's deviation, that child carries. The route and node terms are this thesis's naming for the drill paths of the analytics layer, while cube, drill-down, and roll-up are standard terms.

An outlier in this data is a value that differs clearly from its expected value [8]. In the analytics layer, the expected value is the mean of the node's history, the deviation is the difference between the value and this mean, and its sign is the direction of the change. Outliers are flagged at every level of aggregation, so a single underlying deviation can appear as a flagged outlier on several routes and levels at the same time.

A parent's deviation decomposes over its children when they form a complete partition with consistent expected values. This distribution can take different shapes. As a small example, a division may show a deviation of 100 million euros. One customer may carry 86 of the 100, three customers may carry roughly equal parts of it, or the deviation may be spread over dozens of small customers. In this thesis, the degree to which few children carry the parent's deviation is called the concentration of the node. How the reporting level is selected from this property is a design decision of this thesis and is described in Chapter 4.

2.2 Large Language Models and Agentic Systems

LLMs are neural networks that generate text one token at a time. Given the tokens produced so far, the model predicts the next token, appends it, and repeats this process until the answer is complete. Because such models are trained on very large text corpora, they can follow instructions written in natural language and solve new tasks from a description and a few examples in the prompt, without task-specific training [1]. More recent models are also trained to work through a problem step by step before they answer, which improves their performance on multi-step reasoning tasks [2]. Two practical properties make LLMs useful as system components. They can produce output in a required structure, such as a JavaScript Object

Notation (JSON) object that follows a given schema, and they can be reused for many different tasks by changing only the prompt.

The implementation uses OpenAI’s GPT and Anthropic’s Claude model families. Its configuration includes `gpt-5.6-luna`, `claude-sonnet-5`, and `claude-opus-5` for different agent roles. Section 7.2 records the models selected for each experiment.

An LLM on its own can only produce text. It cannot read a database, run a calculation, or check its own result against the real data. Agentic LLM systems remove this limitation by placing the model in a loop with software. The model receives a task together with a description of the actions it may take, and it selects an action [9]. The surrounding software then performs the action and returns the result, and the model decides what to do next. The mechanism that connects the model to software is usually called tool use or function calling. Each tool is described to the model with a name, a purpose, and a typed parameter list. The model emits a structured call, the software executes it, and the output is returned to the model as text. Wang et al. [9] describe LLM-based agents through four components. A profile defines the agent’s role, a memory stores past information, a planning component decides how to approach the task, and an action component executes the chosen step. The software layer around the model has its own name. Ning et al. call this layer the agent harness. It consists of the tools, validators, memory, permission boundaries, execution loops, and feedback channels that turn a stateless model into a functional agent [4]. The harness decides which tools the model can see, checks the model’s outputs before they take effect, and stores state between steps. This thesis uses the term agentic LLM system for the whole system, that is, the model together with its harness, and uses the term harness only when the surrounding software layer itself is discussed.

A central design idea in this literature is that computation should be delegated to code rather than performed by the model. Gao et al. showed with program-aided language models that an LLM makes fewer errors when it writes a small program for the arithmetic or logic of a task and lets an interpreter run it, instead of computing the answer in text [10]. The Code as Agent Harness survey generalizes this point. Programs and tools act as deterministic sensors whose results the model interprets, and reliability improves when the model reasons about outputs it did not have to compute itself [4].

This thesis adopts the same division of labor as a design requirement. Reported values should come from deterministic tools or executed queries, for example when the analytics layer is filtered, shares and concentration are calculated, same-effect findings are grouped, or a Structured Query Language (SQL) query is executed. The LLM selects which tool to call, decides how deep to drill or which answering path to take, and turns tool outputs into plain language. As a result, the parts of the system that can be tested are tested as ordinary software, and the model’s role is limited to decisions and interpretation. This requirement does not guarantee that every generated claim is supported. Chapter 7 checks that distinction. Section 2.4 describes agent loop patterns, and Section 2.6 discusses model-based assessment and its limitations.

2.3 Tool Interfaces

The application must define how the model can access and call its tools. This section introduces function calling, the Model Context Protocol (MCP), and progressive tool disclosure.

2.3.1 Function Calling

The most common method sends the tool definitions together with every request [11]. Each tool is described with a name, a short description, and a parameter schema, the model returns a structured call, and the calling application executes it and sends the result back. This method became standard through the function-calling interfaces of the large model providers [12], and it is the method this thesis uses through its model gateway.

2.3.2 The Model Context Protocol

The MCP is an open protocol through which applications discover and invoke tools exposed by servers. It standardizes the connection between an application and a tool server, rather than the model’s function-calling interface. [13] In this thesis, adapters expose capabilities to external clients, while the conversational host calls the underlying components directly (Section 5.1).

2.3.3 Tool Exposure and Progressive Disclosure

A related harness decision is which tools the model is allowed to see. The Code as Agent Harness paper treats tool use as a governed interface between the model and external systems [4]. The harness must decide which tools are available, how their schemas are exposed, which permissions each tool receives, and how results are sanitized or compacted before they return to the model. The paper calls these control points tool lifecycle control and makes them explicit as hooks. Before a call, pre-use hooks can validate arguments and enforce permissions, and after a call, post-use hooks can summarize large outputs or update memory [4].

The paper also organizes tool use itself by the primary function the tools serve, and it names four kinds. Function-oriented tool use fills gaps in the model’s knowledge, environment-interaction tool use lets the agent act inside its working environment, verification-driven tool use treats tools as deterministic sensors that check progress, and workflow-orchestration tool use coordinates several tools and roles into one controlled process [4]. This taxonomy is defined for software engineering agents, but it transfers to this thesis. Most tools of this system serve a function-oriented role by computing values and fetching data. Tool argument checks and read-only query guards enforce execution constraints. The orchestrator’s dispatch is a form of workflow orchestration.

Since every exposed tool description occupies context and widens the model’s choice, exposing only the tools that are relevant for the current destination is one application of this control. In current agent frameworks, the staged form of this idea is known under the engineering term progressive disclosure [14]. The model first sees only short definitions of the available tools, and the full schema of a tool is loaded when the model selects it. The same retrieve-then-expose idea appears in the academic tool literature, where a retriever selects a small subset of relevant tools from a large collection instead of exposing every schema to the model [15, 16], and the term itself has recently entered the agent literature as well [17]. This thesis applies the same control at two levels. The selector first sees short definitions of the question-specific tools. When the Predefined Tool Step is selected, only the chosen tools and three core orientation tools are exposed with their full schemas. The Open Analysis Step uses a separate and smaller tool set for reading the semantic overview, grounding business names, and running guarded queries.

2.4 Agent Loop Patterns

Agentic LLM systems differ in how the loop between the model and its tools is organized. Reasoning and acting (ReAct) provides the loop pattern used in this thesis. Yao et al. introduced this approach. The model writes a short reasoning step, then chooses an action, observes the result, and continues with the next reasoning step [18]. Because every decision is made in the light of the latest observation, the pattern is flexible and easy to apply. However, the plan exists only inside the model’s generated text. When a task requires many steps, the model can lose track of its own plan, repeat work, or stop before the task is complete.

A contrasting pattern separates planning, execution, and verification into distinct stages. The harness survey describes this plan-execute-verify structure as a loop with an explicit plan and checks on completed steps [4].

The conversational system combines rule-based dispatch with ReAct-style tool loops inside its answering steps. The catalog selector uses a model to assess tool suitability. The orchestrator uses the selection

result and the returned outcomes to decide whether to deliver, ask, escalate, or refuse. Outcome handling follows explicit rules without an additional model call.

The Predefined Tool Step works with selected analytical tools. The Open Analysis Step works with the semantic overview, value lookup, and guarded queries. Tool results return to the calling model as observations that can guide its next action. The weekly-summary pipeline also uses bounded agent roles inside a staged analytical process.

2.5 Memory in Agentic Systems

An LLM processes each request within a limited context window, so an agentic system has to decide which information stays in the model's context, which is compressed into summaries, and which is stored outside the model and loaded only when needed. The Code as Agent Harness survey describes this state-management layer as the memory of the system and distinguishes several memory types by the role they play [4]. Wang et al. treat memory as one of the four core components of an LLM-based agent, since without it the agent cannot connect its current step to what it has already seen [9]. The following memory roles matter for this thesis.

2.5.1 Working Memory

Working memory holds the state of the task that is currently running, for example the steps taken so far, intermediate results, and open questions [4]. It is the part of memory that changes with every step. In this thesis, the conversation state of the chat agent is working memory. It carries the previous questions and the values that were already resolved, and because of this the agent can answer follow-up questions that depend on the earlier turns.

2.5.2 Semantic Memory

Semantic memory stores external knowledge in a form the agent can query, so that facts about the domain do not have to be packed into the prompt [4]. In this thesis, the semantic layer that describes the data warehouse is such a memory. It contains the meaning of tables, columns, and values, the business rules, and the time semantics, and both the executive summary pipeline and the conversational agent read from it at runtime.

2.5.3 Experiential Memory

Experiential memory keeps reusable experience from earlier runs, such as successful strategies or known failure cases [4]. The survey notes that the quality of the stored experience matters more than its amount. In this thesis, this type appears in a smaller role. The pipeline keeps its intermediate findings only for the run that produced them, which is working memory. Experiential memory proper is kept on the conversational side: every question is stored as a trajectory with its final outcome, and only trajectories that delivered an answer are read back, as previous approaches rather than as verified answers. The drill and story decisions of each weekly run are recorded with their reasons as audit material, not yet as experience the next run consults.

2.5.4 Long-Term Memory

Long-term memory governs when stored experience is compressed and promoted into durable knowledge, so that validated lessons survive in a compact and controlled form [4]. In this thesis, the durable knowledge is the semantic context and the predefined tool set themselves. Long-term memory is therefore the governed promotion process into them, described in Section 4.3.7: the mining of candidates from experiential memory is implemented, and the decision remains with an expert.

2.5.5 Multi-Agent Memory

Multi-agent memory is a shared store that several agents read and write, described in the survey as a shared blackboard between roles [4]. In this thesis, the artifacts produced by one task and consumed by the other play this role. The weekly summary produced by the executive summary pipeline is designed to be readable by the conversational agent, so that an executive’s follow-up question can be answered against the same findings the summary reported.

2.6 Checking LLM Answers

Since the output of an LLM can be wrong in ways that are hard to detect automatically, agentic systems often use a second model instance to check the first one’s answers. Zheng et al. introduced this approach as LLM-as-a-judge and showed that model-based judgments can agree well with human judgments, although they also carry known biases, for example a preference for longer answers or for the answer that appears first [19]. A further concern is self-evaluation, because a model that checks its own output tends to repeat its own blind spots. The Code as Agent Harness survey therefore argues that the verifier should be independent of the producer, and that a judge should interpret the results of deterministic checks rather than replace them [4]. The conversational system does not use a separate model to review completed answers at runtime. Tool results return to the calling agent as observations. Deterministic guards enforce execution constraints, and the orchestrator applies outcome rules. Answer correctness is assessed in the evaluation against separately queried reference results and recorded outcome requirements.

Model-based assessment remains part of semantic-context construction. Its proposed corrections enter a draft that requires expert approval before publication.

2.7 Text-to-SQL and the Semantic Layer

2.7.1 The Text-to-SQL Task

Text-to-SQL is the task of translating a natural-language question into an executable SQL query over a given database [20]. The input is the question together with a description of the database, and the output is a query that returns the requested values when it is executed. The task is attractive for enterprise use because it gives non-technical users access to data that would otherwise require an analyst. At the same time, a generated query can be wrong in ways that still execute and return plausible values, because the system silently commits to an interpretation of the question, and because of this the task needs careful checking in practice [21].

2.7.2 Benchmarks

Two benchmarks shaped how the task is evaluated. The Spider benchmark established the modern form, in which a system must generalize to databases it has not seen during development [22]. Later, the BIRD benchmark moved the task closer to enterprise reality by using large, noisy databases and questions that cannot be answered from the schema alone [23]. The authors of BIRD showed that models need external knowledge about the domain and about the stored values to produce correct queries, and this gap between schema and meaning is the central difficulty on real company data. The systems built for these benchmarks are discussed in Chapter 3.

2.7.3 The Semantic Layer

Because of this gap, practical Text-to-SQL systems describe the database to the model with more than its table definitions. Such a description is often called a semantic layer, and it is the same artifact that Section 2.5.2 introduced as the semantic memory of the system, seen here in its role for query generation.

For this role, the description must be presented to the model in a readable form. M-Schema is a compact schema representation that lists each table with its columns, types, descriptions, and example values in a form that is easy for an LLM to read, and it was shown to work better than raw table definitions [24].

2.7.4 Ambiguity

Even with a good description, a question can remain ambiguous. Somayajula et al. describe ambiguity as a central failure mode of the task and locate its sources in the question, in the schema, and in the model's interpretation [21]. The same word in a question may match several columns or several stored values, and the question may leave open which KPI or which time range is meant. Ambiguity is therefore a property of the pair of question and database together with its reading, not of the question alone. How a system detects and resolves such ambiguity, for example by asking the user or by probing the data, is one of the differences between the systems discussed in Chapter 3.

The context-grounded conversational agent of this thesis uses Text-to-SQL inside its Open Analysis Step. This step reads the approved semantic context, grounds business names through the value lookup, and constructs guarded read-only queries in a bounded ReAct-style loop. Text-to-SQL is therefore not a separate production arm. It is the method used when no predefined analytical tool can provide a supported answer. Chapter 4 describes this method and compares it with a SOMA-inspired sampling-and-selection method.

2.8 Retrieval and Value Grounding

A language model can only use knowledge that is either stored in its weights or provided in its context. Retrieval-augmented generation connects the two. Before the model answers, the system retrieves relevant pieces of external knowledge and places them in the prompt, so that the answer is grounded in retrieved evidence rather than in the model's memory alone [25]. In an agentic system, retrieval does not have to be a single step. The semantic memory described in Section 2.5.2 is queried in the same spirit, and the model can retrieve again whenever a step needs more evidence.

A special form of this grounding problem appears in Text-to-SQL. A question mentions values in the user's words, but the database stores them in its own spelling, and a query only matches when the stored form is used. The BIRD benchmark showed that finding the correct stored values is one of the main difficulties on realistic databases [23]. A practical solution is to index the values of the relevant columns and to match the literals of the question against this index with a string-similarity measure. This thesis follows this approach. The value lookup supports this grounding step. It contains values from the reviewed warehouse columns and searches them with Jaro-Winkler similarity. Composite fields are also split into their members, so a question can match one member without using the complete stored string. The returned matches are evidence for selecting the database spelling. A close match does not by itself trigger clarification. Ambiguity is handled by the policy described in Section 4.3.5.

2.9 Grouping and Ranking Findings

Because outliers are flagged at every level of aggregation, the same underlying deviation usually appears several times among the findings, and these duplicates must be grouped before anything is reported. The general problem of recognizing duplicate records is well known in the database literature [26], but in this thesis the grouping works on a different signal. Two outlier nodes describe the same deviation when they attribute the same deviation to the same underlying entities in the raw transactions, so the clustering step compares attributions rather than names, and two routes that reach the same customer through different dimensions can be recognized as one effect. The grouped effects are then ranked, because an executive summary can only contain a few of them. The ranking uses magnitude and concentration as its signals, since both are computable for any organization's data, and the thresholds that turn them into decisions are

parameters of the system rather than fixed constants. How these signals are combined into a score is part of the methodology and its evaluation.

3 Related Work

This chapter positions the system against the work that addresses its main parts. The discussion starts with methods that build a semantic description of a database. It then reviews research on hierarchical exploration, KPI localization, raw-data explanations, and automatic insight discovery. The conversational side is placed against data-analysis agents, reusable tool creation, Text-to-SQL systems, and work on ambiguous questions. The chapter ends by bringing these areas together and stating the gaps addressed by this thesis.

3.1 Automatic Semantic Context Construction

Section 2.7 introduced the semantic layer as the description that connects a database schema with its business meaning. Building this description by hand requires knowledge of both the database and the organization. Recent systems therefore try to generate at least part of it automatically.

3.1.1 Automatic Database Description Generation

Gao and Luo present a method for automatically generating missing database descriptions for Text-to-SQL systems. Its implementation is available as XiYan-DBDescGen [27, 28]. The method works in two directions. A coarse-to-fine pass first forms an overview of the database and then examines its tables and columns. A fine-to-coarse pass uses the detailed column information to improve the table descriptions. Column profiles and example values help the model recognize types such as codes, enumerations, dates, and free text.

The method processes tables and columns in separate LLM steps, but these steps do not work without context. A column step receives the database overview, the preliminary table description, and the relations between similar columns. The generated descriptions are then written into M-Schema and evaluated through their effect on downstream SQL accuracy. On the BIRD benchmark, they improve SQL accuracy over an undocumented schema, although they do not close the full gap to human-written descriptions [27].

This evaluation makes the intended role of the method clear. It prepares metadata for a Text-to-SQL pipeline rather than building a complete enterprise semantic layer. XiYan-SQL, discussed in Section 3.4.1, uses the same M-Schema representation together with schema filtering and specialized fine-tuned SQL generators. Together, the two works cover consecutive parts of a Text-to-SQL pipeline. The description method prepares missing metadata, and XiYan-SQL uses structured database context during query generation [27, 29].

The connection also shows the remaining limitation for this thesis. The generated context comes from the schema, stored values, and knowledge already present in the model. These sources may not explain which measure a company means by a business term, which calendar defines a quarter, or why two similar columns represent different organizational roles. The semantic context of this thesis therefore also includes existing business documentation, approved interpretation rules, and an explicit review process.

3.1.2 ktx

ktx is an open-source context layer for data agents [30]. Its main goal is to prevent an agent from rediscovering the warehouse and recreating metric logic for every question. Instead, the agent reads approved definitions from a prepared context.

The open-source system has two connected parts. The ingest prepares the context before a question is asked. At runtime, an agent searches this context and uses it while answering the question. This section focuses on the ingest; Section 3.4.2 returns to the agentic runtime. Keeping the two parts separate makes it possible to examine how prepared business knowledge changes the work of the agent.

The ktx ingest connects to data sources, samples tables, and records metadata such as data types, example values, usage patterns, and possible joins. It also reads existing organizational knowledge, including wiki pages and other documentation. The ingest organizes this material, removes duplicate content, and flags contradictions for review. Its output combines a readable wiki with structured semantic files that describe the data and its metrics.

This design addresses an important limitation of database-only description generation. Business knowledge can enter beside the schema instead of being inferred from column names. The semantic-context builder of this thesis uses the ktx ingest structure as its base. It extends this structure with explicit time semantics, a business glossary, approved interpretation rules, disambiguation entries between similar columns, and the calibration values used by the summary pipeline. The extensions are described in Section 4.1.

Table 3.1 summarizes the main difference between the reviewed approaches. The table describes the knowledge each approach makes available to the runtime. It does not compare their answer accuracy, because the systems were evaluated in different settings.

Table 3.1 Knowledge sources and controls in the reviewed semantic-context approaches.

Approach	Schema and stored values	Business documentation	Explicit time semantics	Stored ambiguity guidance
XiYan-DBDescGen	Yes	No	No	No
ktx	Yes	Yes	Not explicit	Contradiction review
Semantic context in this thesis	Yes	Yes	Yes	Yes

3.2 Explaining and Summarizing KPI Deviations

The executive-summary task is connected to earlier work on data cubes, KPI localization, raw-data explanations, and automatic insight discovery. This thesis does not try to replace these methods. They are reviewed because they provide useful parts of the evidence that the agent later uses to build a report.

Research on data cubes studies where an unusual movement appears in a business hierarchy. For example, an analyst may start with total sales and move through regions, customers, and products to find where the movement becomes visible. Discovery-driven exploration guides this search toward unusual parts of the cube. The RELAX operator starts from a detailed case and searches upward for broader levels where the same behavior can still be seen [31, 32].

KPI-localization methods start with one unusual KPI value and look for the parts of the data most strongly connected to it. These parts may be one region, one customer group, or a combination of several business dimensions. Adtributor, HotSpot, and Squeeze follow this general idea [33, 34, 35]. Adtributor combines explanatory power, succinctness, and surprise to rank candidate explanations. Its surprise measure compares distributions using Jensen–Shannon divergence. [33] This differs from the band-relative surprise signal used in this thesis. The distinction between the tasks is not anomaly detection, which can already precede localization. Here, an upstream system has detected many deviations across the hierarchy. The task is to decide which existing findings belong in the same weekly report and at which level they should be presented.

Database explanation methods show why the underlying records still matter. An aggregate value shows the total movement, but it does not show which records produced it. Scorpion searches for groups of

records that are strongly related to an unusual aggregate result. DIFF compares two groups of records and finds the values that separate them [36, 37]. This thesis uses raw records for a more limited purpose. It checks which records are shared by findings reached through different paths of the hierarchy. This shows whether two findings describe the same movement, whether one is mostly inside the other, or whether they only partly overlap. The comparison helps the system avoid reporting the same amount in several stories. It supports attribution, but it does not prove a business cause.

Automatic insight-discovery systems address another problem. A large dataset can contain many interesting patterns, but only some of them are useful to the reader. QuickInsights finds and ranks such patterns. MetaInsight groups related patterns and shows common behavior together with exceptions [38, 39]. Its ranking also reduces redundancy among the selected findings. The overlap model compares scope filters and pattern attributes, rather than assigning raw records to individual stories. [39] MetaInsight therefore provides a comparison for both grouping and selection, not only pattern discovery.

DataShot turns tabular data into graphical fact sheets. It groups facts into topics through shared context or focus filters. Within each topic, it selects a limited number of facts by balancing importance and diversity, then presents them through visualizations and descriptions. [40] This addresses the selection of a compact set of findings. The thesis’s Same-Effect Clustering has a different purpose. It compares the measured movements behind candidate views before those findings compete for space in the report.

Calliope extends automatic fact selection to narrative organization. A logic-oriented Monte Carlo tree search develops a sequence of facts and their connections. Its objective combines information, diversity, logical coherence, and data coverage. Coverage measures the union of records represented by the facts. [41] This is different from the signed amount assigned to each story in this thesis. The comparison shows that automatic selection and narrative organization do not themselves require an LLM.

The reporting task of this thesis also needs selection and structure, but its output is more specific. It produces a short report for one period under a limited story budget. Counter-movements and the unexplained remainder must stay visible, and the amounts in the final stories must agree with the measured total movement. Within these constraints, the agent can choose reporting levels near a concentration boundary and arrange related findings as stories or supporting detail. Tools retain control of the measurements and record ownership. These choices are evaluated without assuming that an agent produces a better report than the deterministic procedure.

These research areas provide methods for finding, explaining, and organizing unusual patterns. This thesis does not propose a new anomaly detector or a new root-cause localization algorithm. Its contribution is at the system level. Deterministic tools prepare traceable evidence from the hierarchical and raw data, while a bounded agent uses this evidence to construct the reporting story. Guards check the agent’s plan before any result is presented. Section 3.3 places this tool-augmented agentic design against earlier work on data agents and reusable tools.

3.3 LLM Agents for Data Analysis

LLM-based data agents extend natural-language access beyond a single generated query. They can plan an analysis, select tools, execute code or queries, inspect the results, and continue until they can answer the request. Recent surveys describe this area through properties such as semantic understanding, tool-augmented workflows, autonomous pipelines, and support for different data types [42]. This flexibility makes data agents useful for open analysis tasks. It also creates a control problem, because the system must decide which actions the model may perform and how the result is checked.

Ning et al. use the term agent harness for the code infrastructure that connects a model with tools, state, feedback, and execution-based verification [4]. This view is relevant here because the contribution is not a new language model. It is the controlled system around the model. In this thesis, the harness includes the shared semantic context, the orchestrator, the analytical tools, the guards, and the fallback paths.

InsightPilot provides a concrete example of an LLM directing a specialized insight engine. Given a business question, the model selects analytical actions such as understanding, summarizing, comparing, and explaining. The engine performs the analysis and returns insights, which guide further exploration

and the final answer. The system also reduces redundant insights and considers their relevance and diversity. [43]

This division of work is close to the weekly subsystem’s use of analytical tools. The difference is the reporting procedure around those choices. Here, the agent works with measured candidate relationships under a story budget, while tools enforce ownership and reconciliation. The conversational subsystem provides separate access to the same business definitions and data. InsightPilot is therefore a comparison for analytical orchestration, not a component reused in the implementation.

Data-Copilot separates preparation from runtime [44]. During data exploration, the system discovers common requests and turns them into reusable code interfaces. The interfaces are checked by a compiler before they are used. At runtime, the agent can call these prepared interfaces instead of generating a complete analysis from the beginning. This reduces repeated work and makes common requests more reliable. Data-Copilot therefore provides a close comparison for the predefined tools of this thesis.

The difference is the control over when a reusable tool enters the system. Data-Copilot creates interfaces through autonomous exploration. This thesis starts with approved analytical tools and tries them before a more open query method. Trajectories of delivered answers can later identify a repeated request as a candidate for a new tool, but an expert decides whether the candidate is implemented and approved. The runtime can therefore become cheaper over time without allowing the agent to change its own tool set directly.

Cai et al. study reusable tool creation more generally in Large Language Models as Tool Makers [45]. A tool-maker model creates a Python function for a family of tasks, while a tool-user model reuses that function for later problems. This separates the high cost of creating a tool from the lower cost of using it. The promotion process in this thesis has a similar motivation, but it begins with repeatedly answered enterprise questions and includes an expert gate. The system identifies where a tool may be useful; it does not automatically accept model-generated code as a new production tool.

The conversational system in this thesis is narrower than a general data analysis agent. It does not receive an open programming environment or choose freely among every available operation. The Ladder Orchestrator first tries a predefined analytical tool and moves to the Open Analysis Step only when needed. This step uses a bounded ReAct-based Text-to-SQL method, and the system refuses when the available data cannot support an answer. The research question is therefore not whether an LLM can perform arbitrary data analysis. It is whether a bounded, context-grounded system can answer executive questions while keeping calculations and evidence under tool control.

3.4 Text-to-SQL Systems

Text-to-SQL is one answering method inside the conversational agent. The task and its main benchmarks were introduced in Section 2.7. The closest systems for this thesis are XiYan-SQL, the ktx runtime, and SOMA-SQL. They show how prepared database context, agentic access, and candidate comparison can support an open business question.

3.4.1 XiYan-SQL

Section 3.1.1 described how XiYan-DBDescGen prepares missing database descriptions in an M-Schema-compatible form. DBDescGen and XiYan-SQL cover two connected stages of the same Text-to-SQL pipeline. DBDescGen prepares the database context, and a schema filter in XiYan-SQL selects the relevant tables, columns, and stored values from this context. Several specialized SQL-generation models then produce candidate queries, and a selection model chooses the final query.

The SQL-generation models are fine-tuned for Text-to-SQL and for different SQL forms. This gives them different ways of producing a query instead of depending on one general model. The generated descriptions are therefore used together with models adapted to the downstream task. The descriptions alone do not explain the performance of the complete pipeline [29].

The conversational agent in this thesis does not send every question directly to such a generator. It first tries a predefined analytical tool. Text-to-SQL is used when no suitable tool exists or the tool step cannot complete the request. The comparison with XiYan-SQL therefore concerns both how a query is generated and when query generation is needed.

3.4.2 ktx Runtime

Section 3.1.2 described how the ktx ingest prepares business and database context before a question is asked. The ktx runtime is the agentic side of the same design. An agent searches the prepared wiki and semantic files for the definitions relevant to a question. Requested metrics are compiled into read-only SQL instead of being redefined by the model. The approved meaning of a metric therefore remains in the prepared context [30].

The runtime is more than a single Text-to-SQL model. It places an agent around the query pipeline. The agent first retrieves the relevant context, then calls data tools and continues from their results. A question is therefore handled through a sequence of grounded actions rather than one model response.

ktx also has a reported result on the official Spider 2.0-Lite leaderboard. At the time of writing, the leaderboard reports a score of 73.67 for the complete system [46]. Spider 2.0-Lite contains 547 Text-to-SQL tasks over BigQuery, Snowflake, and SQLite. Its databases include metadata and documentation, which makes the benchmark relevant to context-grounded systems in enterprise-style settings. [47]

The leaderboard result provides external evidence for the complete ktx system. It does not show how much of the result comes from the ingest, the runtime, or one specific context feature. In addition, ktx is an actively developed open-source project. This thesis fixes the examined baseline at version 0.16.0 so that the implementation comparison remains repeatable. This does not make a claim about the production readiness of the complete ktx stack.

This thesis is inspired by the same context-first pipeline. It prepares shared business knowledge before runtime and lets the agent work through guarded data tools. Its two Text-to-SQL methods read this context, ground business names to stored values, and execute only read-only queries. The thesis adds an escalation step before this open process. An orchestrator first tries a predefined analytical tool and reaches Text-to-SQL when no suitable tool exists or the first step cannot complete the request.

3.4.3 SOMA-SQL and Ambiguity

Ambiguity is not limited to one system. AmbiQT and AMBROSIA show that one question can support several SQL interpretations even when the schema is available [48, 49]. PRACTIQ includes ambiguous and unanswerable questions together with clarification turns, while AmbiSQL presents possible interpretations to the user as multiple-choice questions [50, 51]. These works show why a conversational data system may need to clarify or refuse instead of always returning a query.

SOMA-SQL takes an autonomous approach with a preparation stage and a runtime stage. During preparation, it creates a synthetic query log containing examples of underspecified questions and their possible interpretations. At runtime, the system retrieves relevant examples, generates several SQL candidates, and compares them. A disagreement between the candidates points to a decision that may still be open. Targeted probe queries then collect evidence from the database for the final selection or repair [21].

The paper reports an average execution-accuracy improvement of 13.0 percent over its baselines across six public benchmarks, with gains of up to 16.7 percent on ambiguous questions [21]. These results cannot be compared directly with the ktx leaderboard score because the systems use different benchmarks and comparison settings. The SOMA-SQL paper defines the main pipeline, but it does not provide a drop-in implementation for the data and runtime constraints of this thesis.

The sampling-and-selection method of this thesis is inspired by SOMA-SQL rather than presented as its reproduction. It follows the same central idea by sampling several queries, using disagreement as a signal, and probing the database before selection. The ReAct-based method instead develops one interpretation through a bounded sequence of tool calls and can ask one short clarification question. Both methods use the same semantic context, value grounding, and read-only query guards. The evaluation compares them

on the same questions and data, so their practical trade-offs can be studied in one setting. Section 4.3.4 describes both methods.

3.5 Summary of Research Gaps and Positioning

InsightPilot provides a close comparison for agent-directed analysis. MetaInsight, DataShot, and Calliope address finding selection and organization. ktx prepares business context, while PRACTIQ and AmbiSQL address clarification. This thesis connects these research directions through weekly reporting with explicit movement accounting and conversational access over shared business definitions.

The first system-level gap concerns the step after deviation detection. The starting point of this thesis is a set of already-flagged deviations that can refer to the same raw records through different paths of a hierarchy. A useful weekly report must combine repeated views, preserve counter-movements, select a small number of stories, and account for the movement that is not explained. The reviewed methods already provide localization, finding selection, and narrative organization. The remaining design task here is to combine these with record-level accounting and bounded agent choices for the weekly reporting setting.

The second system-level gap concerns controlled conversational access over the same data. An open data agent is flexible, but its calculations, interpretations, and queries are difficult to control with one prompt. This thesis therefore studies an agent harness in which a shared semantic context provides business meaning, deterministic tools provide the numbers, and guards check the agent's actions. The harness first uses predefined tools and opens a Text-to-SQL path only when the question requires it. It can also clarify an unresolved question or refuse one that the available data cannot support.

The contribution is the design, implementation, and evaluation of this combined system in one enterprise data setting. It is a system-level contribution rather than a new anomaly detector, causal root-cause method, or foundation model. The evaluation examines whether the combined design produces traceable executive reports and reliable conversational answers, and it compares the two adapted Text-to-SQL methods under the recorded conditions. The evaluation identifies the implementation-version difference for the retained SOMA-inspired baseline.

4 Methodology

This chapter presents the method behind the system. It focuses on the design and the processing steps. The implementation follows in Chapter 5. The method has three parts. Section 4.1 defines the shared data foundation and the semantic context that both tasks read. Section 4.2 presents the executive-summary pipeline, which turns the flagged outliers of a week into a small number of traceable stories. Section 4.3 presents the conversational agent, which answers executive questions over the same data.

The two parts form a shared approach to weekly reporting and conversational analysis. The executive-summary pipeline organizes selected developments into a short summary. The conversational system lets top-level management examine those developments and ask further questions about the underlying data. Both read the same semantic context, giving them a common reference for KPIs, periods, names, and business rules. Whether an execution applies those definitions consistently remains a question for evaluation. Published summaries enter Shared Memory as evidence for follow-up questions. The chapter therefore builds the foundation first and then explains the decisions each task requires. Recorded questions and ambiguities can produce proposals for new tools and context entries, subject to expert approval.

Two principles guide the design. First, calculations are delegated to deterministic tools and queries. Agents select actions and interpret observations rather than supply business measurements. The summary composer checks reported amounts against its inputs. Conversational answers remain generated text, so compliance with the same requirement is assessed in evaluation rather than guaranteed by the prompt. Second, dataset-specific definitions and analytical parameters are separated from the core procedures through the semantic context. This is intended to limit the changes needed for another organization, although that transfer is not established here. Controls restrict actions and defined output conditions. The summary has a deterministic fallback, while the conversation has explicit escalation, refusal, and execution-failure outcomes.

4.1 Data Foundation and Semantic Context

Both tasks read the same data foundation. This section defines the foundation once, together with the semantic context that describes it. The rest of this chapter refers to these definitions.

4.1.1 Two-Tier Data Model

The system works on two data tiers. The first tier is the analytics layer, a pre-aggregated view of the KPIs that is prepared upstream of this work. It is the starting point of every analysis. The second tier is the raw transactional data, which contains the individual records behind the aggregated values. The raw tier is consulted only when the analytics layer cannot answer a question. Three situations require this. First, a question can refer to a combination of dimensions that the layer does not contain. Second, the method may need to determine whether two findings are based on the same records. Third, the analysis may require detail below the aggregated grain. In each case, access is restricted to the relevant records and dimensions. Figure 4.1 shows the two tiers and the situations in which the method crosses from one to the other.

The analytics layer is organized as a hierarchy. A route is an ordered combination of business dimensions, for example from an organizational unit down to a customer. Each node of the layer belongs to one KPI and one route and holds the aggregated measure for concrete values of these dimensions. Moving along a route adds one dimension at a time, so the layer can be read from aggregated levels down to detailed ones. Each node stores this construction explicitly, through the route, series, and parent fields

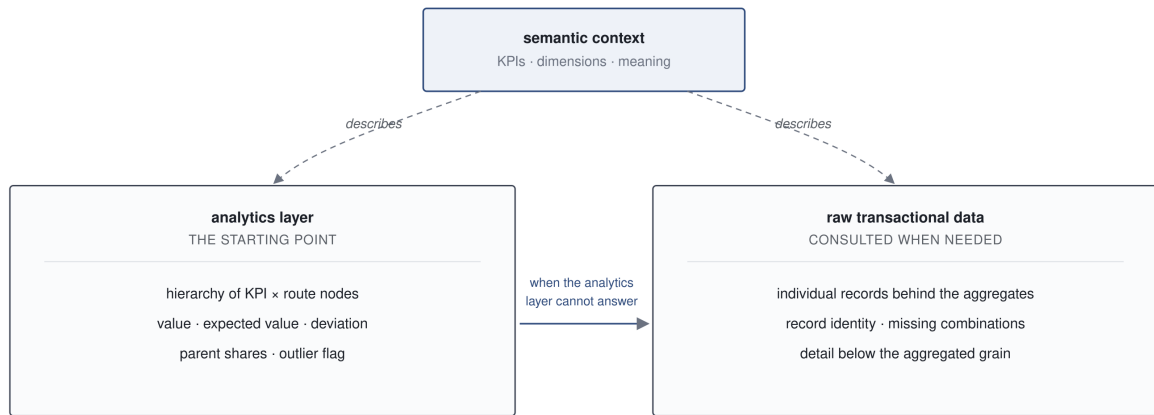


Figure 4.1 The two data tiers and the semantic context that describes both. The analytics layer is the starting point of every analysis; the raw tier is consulted only when the layer cannot answer a question.

of Table 4.1, so the hierarchy is present in the data itself and does not have to be reconstructed. The same detailed slice can often be reached over several routes. This property matters later, because it is the reason the same deviation can appear in several places at once (Section 4.2.1). Figure 4.2 shows the route hierarchy of the analytics layer used in this work.

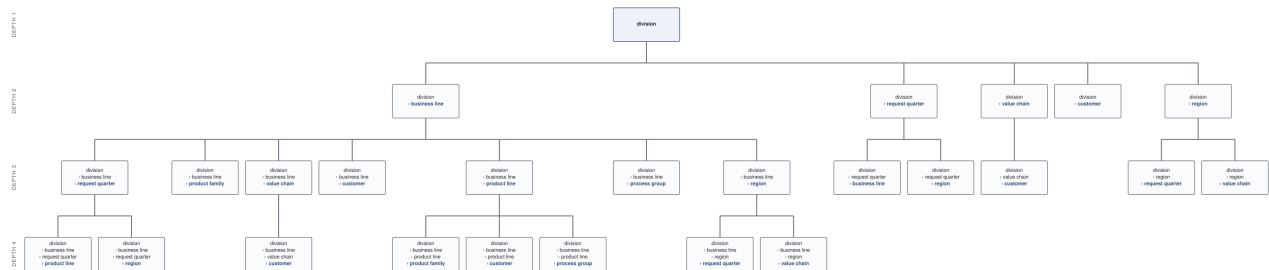


Figure 4.2 The unified route hierarchy of the analytics layer in this work: 26 routes over depths one to four. Routes rooted at a nested dimension are merged into their parent-prefixed twins, the same unification the pipeline applies before drilling. A larger landscape version is provided in Appendix C.

Table 4.1 lists the fields of a node. Two conventions from this set are used throughout the work. The deviation is the difference between the value and the expected value, and its sign is the direction of the change. The upper and lower bound frame the range the node's values normally stay inside, learned by the detection system from the node's recent history. How the expected values, the bounds, and the flags are produced is outside the scope of this thesis.

The measure and the reporting period are not fixed by the method. Both come from the semantic context of the organization. In this work, the measure is order value in euros, and the reporting period is one week.

4.1.2 The Semantic Context

The semantic context stores what the system knows about the dataset. Both tasks use it, and dataset-specific knowledge enters the system only through this context. Its structure is adapted from the ktx [30] context layer introduced in Chapter 3 and extended for this setting. This section explains what the context contains. Section 4.1.3 explains how it is built.

Table 4.1 Fields of one node of the analytics layer.

Field	Meaning
Analytics route	The ordered dimensions the node was built along
Series name	The concrete value of each route dimension, in the same order
Upper series	The parent node one level up; the hierarchy link
Value	The aggregated measure for the reporting period
Expected value	The normal level of the node, provided by the detection system
Upper and lower bound	The range the node's values normally stay inside
Deviation	The value minus the expected value; its sign is the direction of the deviation
Value share	The portion of the parent value this node represents
Deviation share	The portion of the parent deviation this node carries
Outlier flag	Marks the node as flagged by the detection system

The context is organized around the information needed to answer a question. The table and column entries explain what each table can answer and what each column means. They also record the available categorical values and how terms that match several columns should be resolved. The time entries describe the role of each date column, the meaning of the weekly labels, and the organization's fiscal calendar.

The business knowledge wiki stores the language used in business questions. It includes common abbreviations, approved meanings of business terms, and notes for expressions that can have more than one reading. Application parameters store the calibration values used by the summary pipeline. The table and column entries and the wiki build on the two outputs of the ktx ingest. Time information and application parameters were added for this work. Figure 4.3 shows the resulting structure.

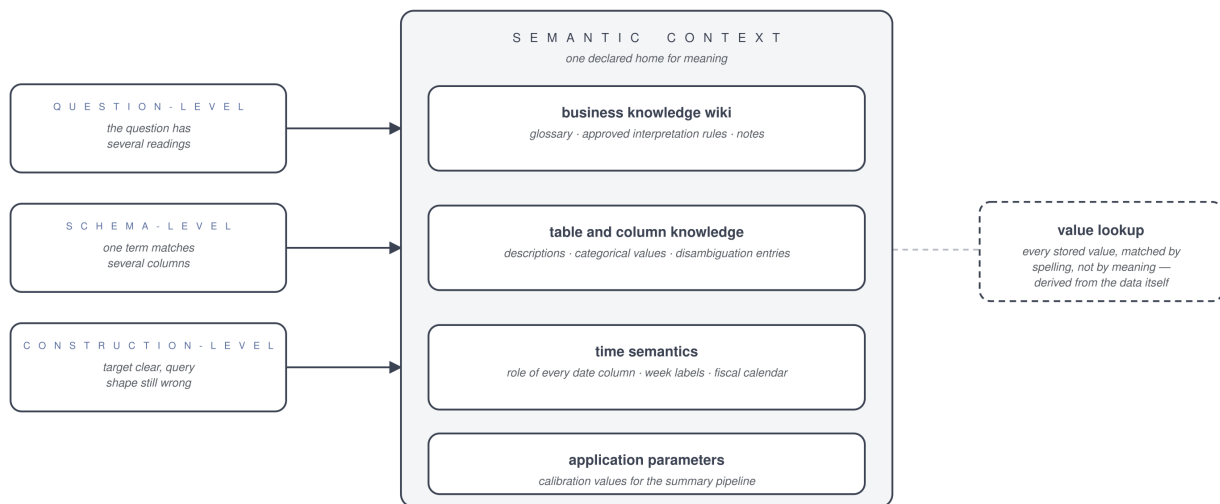


Figure 4.3 The semantic context and the three ambiguity classes. Each class of ambiguity has a declared home inside the context; the value lookup stands beside the context because it is derived from the data itself rather than curated.

The stock ktx runtime searches its context at query time with combined full-text and semantic search, because the context of a large warehouse does not fit into the model's context window. In this work, the warehouse of Section 4.1.1 has two table types. The conversational steps receive semantic definitions and approved rules directly, without a separate semantic-search stage. Retrieval could omit the rule that determines an interpretation, without being necessary to fit this context. Value grounding remains necessary. A question says a business name, and the query needs the exact stored spelling. A value lookup provides

names from eligible warehouse columns, ranked by spelling similarity rather than business meaning. It is derived from the data, not curated knowledge, and can be rebuilt after a refresh. Column eligibility and size limits bound its coverage.

The structure is shaped by ambiguity. During the construction of the context, three classes of ambiguity appeared, and each received its own home.

Question-level ambiguity. The question itself has several readings.

How many orders did we receive last week? — orders can mean the confirmed order value or the value including unconfirmed orders. Both are correct readings, and the numbers differ.

The business knowledge wiki answers this class with a default and a note that discloses the choice.

Schema-level ambiguity. One term matches several columns.

What did customer A order? — customer A can be the main customer of the order or the party the order was sold to. The warehouse stores both, in two different columns.

The disambiguation entries answer this class, and they sit on both rival columns so that either path meets the warning.

Construction-level ambiguity. Question and target are clear, but the query shape can still be wrong.

How did the last quarter go? — the declared fiscal calendar determines the boundaries. An exact date interval and a sum of whole reporting weeks can cover different records, so the chosen basis must match the request.

The time semantics and the interpretation rules answer this class.

The classes follow the sources of ambiguity that Somayajula et al. [21] describe. The extension of this work over the ktx structure is that each class receives a declared home and a solution inside the context. Each home is a small adaptation introduced for its class: the glossary and the approved rules, the disambiguation entries, and the declared time semantics. Section 4.1.3 shows how they are filled, and their implementation follows in Chapter 5. The policy is to use a declared interpretation or ask the clarifying question of Section 4.3.5. The evaluation checks whether the agents follow this policy.

4.1.3 Semantic-Context Construction

The semantic context must be buildable again. For a new organization, the promise of the chapter's second principle only holds if the context can be constructed there without repeating this work by hand. The construction was therefore designed to be automated end to end, and that goal did not survive contact with the data: a generated description can read well and still mean the wrong thing. What proved automatable is what the data itself can verify. The construction therefore splits the work. Automated stages draft the context and measure it, and a human decides what becomes truth. This is where the method takes its human-in-the-loop form. Figure 4.4 shows the pipeline.

The first stage drafts the context. It is a credited Python port of the ktx ingest introduced in Chapter 3: it connects to the warehouse, samples the tables, drafts the table and column descriptions, and absorbs the organization's existing documents into the wiki. The extensions of this work write into the same draft: category typing for the columns, the time blocks, and the disambiguation entries. The output of this stage is a draft that requires checking.

The second stage checks time semantics through data probes. These compare candidate interpretations of date columns and reporting windows with recorded values. The context retains the supported rule and its evidence. Business meanings cannot always be resolved by numerical reconciliation, so the remaining stages combine model-assisted challenge with expert review.

The third stage challenges the draft with model-generated questions organized by the ambiguity classes of Section 4.1.2. The construction loop uses model assessment to identify possible interpretation problems from the questions, query plans, and returned results. Proposed corrections are tested in a draft

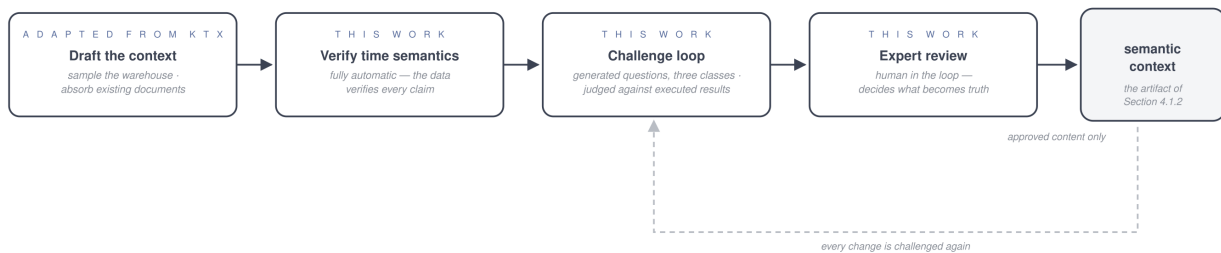


Figure 4.4 The construction of the semantic context. The drafting stage is adapted from the ktx ingest; time verification, the challenge loop, and the expert review were added in this work. The procedure submits proposed changes for challenge and expert approval before adoption.

context. An expert reviews the revised context before publication, so model suggestions do not enter the approved context directly. This assessment belongs to semantic-context construction, not to the answer delivery of the conversational ladder. Chapter 7 assesses answers against reference queries and recorded observations. The three ambiguity classes were not invented in advance; they are how the failures of this loop grouped themselves.

The last stage reviews the complete draft against the previously approved context. Expert approval applies to the context as a whole, and the runtime loader requires its approval marker. The marker is a loading condition, not proof that every stored interpretation is correct.

The intended lifecycle returns corrections to the builder and challenge stage before adoption. Observations from Section 4.3.5 and proposed promotions from Section 4.3.7 follow the same review policy. Chapter 7 distinguishes the retained construction evidence from an unmeasured before-and-after improvement.

The shared foundation supports the decisions in both tasks. The summary pipeline uses the node values and route information to compare deviations at different levels of the hierarchy. Dimension definitions identify how a slice can be divided, while KPI definitions describe the components of an aggregated measure. The pipeline also reads its working thresholds from the calibration block.

The conversational system uses these dimension and KPI definitions to select tools and construct queries. Approved business rules help it interpret the question, and the value lookup provides the stored forms of business names. Both tasks therefore use a common source for the meaning of the data. Updates to a shared definition can affect both the weekly summary and answers to later questions.

4.2 Executive-Summary Pipeline

The executive-summary pipeline turns one week's flagged KPI deviations into a short report for one or more divisions. It selects findings for executive review and reports the movement outside the selected stories. The accounting guard accepts a plan only when the stories, counters, context, and remainder reconcile with the scope's total deviation.

The main running example uses the constructed Week A records from Section 6.3. Division Example has a net movement of +61.6M.

The rest of this section follows the pipeline from its input to the final document. It begins by defining the expected output and then filters and drills through the analytics layer to collect candidate findings. These candidates are compared on raw records, and duplicate views of the same effect are merged. The system then calculates impact evidence, and a bounded agent proposes the story plan. Deterministic tools check record ownership and accounting before each selected story is explained through its representative's

records and composed into the summary. The pipeline follows the routes described in Section 4.1.1 and compares the deviations stored at their nodes. The semantic context supplies the dimension and KPI definitions used during this analysis. The following stages explain how these definitions and the working parameters guide candidate collection, comparison, and story selection. Figure 4.5 gives an overview of this process, the data tier used at each stage, and the semantic context shared across the stages.

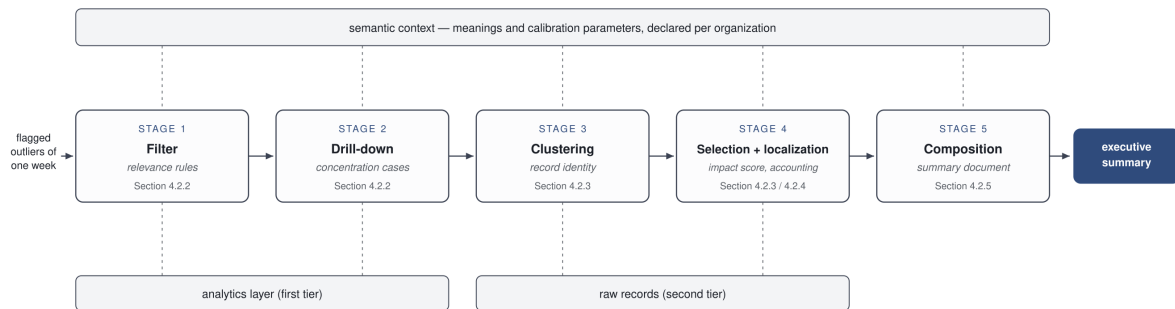


Figure 4.5 The executive-summary pipeline from flagged outliers to the final document. Initial analysis uses the analytics layer, verification and selection use raw records, and the semantic context supplies meanings and calibration parameters throughout the process.

The pipeline follows a simple reporting rule. The summary should report as little as possible, but as much as necessary. The pipeline therefore prefers the most aggregated level that still represents a deviation faithfully, and it drills deeper only where the data concentrates. How each stage applies this rule is made precise in the following sections.

4.2.1 Problem Setting

The analytics layer flags outliers at single nodes of the hierarchy. Each flagged node reports a deviation, the value minus the expected value from Table 4.1, and the sign of the deviation is its direction. The deviation of the top node of the scope is the total deviation that the summary must explain. In this work, this is the weekly order deviation of a division.

Every node carries two numbers, its value and its deviation (Section 4.1.1). The pipeline reasons mostly on deviations, because the summary explains change rather than size. Values are used where size matters, for example to judge whether a deviation is large relative to the size of its slice.

The unit of the summary is a story. A story is one underlying deviation, reported at the level where an executive can act on it, together with its amount and its explanation.

Turning flagged outliers into stories raises four challenges. First, most flagged nodes are not worth reporting. A week can contain many flags, and most of them are small, duplicated, or routine. Second, the same deviation appears on many routes at the same time. In this work, for example, a single customer's increase raises the customer route, the business line route, and their combinations together. Counting these as separate findings would report the same value several times. Third, the right reporting level differs per branch and per week. Sometimes one child explains the whole parent deviation, sometimes a few children carry it together, and sometimes it is spread thin across many children. A fixed rule that always reports the same level cannot handle all three situations. Fourth, the analysis visits many nodes, and its findings must be recorded, or later steps lose the context that earlier steps discovered.

How the outliers are flagged is outside the scope of this thesis. The pipeline starts from the flags that the analytics layer already contains.

4.2.2 Filtering and Concentration-Based Drill-Down

The initial scan identifies the reporting week and KPI from the flagged rows. The drill reads the hierarchy directly and applies the relevance floor and placeholder rules as it visits nodes. Surprise supports the agent's review of a decision, but does not act as a rejection threshold.

The relevance floor is the larger of a fixed minimum amount and a share of the absolute scope deviation. When the net deviation is small, the fixed minimum determines the floor. Large increases and decreases can still occur within a scope whose total is flat, and a zero net deviation does not stop the search for material movements in either direction.

When the scope's own movement lies below the fixed minimum, its sign is not a meaningful reference. The case rule then measures each child against the gross movement of all children, records no counter, and the offsetting movements become findings with their own signs. Later stages assign no counter role by the scope's sign either, and the summary states that the total is flat because large movements offset each other. Stronger cancellation above this line is handled by the ordinary rule, with the opposing movement recorded as a counter.

For the selected week and KPI, the drill starts at the top of the hierarchy and moves down one level at a time. The agent makes one drill decision per node and dimension, each in a fresh and independent episode. A drill decision states whether the finding should stay at the current level or continue to one or more children. The stage returns route-specific candidates, protected counters, and observations for clustering.

The case tool distinguishes three cases, and Figure 4.6 shows them side by side.

In case A, one child carries most of the parent deviation. In Figure 4.6, the division moves by +100M, and a single customer carries +86M of it. The drill follows that child and repeats the decision one level deeper, where one product line carries +82M of the customer's deviation. The finding is the node where the concentration stops, at whatever depth that happens.

In case B, no child dominates, but a few children each carry a material share, and together they carry most of the deviation. In the figure, three customers carry +42M, +40M, and +38M of a +120M division deviation. The drill picks those children as findings. Reporting the parent instead would hide that the deviation has a small number of concrete carriers.

In case C, the deviation is spread thin across many small children. In the figure, the largest of six customers carries +13M of a +100M division deviation. No child is worth reporting alone, so the parent itself is the finding. Drilling deeper would replace one clear number with many small ones that say less.

Deviations carry a direction. A positive deviation means the value is above its expected level, and a negative deviation means it is below. The three cases judge the children that move in the same direction as their parent. A child can also move against that direction. In Figure 4.6, the division rises while one customer falls. Such counters are recorded in every case when their share is material, and they are protected. They cannot be dropped in later stages, because a summary that hides an opposing deviation would overstate the net picture.

Outside the flat-parent case, the share of a child is its deviation divided by the deviation of its parent:

$$\text{share}(\text{child}) = \frac{\text{deviation}(\text{child})}{\text{deviation}(\text{parent})}$$

Case A applies when the largest child's share reaches the leader threshold of 0.6. Case B names children whose shares reach 0.05 when their combined share reaches 0.5. A candidate that later turns out to be unimportant is cheap, while a deviation that is never collected is lost. The low share rule therefore favors collecting candidates, while the combined-share requirement keeps case B selective. An opposing child is recorded as a counter when the magnitude of its share reaches 0.2. All four values are parameters from the semantic context, not constants of the method. They were set by review informed by the calibration week's share and deviation distributions. Near the leader threshold, the agent may reconsider the recommended action within the permitted rules. A margin of five percentage points flags nearby top-child shares for closer inspection. It is a working tolerance, not a statistical uncertainty interval. The thresholds

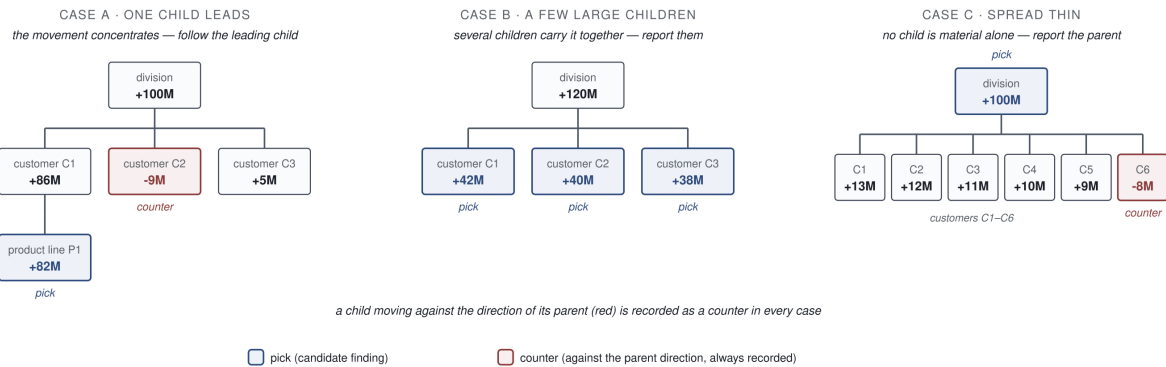


Figure 4.6 The three concentration cases side by side. Picks are shown in blue; a child moving against the direction of its parent (red) is recorded as a counter in every case.

remain fixed even when an exception is accepted. Table 5.4 records their working values. The controlled evaluation tests decisions near these boundaries, not an end-to-end sensitivity sweep.

$$\text{lift}(\text{child}) = \frac{\text{deviation_share}(\text{child})}{\text{value_share}(\text{child})}$$

A child has high lift when it carries more of the deviation than its relative size would suggest. The second signal is surprise, which compares the child's deviation with the width of its normal band:

$$\text{surprise}(\text{child}) = \frac{|\text{deviation}(\text{child})|}{\text{upper_bound}(\text{child}) - \text{lower_bound}(\text{child})}$$

A high surprise value means that the movement is unusual for this analytics node. Lift and surprise remain separate evidence and do not replace the concentration cases.

To place these signals in context, tools calculate their median and 90th percentile over a recent window of comparable nodes. The reference uses the same KPI and reporting route and states the observation count and period. These statistics support interpretation but introduce no new selection threshold. If the reference is unavailable, the agent uses the original signals and guards.

The drill returns route-specific candidates. Section 4.2.3 checks which candidates describe the same effect before story selection in Section 4.2.4.

The agent makes one drill decision for each node and dimension examined. A decision may require several model calls. It receives the children with their deviations, shares, lift, and surprise. The case tool applies the rules above and recommends an action from Table 4.2. Lift and surprise support the agent's review of the recommendation. The tools perform all calculations.

Table 4.2 The actions of a drill decision. The names are used in the case descriptions above and in the walk.

Action	Effect on the drill
Follow one child	Continue the analysis below the selected child to find a more detailed reporting level.
Pick children	Record the selected children as candidates. The walk can continue below them to collect supporting detail.
Stop	End exploration along this dimension. If no other dimension continues, retain the current node as a candidate when it meets the relevance floor.

The agent normally accepts the recommendation. When it chooses another action, it must give a reason and use only actions and targets permitted by the tools. The available actions depend on the children and the relevance floor. Time dimensions are excluded from the drill. On a flat parent the case rule uses the

gross movement of the children, as described above. If the agent cannot complete a valid decision, the system follows the deterministic recommendation. The run records the proposed action, the action taken, the reason, and any fallback. The drill ends when its queue is empty or an execution limit is reached. The implementation details follow in Chapter 5.

Two execution limits bound the walk. The working configuration uses 100 agent decisions and 900 seconds for each division, KPI, and week. Every node and dimension examined counts as a decision, including dimensions with only one admissible action. These are configurable working allowances, not optimal values derived for every possible hierarchy. They are checked between nodes, so the current node finishes before the walk stops. Queued nodes above the relevance floor remain provisional candidates at their current level, although finer findings may remain undiscovered. Chapter 7 reports actual workloads and stopping outcomes.

The drill agent considers shares, lift, surprise, and business meaning together when reviewing a recommendation near a decision boundary. It can propose a different reporting level within the permitted actions and record observations for later story selection. This allows contextual judgment without defining a separate decision rule for every combination of signals. Tools continue to calculate the evidence and enforce the constraints.

Thresholds are hard lines, and a share just below a threshold is not meaningfully different from a share just above it. Suppose the largest child of a +100M parent carries +58M. Its share of 0.58 narrowly misses the leader threshold of 0.60, so the rule does not follow it. However, its deviation share is three times its value share, and its deviation is four times the width of its normal band. The walk marks such a case for closer inspection. With this evidence, the agent can keep the computed verdict or follow the child through a logged exception if the action passes the guards. The threshold itself does not change. The review can also consider an excluded child near the naming threshold, provided its amount clears the relevance floor and the proposed action passes the guards.

A decision may also include a short note, such as a possible story, a possible duplicate route, or something to verify on the raw data. These notes are hypotheses rather than verified findings. They travel with the candidates so that later stages retain the observations from the walk.

Week A has a relevance floor of 3.08M. Figure 4.7 shows twelve candidate views prescribed for the illustration. They include overlapping customer, business-line, product-family, and value-chain views, with Nova and Lumen as opposing movements. The next stage measures which findings can be combined.

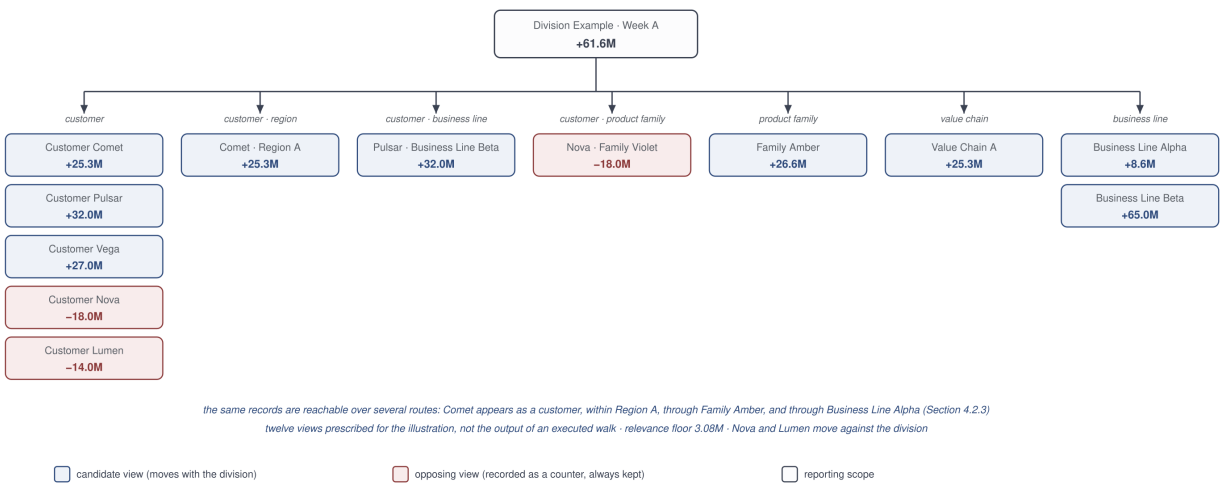


Figure 4.7 Prescribed candidate views for constructed Week A, including overlapping views and opposing movements. Their pair measurements determine the clusters in Figure 4.8.

4.2.3 Same-Effect Clustering

Candidates can be found through different drill-down routes even when they represent the same movement. For every candidate pair, the system identifies their shared raw records and measures the deviation on those records. It divides this shared deviation by the deviation of each candidate. The candidates merge only when both ratio magnitudes reach 0.95 and their directions agree. Otherwise, they remain separate, while asymmetric ratios can still show containment or partial overlap. A final cluster contains only candidates for which every pair passes the merge rule.

The stage uses the selected candidates from the drill-down, including the recorded counters. Their route information and agent notes remain attached as supporting metadata, but they do not decide which candidates merge. As Section 4.1.1 showed, the same slice of data can be reached through different routes. In Week A, Customer Comet, Comet within Region A, Product Family Amber, and Business Line Alpha form one cluster. All six internal pairs pass the merge rule despite their different descriptions. The record measurement, not the entity name, decides.

Let A and B denote two candidates. Their intersection, $A \cap B$, contains only the raw records that belong to both candidates. The deviations of A , B , and their intersection are measured consistently using the allocation described below. The method then calculates two net-contribution ratios:

$$\begin{aligned}\text{net_ratio}(A \text{ in } B) &= \frac{\text{deviation}(A \cap B)}{\text{deviation}(A)} \\ \text{net_ratio}(B \text{ in } A) &= \frac{\text{deviation}(A \cap B)}{\text{deviation}(B)}\end{aligned}$$

In Week A, candidate A is Customer Pulsar with a measured deviation of +32.0M, and candidate B is Business Line Beta with +65.0M. Their shared records contribute +32.0M. The ratios are therefore $32 / 32 = 1.00$ and $32 / 65 = 0.49$. Pulsar is contained in Beta, but the two views remain separate because they do not both reach the merge threshold.

These ratios measure net contribution and are not bounded percentages. A ratio can exceed 1 when the shared records contribute more than the candidate's final deviation because records outside the intersection move in the opposite direction. The sign is also preserved. The pair measurement therefore records whether the shared records move with both candidates, against one of them, or in a mixed way. The automatic merge decision uses the two net-contribution ratios and this direction relation. The pipeline also computes bounded directional coverage, but records it only for analysis and does not use it for automatic merging.

Ratios are computed only for candidates that passed the relevance floor. If either candidate has zero measured deviation, the pair remains separate and its net-contribution relation is recorded as not measurable. The support relation compares the magnitudes of the two ratios with the thresholds, while the direction relation reads their signs separately, so a negative ratio never passes a threshold by its magnitude alone.

The current value of the shared slice is measured directly from this week's raw records. However, the analytics layer may not provide an expected value for this slice. The pipeline therefore measures the slice's historical share within the nearest available analytics node and uses this share to allocate the node's stored expected value. For example, if one product historically represents 60 percent of a customer and the customer has an expected value of 100M, the allocated expected value of the customer-product slice is 60M. The slice deviation is its current value minus this allocated expected value. The expected value used for the allocation is the one the analytics layer stores for the nearest analytics ancestor of the slices being compared, and the historical share is measured over the weeks before the reporting week. The historical share is measured, but the allocation assumes that the historical mix represents the normal mix. When a slice has no recorded history in the available extract, the current fallback assigns it an expected value of zero, so its full current value counts as deviation. This assumption may overstate the deviation when earlier activity is missing from the extract. Validation of this allocation model and the effect of missing

history remain future work. The implemented checks assess consistency of the resulting assignments and reported amounts.

The magnitudes of the two ratios determine the support relation. Every measurable pair belongs to one of the relations summarized in Table 4.3.

Table 4.3 The support relations between two candidates, decided by the magnitudes of their net-contribution ratios.

Support relation	Ratio magnitudes	Result
Strict equivalent	Both are at least 0.95	May merge
Ambiguous equivalent	Both are at least 0.85, but not both reach 0.95	Keep separate
Containment	Only one reaches 0.85	Record containment
Partial overlap	Neither reaches 0.85, but at least one reaches 0.20	Record overlap
Disjoint	Both remain below 0.20	Record as separate

The 0.95 threshold provides the strict merge rule. The 0.85 and 0.20 thresholds distinguish containment, partial overlap, and disjoint pairs that remain separate. The three thresholds are initial design choices. The strict value of 0.95 requires near-complete agreement of the two ratios before two findings are combined, because a false merge hides a finding while a missed merge only leaves a duplicate for the story seat to arrange. Their working values are listed in Table 5.4. They are not claimed to be empirically optimal. When several candidates are connected by merge decisions, the system checks every pair before forming the final cluster. This prevents a chain in which A matches B and B matches C from merging A with C when their own pair does not match. After the cluster exists, its widest member, meaning the view with the fewest dimensions, becomes its representative. This choice gives the cluster its most aggregated description but does not affect its membership. The stage returns the final clusters and the recorded pairwise relations for scoring and story selection in Section 4.2.4.

Figure 4.8 shows the Week A grouping after all 66 candidate pairs have been measured. The twelve views form seven clusters. Three clusters contain several views, while four remain single views. Value Chain A remains outside the Comet cluster because its ratios with Comet are both approximately 0.9486, just below the merge threshold. Comet is the prescribed representative among equally wide views in this illustration.

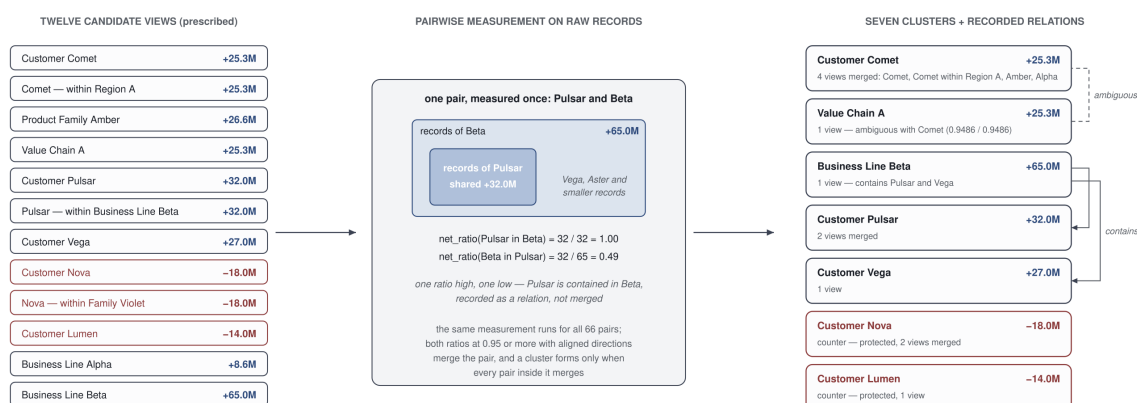


Figure 4.8 Same-effect clustering for constructed Week A. Twelve prescribed candidate views form seven clusters. Every pair within a cluster passes the merge rule, while relations between separate findings remain available for story selection.

4.2.4 Impact Scoring and Story Selection

Clustering removes duplicate views, but different clusters can still contain or overlap each other. The recorded relations show how the effects intersect, but they do not decide which effect should lead the executive storyline. This stage combines deterministic impact evidence with a bounded agent decision. Tools calculate the evidence, the agent proposes the story plan, and deterministic guards check the result.

The candidates inside a cluster are not added, because they may describe the same records through different drill-down routes. The system calculates the cluster's current value and deviation from the records of its representative, counting each record once. It then calculates one impact score for the cluster.

$$\text{impact}(\text{cluster}) = |\text{deviation}| \times \text{lift}^{0.3} \times \text{cluster_share} \times \text{depth_discount}$$

The absolute deviation keeps the amount of movement as the main factor. Lift asks whether the cluster carries more of the scope's deviation than its relative size would suggest. Outside the flat-scope case, it is calculated in three steps:

$$\begin{aligned} \text{deviation_share}(\text{cluster}) &= \frac{|\text{deviation}(\text{cluster})|}{|\text{deviation}(\text{scope})|} \\ \text{value_share}(\text{cluster}) &= \frac{|\text{current_value}(\text{cluster})|}{|\text{current_value}(\text{scope})|} \\ \text{lift}(\text{cluster}) &= \frac{\text{deviation_share}(\text{cluster})}{\text{value_share}(\text{cluster})} \end{aligned}$$

In Week A, Division Example has a current value of 275.0M and a deviation of +61.6M. Pulsar has a current value of 62.0M and a deviation of +32.0M. Its value share is 62 / 275 and its deviation share is 32 / 61.6, giving a lift of about 2.30. This value is measured on the representative's records. The earlier lift values of its individual candidates are not reused. When the value share is zero, lift is set to one. Lift is also set to one when the scope is flat under the rule of Section 4.2.2, because its net deviation is too small to serve as the reference. The depth discount is applied once for each dimension beyond the first, so clusters at the same depth receive the same depth factor.

Cluster share asks a different question: whether a broad cluster adds information beyond the distinct clusters inside it. When one or a few contained clusters explain most of its movement, only the remaining share contributes to the broad cluster's score. If the movement is spread across many small parts, the cluster share remains 1 and the broad cluster can still represent the effect. In Week A, Beta moves by +65.0M, while the contained Pulsar and Vega clusters contribute +32.0M and +27.0M. Their combined +59.0M leaves Beta with a cluster share of 6 / 65, about 0.092. The depth discount then gives a small preference to a more aggregated description when two clusters otherwise have similar evidence. In the working calibration, each additional dimension multiplies the score by 0.85, and lift enters with the exponent 0.3; both are design choices intended to moderate lift and favor broader descriptions. The exponent reduces lift's influence but does not bound it, and these values are not empirically optimal.

The impact score gives the agent a common starting point, but does not fully determine the storyline. Several related clusters may support different ways of presenting the week. The agent considers their scores, measured relations, localization evidence, and business context when choosing which findings lead and which provide supporting detail. It may select a lower-scoring finding, but must record its reason. Deterministic placement provides a fixed-rule alternative. Using an agent allows these presentation choices to be proposed from the combined evidence, while tools retain control over the measurements and accounting.

The agent receives the clustered findings, their impact evidence, their containment and overlap relations, a one-level preview of where each cluster's movement sits, computed by the localization method of Section 4.2.5, the protected counters, the observations recorded during the drill, and a story budget from the semantic context. The preview lets the agent tell a concentrated cluster from a diffuse one when it

Table 4.4 The actions of story selection. A relation remains a measured fact: the agent can decide how to present it, but it cannot declare a relation that the tools did not find. Every action includes a short reason and is written to the run trace. A rejected action leaves the plan unchanged. The agent receives the reason and may try again within the remaining planning-action budget. The trace records the proposed action, whether it was accepted, and the results of the guard checks.

Action	Effect on the plan
Select a story	The cluster becomes a leading story and consumes one unit of the story budget.
Attach a branch	The cluster joins a selected story as inside detail. It must be a measured refinement of that story. Its deviation stays measured, but it receives no separate amount in the summary's accounting.
Attach context	The cluster is shown under a selected story it contains or overlaps. It receives only the movement not already assigned to leading stories.
Protect a counter	An opposing cluster is kept visible. It never consumes the budget and can never be dropped.
Select a remainder movement	A currently eligible tool-listed remainder movement becomes a leading story. Non-opposing movements consume the story budget. On a flat scope, either sign consumes it.
Leave in remainder	A non-counter cluster is not presented and receives no ownership. Its unowned movement remains in the disclosed remainder.
Finalize	The plan is closed. It is refused until every cluster has a recorded decision.

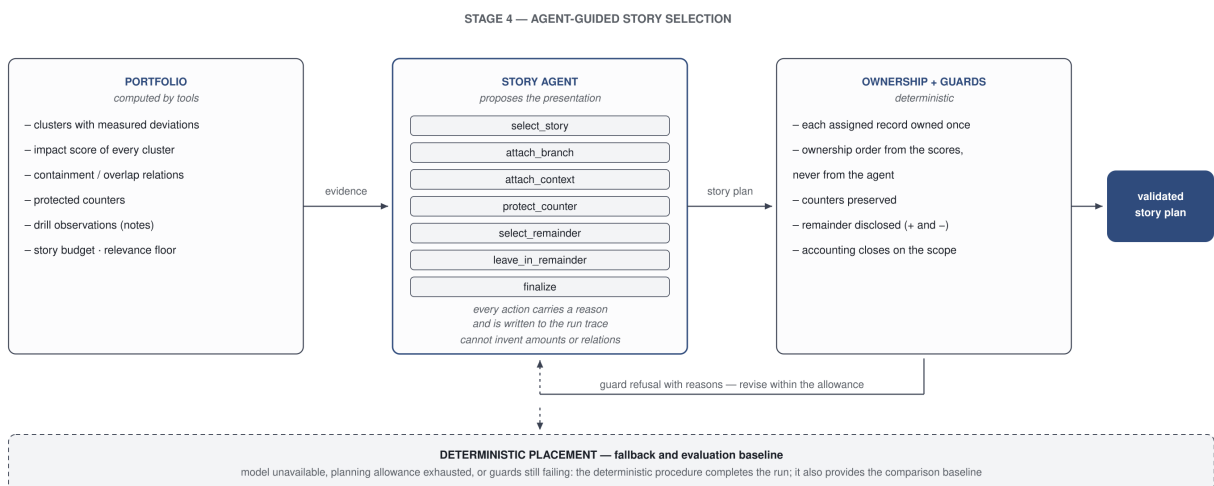


Figure 4.9 Agent-guided story selection: tools compute the portfolio, the agent proposes through typed actions, deterministic guards validate; deterministic placement stays the fallback and baseline.

assigns the roles; like the score, it is evidence and not an amount. The budget limits the number of leading contributor stories. The agent proposes the story plan through the typed actions of Table 4.4.

For context attachment based on overlap, the shared net deviation must reach 0.5 of the proposed context cluster's net deviation in magnitude. This working cutoff requires substantial shared movement, while measured containment can qualify separately. The agent still chooses whether to attach the cluster.

After the agent proposes a story plan, a deterministic ownership tool assigns the records in a fixed order based on the roles and the impact evidence. The agent chooses the story roles, but it does not control the ownership order or assign any amount. Each raw record can belong to at most one primary story. The summary reports the deviation of the records assigned to each story, which can differ from its measured cluster deviation. Branches do not receive an independent amount, and context receives only the movement that is not already owned by a story. In the prescribed Week A plan, Pulsar and Vega receive +32.0M and +27.0M. Beta receives the remaining +6.0M as context, rather than adding its full +65.0M again.

The ownership tool also scans the unowned records. A remainder movement is material when its absolute deviation reaches the relevance floor from the semantic context. The agent may select only a movement returned by this tool and only while the story budget allows it. Anything not selected remains disclosed with its positive and negative parts, so cancellation cannot hide what was not chosen as a story. The accounting guard then checks that the stories, counters, context movements, and remainder add up to the total deviation of the scope. A plan that breaks this equality is rejected.

The agent records a decision for every cluster through the permitted actions. A bounded execution allowance prevents repeated unsuccessful proposals from continuing indefinitely. If the model fails, the allowance is exhausted without a final plan, or the final plan fails the accounting or counter checks, deterministic placement completes the run. The trace records when this fallback is used. This procedure is also kept as the baseline for evaluation. The output of the stage is a validated story plan with record ownership, context, protected counters, and a disclosed remainder. Section 4.2.5 explains the internal structure of its selected stories.

The thresholds of the pipeline serve different purposes. The drill favors collecting candidates, while clustering requires stronger agreement before combining findings. Near selected drill boundaries, the agent may reconsider the recommended action within the permitted rules. Other thresholds remain deterministic. Most working settings can be configured for another dataset, although placement retains code constants and defaults identified in Section 5.3. This flexibility does not establish that the selected values are optimal. An end-to-end sensitivity study remains necessary to assess their effect on report selection.

4.2.5 Story Localization

Localization examines the representative's full series. When the story owns only part of those records, the explanation is marked as evidence about the broader series. The downstream composition stage calls the deterministic localization routine for every selected story. The method is explained separately here because its measurements provide the evidence used during composition.

Inside the representative's records, the same concentration logic of Section 4.2.2 runs again, now over the dimensions that the story's description does not use yet. For every such dimension, the records show where the deviation sits. When one value carries most of it, the explanation names that value and continues one level deeper inside it. When a few values carry it together, the explanation reports the split with the amount of each carrier. When the deviation is spread, the explanation stops at this dimension. Localization uses its own thresholds, listed in Table 5.4.

The working tiers use 0.95 for a location and 0.60 for a lead when no location is found. In a shared split, each named contributor must reach 0.25 and pass the amount floor, with at most three named. These are reporting conventions for near-total concentration, a leading contribution, and shared movement. Wording a route as "almost entirely in" additionally requires 0.95 directional coverage. To limit detail, recursion stops at depth three or below the larger of one million and 0.10 of the absolute owned amount. The depth limit bounds the explanation, while the floor keeps its detail material.

Each net-contribution ratio compares a child's deviation with that of its immediate parent slice, not necessarily the whole story. The wording follows the measurement. Every explanation line carries the measured share it stands on, and the language is chosen by that share, so a line does not claim more than the records support. A deviation that sits at 93 percent in one product family is worded differently from one that sits at 60 percent, and both lines show their numbers. The analysis thresholds and localization limits are supplied by the semantic context. Rendering constants remain implementation settings.

In Week A, Beta's full series moves by +65.0M. Value Chain C carries +59.0M, split between Pulsar at +32.0M and Vega at +27.0M. Pulsar's own localization stops at the process-group level, where thirty groups contribute equally and no single group is named.

The honest stop is a finding, not a failure. When a story's deviation is spread across many values of every remaining dimension, the explanation says that the deviation is broad. For an executive reader, a genuinely broad deviation is different news than a concentrated one, and pretending to have found a driver would be the only wrong answer.

Every line of an explanation is traceable. It names the records it was measured on, so a claim in the summary can be followed back to the data. The notes the agent wrote during the walk are carried into the audit part of the report as the agent's observations. They do not enter the explanation, which contains only measured lines.

The result of this stage is, for each story, a small tree of measured explanation lines with amounts and shares. Composing these skeletons into readable text is the task of the next section.

4.2.6 Composition and Output

Composition turns the validated story plan into the final summary. It first calls the localization method of Section 4.2.5 for each selected story and then arranges the returned findings. It does not reopen the story roles or calculate new business values. Every amount, relation, and localization line comes from the tools.

The deterministic composer provides the complete baseline. It places the scope movement in the headline, presents the selected stories with their owned amounts and localization lines, and keeps counters visible. A closing line reports the explained share, context movements, and the positive and negative remainder. Scores, warnings, and reconciliation details remain in the audit output instead of the executive text. The same measured input therefore produces the same summary, without another model call.

The guarded extension gives an agent the same measured findings through a restricted information contract. The agent receives business findings, not database field names. The validated plan from Section 4.2.4 determines the leading story and the placement of its branches and context. The phrasing extension can improve how these findings are expressed. It must preserve their arrangement and measured content, and it may not introduce an amount, change a relation, remove a protected counter, or hide the remainder. Numbers remain references to tool outputs rather than text generated by the model.

Before publication, deterministic guards compare the composed output with the validated story plan. They check the required stories and counters, the reported amounts, the disclosed remainder, and the accounting closure. If the model is unavailable or its output fails a guard, the deterministic composer produces the final document. The original template remains available for checking the reworded text. Readability improvement was not assessed in this thesis.

The pipeline produces a weekly summary with selected stories, their assigned amounts, and supporting explanations. It also reports counters, context, and the remaining movement. The summary and its supporting record are written to Shared Memory, where the conversational system of Section 4.3 can read them.

The evaluation checks reporting arithmetic, ownership, analytical routes, and controlled agent decisions. It records the story selection and remaining movement without treating signed coverage as a quality score. The source of each story is recorded to distinguish findings collected during the drill from those selected during the remainder scan. Agent-guided plans are compared with deterministic placement. This comparison records proposed and executed actions, guard outcomes, and fallback use separately.

4.3 Context-Grounded Conversational Agent

In addition to the weekly summary, this thesis develops a Context-Grounded Conversational Agent for top-level management. It answers questions over the same data foundation in business language. The conversational system shares the semantic context and data access with the weekly pipeline and additionally uses the value lookup to ground names. The weekly summary presents selected findings, while the conversational system lets top-level management ask questions about them and explore their supporting data. The summary is provided to the agent as memory to support these follow-up questions. The conversational system also answers questions about the underlying data beyond the findings included in the summary. The system follows the same design principles as the summary pipeline. Deterministic tools perform calculations and data access, while the model interprets questions and selects among the actions allowed within each step. The Ladder Orchestrator controls how each question moves through the system. Dataset-specific knowledge comes from the approved semantic context. The design requires reported numbers to be traceable to a tool result or an executed query. The evaluation checks the delivered response against that requirement.

The previous section followed one week through the summary pipeline. This section follows one question from its arrival to the final answer. Figure 4.10 shows the escalation ladder. The Ladder Orchestrator first checks whether a predefined analytical tool matches the question. If no tool fits or its result is insufficient, the question moves to the Open Analysis Step. This step continues with the available evidence and constructs guarded read-only queries when needed. Here, **open** describes the form of the question, not the level of access. The step works only with the approved semantic context, the value lookup, and guarded queries. Ambiguity handling, execution checks, and memory support the complete path.

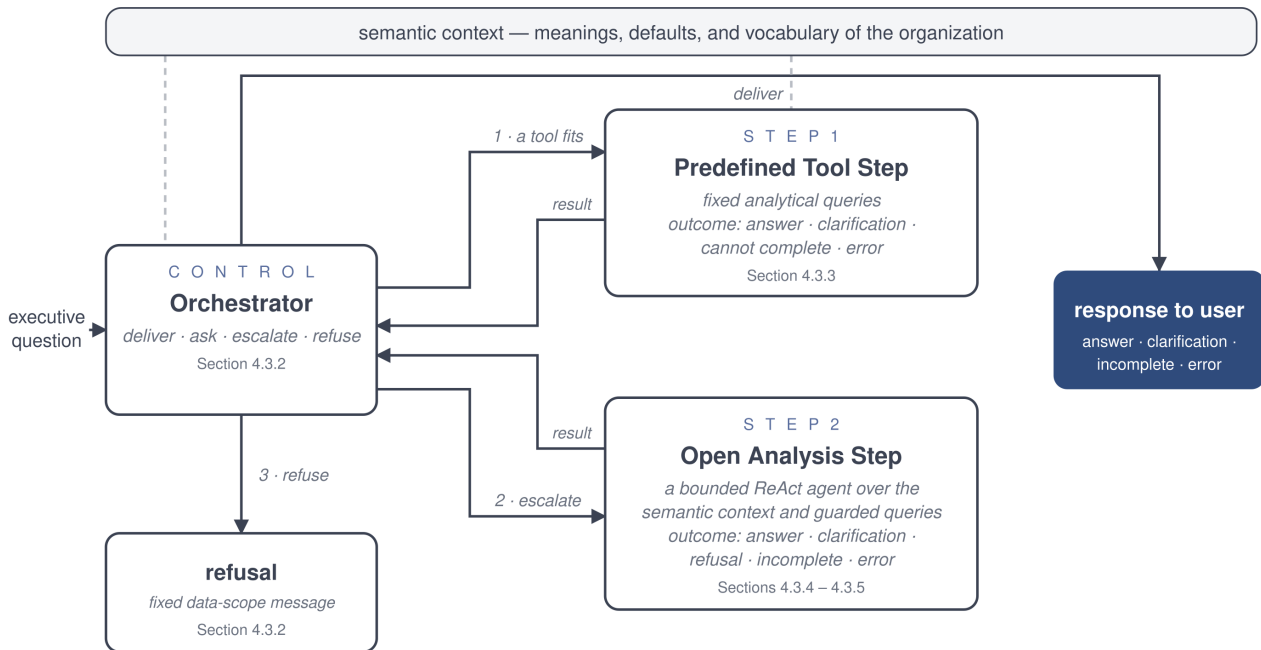


Figure 4.10 The Context-Grounded Conversational Agent follows a bounded escalation ladder from the Predefined Tool Step to the Open Analysis Step. Each step returns a typed outcome to the orchestrator, which delivers an answer or a clarifying question, escalates once, or refuses. Inside a step, tool results return to the agent that called them as observations. An attempt that reaches its execution limit returns an incomplete message. Technical failures remain error outcomes rather than data refusals, and the orchestrator records the path.

4.3.1 Problem Setting

The system receives questions expressed in natural business language. Because the questions arrive within a conversation, a question may continue an earlier one by changing only the period or scope. The system may return an answer, ask one clarifying question, or refuse and explain what is missing. Every reported number must be traceable to a tool result or an executed query. Generated claims without supporting data violate this requirement, even if the runtime delivers them as an answer.

Some questions match an analysis already provided by a predefined tool. One example is the current value of a KPI for a division in the latest week. Other questions combine dimensions, periods, or units in ways that no predefined tool covers. These questions move to the Open Analysis Step, where a bounded ReAct agent interprets the request and reads the data through guarded queries. Questions about information that is not stored cannot be answered. The system must determine which step fits each question and when it should stop.

Business language can also leave important parts of the data request open. An expression such as the orders of last quarter does not state which order measure is intended or which calendar defines the quarter. Resolving this ambiguity silently can produce an answer that looks correct but uses the wrong meaning.

A fluent answer can still be wrong. The model's confidence is therefore not evidence. Confidence must come from a traceable answer path that can be checked against the data. The system must also recognize questions that lie outside the available data. For these questions, an explained refusal is a correct result.

4.3.2 The Escalation Ladder

The ladder first tries the most constrained step whose definition matches the question. Once a predefined tool and its inputs are selected, the tool runs a fixed and tested calculation. This result is easier to check and is designed to require less time and model use. The Open Analysis Step can cover more varied questions by constructing a query at runtime, but it also leaves more analytical choices to the model. The ladder therefore considers reliability, coverage, and cost together.

The Ladder Orchestrator uses a model-based catalog check to select suitable predefined tools. If no tool fits or the first attempt cannot complete the request, the question moves to the Open Analysis Step. The ladder allows one analytical escalation. Outcome handling follows explicit rules without an additional model call. The trace records the path, the reason for escalation, and the available evidence.

Both answering steps use bounded ReAct-style tool loops, whose observation handling is described in Section 4.3.6. Their final outcomes return to the orchestrator rather than passing directly between steps.

Figure 4.10 shows the main paths through the ladder. A request for the current value of a KPI for one division can return directly from the tool step. A request for a total in a unit that no predefined analysis supports moves to the Open Analysis Step, which interprets the question through the semantic context and uses guarded queries. A request for information outside the available data, such as the organization's position against its competitors, ends in a refusal that explains what is missing.

The escalation packet carries the original question and an excerpt of the earlier step's response, including available findings and its stated reason for not completing the request. The next step is instructed to assess the original question with its own tools and not adopt an earlier claim that it cannot be answered.

The answering steps report status, response text, and a reason through a final-answer tool. The wrapper validates their structure, not their correctness. Execution code assigns limit and failure outcomes. A retained unstructured-text fallback is marked separately in the trace. Section 4.3.6 describes the execution checks, and Section 5.4.4 gives the protocol and its limitations.

Answer outcomes from either step are delivered with any stated assumptions. Table 4.5 specifies how reported outcomes and structural failures determine clarification, escalation, and stopping.

The refusal at the bottom of the ladder is distinct from a technical failure. It should explain the missing evidence. The current fixed message describes the general data scope rather than preserving the step's specific reason. The evaluation examines the consequence for name-identification questions.

Table 4.5 How the orchestrator handles step outcomes and execution failures. The process allows one analytical escalation, and the trace records why each attempt ended and what the orchestrator did next.

Condition	Orchestrator response
The catalog finds no predefined tool for the question	The question goes to the Open Analysis Step.
The Predefined Tool Step reports that it cannot complete the request, returns nothing, or fails	Escalation to the Open Analysis Step with the findings and the step's own reason.
The Predefined Tool Step returns an answer or a clarifying question	The response is delivered as the step produced it, with any stated assumption or empty result.
The Open Analysis Step returns a clarifying question	The question returns to the user without escalation. The reply and the question form a new run.
The Open Analysis Step reaches its model-turn limit without an answer	The attempt ends. The user is told that the analysis could not be completed.
The Open Analysis Step reports a refusal	The system returns its fixed data-scope refusal message.
The Open Analysis Step fails, including an empty final model response	The system reports a technical error, not a data-scope refusal.

4.3.3 Predefined Analytical Tools

The first step of the ladder is a set of predefined analytical tools. Each tool answers one recurring question shape over the analytics layer, for example a point lookup, a ranking, a trend, a comparison of periods, or a drill into one deviation. In this work, the set contains 20 tools. The tools are infrastructure rather than a contribution of this thesis, so this section describes only the principles that make them a safe first step.

The tools describe themselves. Each tool carries a short definition of what it answers and what its inputs mean, and everything else is derived from these definitions. The tool catalog and the instructions of the answering step are generated from the same definitions. This keeps the descriptions shown during selection consistent with those shown during answering.

The system exposes tool information in two stages. The selector first receives one short definition for each of the 17 question-specific tools and chooses the tools that fit the question. The three core tools for data orientation and entity lookup are then added automatically, and only this subset is given to the answering agent with full schemas. This applies the progressive-disclosure pattern introduced in Chapter 2 and reduces the tool information placed in the model context. Table 4.6 summarizes the two groups.

Table 4.6 The two tool groups and their exposure. The complete inventory with each tool's definition is listed in Appendix A.

Tool group	Number	How it is exposed	Purpose
Question-specific tools	17	the selector reads short descriptions and chooses a relevant subset	rankings, trends, comparisons, drill-downs, and similar analyses
Core tools	3	added automatically to every selected subset	understanding the data and resolving business names

A tool answers the question shape it declares and no other. A request outside that shape is rejected with a short explanation of which tool covers it, instead of being silently reinterpreted as a similar question. Time expressions follow the same discipline. Periods such as a quarter are resolved through the calendar declared in the semantic context, and a tool that reads one reporting period rejects an expression that spans many. The rejection returns to the calling agent as an observation. The agent may revise the request or report that the step cannot complete it. The orchestrator escalates on that step outcome, not on every tool rejection.

4.3.4 Text-to-SQL Methods

The Open Analysis Step turns a question that has no suitable predefined tool into a guarded database query. The ladder fixes the position and the safety limits of this step. The step runs the production method described below. The second method runs standalone as an evaluation comparison. Both have access to the semantic context and value lookup and execute guarded, read-only statements. Their prompts instruct them to ground business names before querying. Chapter 7 identifies the versions actually compared. They differ in how they form and test an interpretation before producing the final query.

ReAct-based Text-to-SQL method. The production method is inspired by the ktx runtime described in Section 3.4.2. A bounded ReAct agent develops one interpretation through a sequence of tool calls. It first reads the semantic overview, searches the value lookup for the stored forms of business names, and then uses small read-only queries to test the interpretation against the data. Each result informs the next action. The loop stops when the agent can provide a supported answer with the query and its stated assumptions, when it needs one clarification under the policy of Section 4.3.5, when the available evidence is insufficient, or when it reaches its limit of 14 model turns. The limit is a working allowance intended to leave room for reading the context, grounding names, and several database queries. If the limit is reached without a final answer, the attempt ends, and the system tells the user that the analysis could not be completed. The method can also continue a question from the current conversation. Its strength is interpretation: the question is resolved against the declared meaning of the data before it becomes a query. Figure 4.11 shows how this method connects to the orchestrator and the predefined tools.

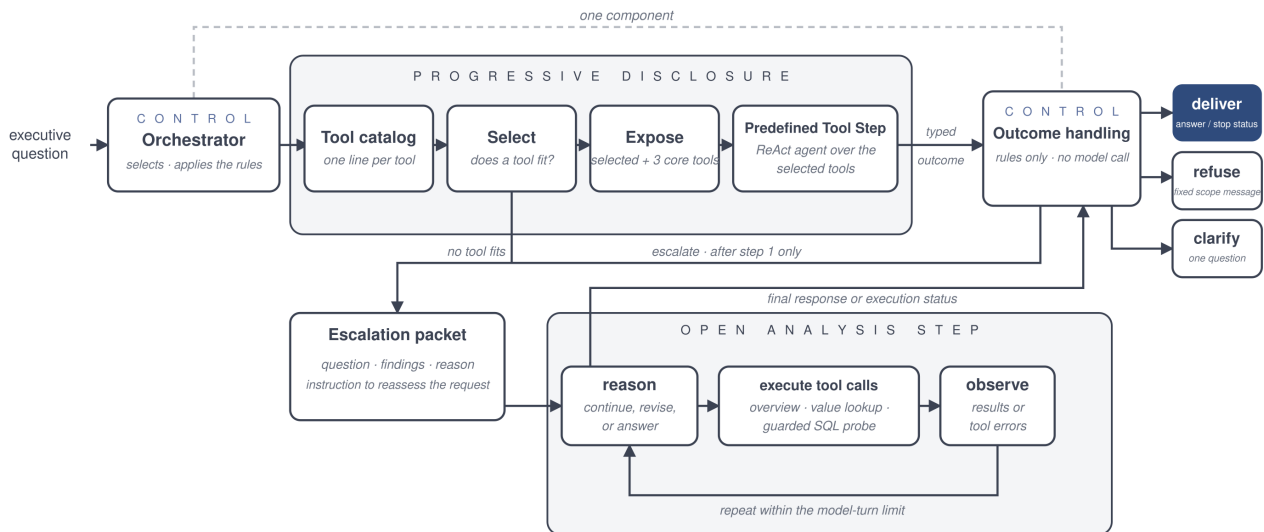


Figure 4.11 The main ladder path in detail. The orchestrator starts the catalog check and routes the question using its result. Returned outcomes are handled by explicit rules without an additional model call. The predefined tools are exposed through progressive disclosure. The selector reads one short definition per question-specific tool, and only the selected tools and the three core tools receive their full schemas. When no tool fits, or when the tool step cannot complete the request, returns nothing, or fails, an escalation packet carries the question, the available findings, and the step's own reason to the Open Analysis Step. Its bounded ReAct agent runs a loop inspired by the ktx runtime. It chooses its next tool after each observation, with the semantic overview required first, the value lookup for grounding names, and guarded queries for reading the data, within a model-turn limit. Its typed outcome returns to the orchestrator. A clarifying question returns to the user, and the reply joins the question for a new run. A larger landscape version is provided in Appendix C.

SOMA-inspired sampling-and-selection method. The second method is inspired by SOMA-SQL, introduced in Section 3.4.3. It treats the open question as a search problem. It samples several candidate

queries, executes them read-only, and lets a selection step compare their results. Where the candidates disagree, it probes the disagreement against the data, and a minimal repair step fixes candidates that fail. The method is autonomous. It does not ask the user for clarification. Instead, it handles ambiguity by comparing candidate queries and probing the data. This gives the evaluation a clear comparison. The ReAct-based method develops one interpretation and can use dialogue, while the sampling-and-selection method compares several candidate queries without asking the user. Figure 4.12 shows the stages of this method.

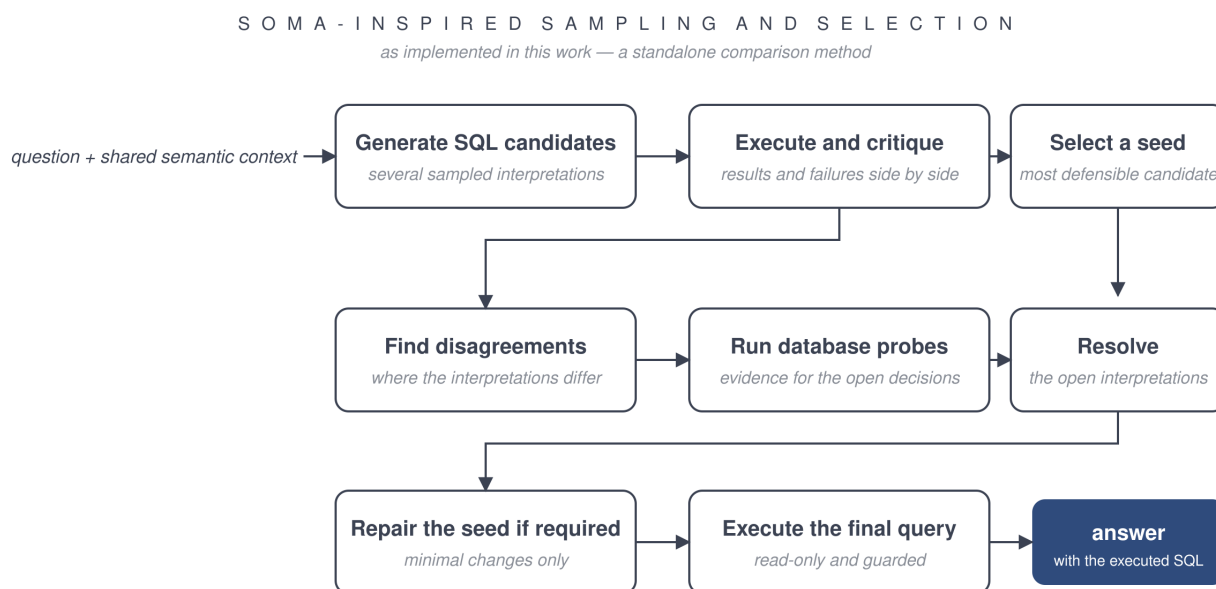


Figure 4.12 The SOMA-inspired sampling-and-selection method as implemented in this work. Several SQL candidates are generated and executed side by side. A seed is selected while the disagreements between candidates become database probes; the resolved interpretations repair the seed when required, and the final guarded query produces the answer. The method runs standalone: it shares the semantic context and the guarded database access with the production ladder but is not part of it.

Both methods record executed queries and their results, providing evidence against which the generated answer can be checked. The evaluation of this thesis compares the two methods and the full ladder on the same questions.

4.3.5 Ambiguity and Clarification

The ambiguity policy uses defaults declared in the semantic context for expressions that are known to be ambiguous, for example which measure a common business word refers to or which calendar defines a quarter. When a default applies, the system uses it and says so. The answer names the reading it chose, so a default is not a silent guess.

When the context provides no default and the readings would lead to materially different queries, the system asks the user. It asks once, and it asks with evidence. The clarifying question presents the readings that were found and what each of them would mean, so the user decides between concrete alternatives instead of being asked an open question. The reply continues the original question through the conversation state, and the system does not ask again within the same question.

Ambiguity that remains after this is flagged, not guessed away. The case is recorded with its evidence in the experiential memory of Section 4.3.7, where an expert can review it. When the expert decides a reading, the decision enters the semantic context as a new default, and the approved default can be reused in later questions with the same meaning. Ambiguity handling therefore continues across runs. Approved

defaults answer known cases, one clarifying question resolves new cases, and the recorded clarifications feed the expert review.

The clarification policy belongs to the conversational path of the system. By design, the sampling-and-selection method of Section 4.3.4 resolves ambiguity without asking the user. It compares candidate queries and probes the data instead. The evaluation compares these two behaviors on the same questions.

4.3.6 Execution Checks and Outcome Handling

The conversational system uses the approved semantic context to interpret the data. The value lookup returns stored names to support grounding before filters are constructed. The prompts instruct the agent to use this lookup, but query execution does not enforce that instruction. Predefined tools expose calculations for declared question shapes.

Execution checks enforce specific constraints. Tool argument and period checks reject requests outside the supported forms. Generated queries pass through a read-only guard before execution. A query rejected by the guard is not sent to the database. These checks concern the permitted request and query forms. They do not establish that the selected calculation answers the user's question.

Tool results, including errors and rejections, return to the calling agent as observations. The agent can continue the analysis, revise a request, choose another tool, or prepare an answer. Receiving an observation does not guarantee that the agent recognizes or resolves a problem. A query may execute successfully while using the wrong filter or reporting period.

The answering step returns its response and outcome to the orchestrator. The rules in Section 4.3.2 determine delivery, clarification, escalation, and stopping. There is no separate model review of the completed answer.

Both tasks use deterministic checks to constrain agent actions. In the weekly pipeline, these include ownership and accounting checks. Their scope differs from answer assessment. Final conversational answers are assessed in the evaluation against separately queried reference results using a rubric for scope, period, amounts, and unsupported claims.

4.3.7 Memory and Learning

The system keeps four kinds of memory, following the taxonomy of the harness survey [4], and each kind exists because something reads it.

Working memory holds the state of one conversation: the recent turns, the subject being discussed, and an open clarification if one exists. It is what lets a follow-up change only the period or the scope. Each conversation has its own working memory, it expires on its own, and the user can clear it.

Semantic memory is the semantic context itself. It is the only store the system reads for meaning, and it changes only through the governed construction process of Section 4.1.3. The agent never writes to it.

Experiential memory records finished question journeys. For every question, the system stores which path answered it, the queries and tools involved, and the final outcome. Only journeys that delivered an answer are read back, while clarifications, refusals, and failed journeys remain stored for audit. On escalation, the Open Analysis Step receives the delivered journeys of the same subject together with its packet. They are previous approaches and not verified answers. The instructions require the step to remeasure every number when reusing an earlier approach or query. In this version the experience is shared by all conversations; a per-user version would use a user identifier in the same store.

Shared memory, the survey's multi-agent memory in the setting of this work, is the bridge between the two tasks. The summary pipeline publishes its weekly results as finished analysis products, and the conversational agent reads them as evidence. A question about the week's critical developments can therefore be answered from work the pipeline has already produced and checked with its guards, instead of being recomputed.

The system has no separate long-term store. Its durable knowledge is the semantic context and the predefined tool set, and both change only through a governed process. The experiential records are

mined for subjects that were repeatedly answered beyond the predefined tools with a delivered answer. Each such subject is a candidate for a new predefined tool, not a demonstrated solution. A repeatedly clarified question is a candidate for a new default in the semantic context. An expert decides both. After expert approval, a recurring question form can move down the ladder: what required the Open Analysis Step this month may be answered by a cheaper, checkable predefined tool in the next version of the system. The ladder of Section 4.3.2 is therefore not static. It is the surface of a system that slowly moves recurring work toward its cheapest step.

5 Implementation

Chapter 4 defined the method and its decision rules. This chapter explains how those rules are implemented through shared data interfaces, executable tools, and controlled agent loops. It begins with the runtime architecture, then follows semantic-context construction and the two task implementations. The emphasis is on how the components exchange evidence, enforce constraints, and record their execution.

5.1 System Overview and Environment

Dash is a Python framework for building analytical web applications [52]. It is one of the preferred frameworks for analytical interfaces within the Infineon team. Its Python-based approach also fits the implementation's analytical components. The application provides access to both tasks, allowing users to ask questions, start weekly-summary runs, and read published reports. Figure 5.1 shows the application components and their shared foundation.

The conversational host calls the answering components directly within its Python process. Full division-week summary runs execute in separate worker processes. The application starts a worker for each selected division and reads its status file to display progress.

Both tasks use the approved semantic context, the shared data-access interface, and the model client when a model is needed. The semantic builder runs separately from question answering and weekly reporting. MCP adapters expose selected tool, Text-to-SQL, and memory capabilities to external clients. The launcher starts the three adapters as separate Hypertext Transfer Protocol (HTTP) servers, but the conversational host calls their underlying components in-process.

The corporate data source is a Hadoop platform, which provides distributed storage and processing [53]. Impala provides SQL access to data in this environment [54]. The implementation connects to the analytics layer and raw records through a shared read-only interface. Backend configuration stays outside the analytical components, so both tasks use the same interface. This follows the existing company data infrastructure.

DuckDB is an embedded SQL database that does not require a separate database server [55]. The system uses it for the value lookup and memory tables. The lookup uses its string-similarity function to rank possible name matches.

LangChain provides interfaces for language models and tools [56]. The implementation uses it through a shared model client, keeping connection settings outside the individual agents. The client connects to the configured model gateway. The model and connection used in each experiment are recorded in the evaluation setup.

Run configuration and reproducibility. Repeating a run requires the data and context used for that run, not only its reporting week. A warehouse refresh can change the records behind the same week label. The approved semantic context and the value lookup must therefore be identified alongside the source data. Table 5.4 lists the pipeline parameters. The reference below identifies the other information needed to repeat an execution.

Replaying a saved drill trace tests the downstream stages on fixed candidates. It does not repeat the model decisions that produced those candidates. A fresh model run may produce different actions even with the same settings. Keeping the approved context also differs from rebuilding it, because construction includes model-generated descriptions and corrections.

User interaction and system integration. The browser brings report reading and business questions into one workspace. A report library opens saved weekly summaries. The report view shows the headline, contributing stories, and opposing movements. Nested lines identify the parent of each percentage, and

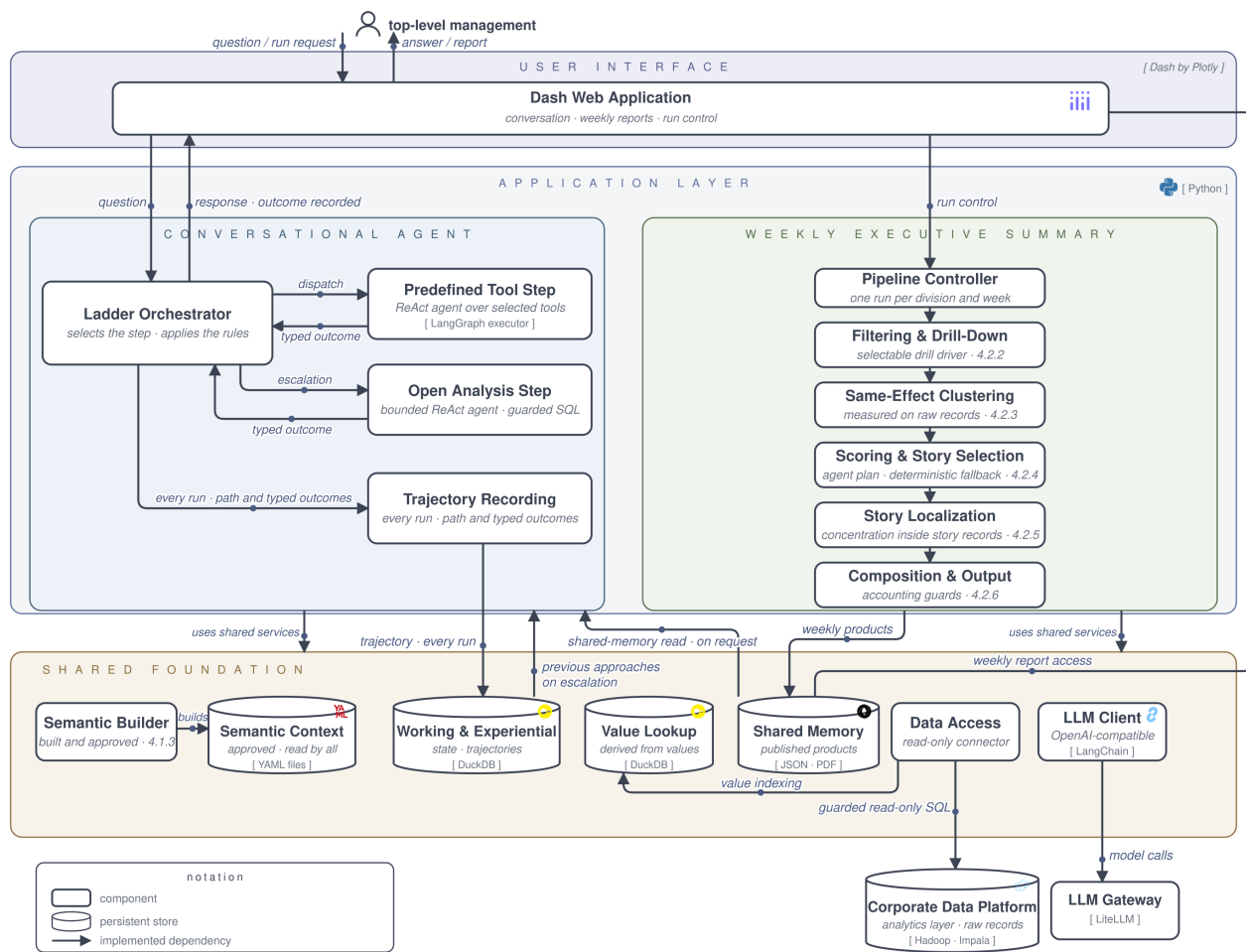


Figure 5.1 Layered architecture of the system. The Dash web application serves both tasks. The Context-Grounded Conversational Agent and the weekly-summary pipeline use shared Python components. Their shared foundation includes the approved semantic context, value lookup, memory stores, read-only data access, and model client. The conversational agent reads published summaries through the digest tool. On escalation, it can retrieve earlier delivered answers as previous approaches. Technology is stated in brackets. Responses are normally implied. A return arrow is shown where the returned result determines the orchestrator's next action. A larger landscape version is provided in Appendix C.

Table 5.1 Information needed to identify and repeat a run.

Part of the run	Information needed
Software	Code revision, Python version, installed dependency versions, and the launch command with its arguments
Data and context	Warehouse snapshot or retained extract, scope and reporting period, history range, approved semantic-context artifact, value-lookup version, and initial memory state
Model and controls	Configured model identifiers, generation settings, prompts and tool schemas, execution limits, and the typed outcome protocol in force
Execution mode	Drill driver, story-selection and phrasing modes, and whether the run uses a saved drill trace or a fresh execution
Outputs	Run identifier and locations of the trace, queries and results, typed outcomes and final status, fallback records, and published products, where applicable

expandable audit details show supporting information. This separates the short executive view from the evidence needed to inspect a finding.

The conversational interface displays the submitted question while the request is being processed. Its callback passes a session identifier to the answering service and replaces the pending message with the reply. The same identifier connects clarification replies and later questions to the session's working state. The forget control clears that state without deleting the messages already displayed. Section 5.4.5 describes the underlying storage.

The Developer area groups operator and diagnostic controls. It provides division-week run controls, a single-tool runner, and read-only context and memory views. The complete-summary controls offer an agent-driven mode and a deterministic mode. By default, they request live drill decisions, live story selection, and guarded phrasing; the selected mode is recorded with the run status. The keyword preview in the Tools view is a separate diagnostic and does not reproduce the ladder's model-based catalog selection. Semantic-context construction and approval remain outside the browser.

Callbacks delegate analytical execution to the service layer rather than implement the calculations in the interface. However, hiding developer controls through a display setting is not access control. The interface is a research prototype, and its usability and multi-user operation have not been established. Appendix D documents the implemented components with constructed example content. The screenshots illustrate presentation and navigation, not live agent outcomes or a user study.

5.2 Semantic Context and Builder

Section 4.1.2 described the content of the semantic context, and Section 4.1.3 explained its construction with limited expert time. This section describes how the context is stored and loaded, followed by the builder that produces it.

5.2.1 The Semantic Context

Storage. The semantic context is stored in a directory of YAML Ain't Markup Language (YAML) files, called the semantic home in the implementation. Its parts follow the content groups of Section 4.1.2. The warehouse part holds table and column entries, while the wiki holds business knowledge and vocabulary. The application part stores the calibration parameters of Table 5.4, the route encoding of the analytics layer, and the meaning of the KPI values. Table 5.2 lists the content, origin, and readers of each part.

A column entry. Listing 5.1 shows a simplified column entry based on a real example. The ported ktx ingest produces the column type and generated description. This work adds the time block and the disambiguation entry. The time block stores checked calendar facts, a probe query, and its expected result. The query allows these facts to be checked again against the warehouse. The disambiguation entry identifies columns that must not be used as alternative interpretations of the term.

Two entries in practice. A glossary entry connects a business term to its interpretation. For example, the main order term maps to the combined order-intake KPI. The entry also lists confirmed-only and unconfirmed-only interpretations that an answer should mention. It records a historical misspelling in the stored KPI name, which queries must reproduce.

Disambiguation entries distinguish columns that can be confused. The transactional date entry states that it is neither the analytics layer's weekly reporting label nor the requested-delivery bucket. The entries for those columns contain the corresponding warnings. The warning is therefore available whichever entry is read first.

The built context in numbers. The whole curated home is small. The generated schema file describes the tables in roughly one hundred column entries and 1,500 lines, and carries 64 disambiguation entries and the six verified time blocks of the warehouse's date columns. The glossary holds 33 business words with their aliases, the interpretation rules about 180 lines, and the wiki seven pages, covering topics such as the two data tiers, the date semantics, and the organizational dimensions. Altogether the curated context is about two thousand lines of YAML, small enough to be given to a model whole: the Open

Table 5.2 The parts of the semantic home.

Part	Files	Content	Origin	Read by
Warehouse schema	one generated file	column types, categories, generated descriptions, time blocks with verified facts, rules, and a probe per date column	builder (ported ktx ingest; time blocks added in this work)	both tasks
Warehouse curated	one file per table	expert table and column descriptions, disambiguation entries	expert, through review	both tasks
Application	one reviewed file	route encoding, KPI value semantics, drill, clustering, and summary parameter blocks	review informed by the probe	summary pipeline (parameters), both tasks (semantics)
Glossary	one file	business words and aliases mapped to schema elements	added in this work	conversational agent
Interpretation rules	one file	approved interpretation rules, several learned from the challenge loop and tagged with their origin	added in this work	both answering steps of the conversational agent
Wiki	text pages	business knowledge, abbreviations, approved term meanings	ingested company knowledge, curated	conversational agent, builder
Approval marker	one file	approver, date, and build record of the current context; renewed by the approve command	expert	the loader

Listing 5.1 The shape of one column entry in the warehouse part.

```

column:
  name: <transaction date>
  type: time
  category: time
  time:
    granularity: day
    role: event_time
    facts: <verified calendar facts of this column>
    rules: <how to query this column safely>
    check: { sql: <probe query>, expect: <expected result> }
    evidence: <path to the verification record>
  descriptions:
    ai: <generated description>
    expert: <curated description; wins when present>
  disambiguation:
    not: <the rival columns this term must not resolve to>

```

Analysis Step reads the composed document in one call, capped at 46,000 characters, and the summary pipeline reads its parameter blocks through the typed accessor, so no component searches the context.

Loading. One loader composes the semantic context at startup. The resulting document contains the KPIs, axes and their types, placeholders, KPI families, semantics, and parameter blocks. Curated descriptions take precedence over generated descriptions of the same column. The loader caches the composed document for the process and exposes it through a typed accessor. The engines therefore do not parse the YAML files themselves.

The approval gate. The loader requires the whole-context approval marker before loading the semantic context. If the marker is missing, loading fails without substituting a default context. The approve command records the approver, date, and build information. The loader checks marker presence, not whether the current files still match the approved content. Individual entries do not carry separate approval statuses.

5.2.2 Semantic-Context Construction

This subsection describes the builder used in Section 4.1.3. The package contains three stage modules that run in sequence and write to the current build's draft. Only the approve command publishes that draft to the semantic home. Figure 4.4 shows the stages and expert review points. The following paragraphs describe their execution and output files.

Running the builder. The builder provides `build`, `diff`, and `approve` commands. Build starts from a copy of the approved context and writes a separate draft. It runs the ingest, time investigation, and challenge stages, then folds their changes into the draft with origin tags. Diff records the changes against the approved context. An expert reviews the recorded results, proposed corrections, and differences from the approved context. The approve command then publishes the revised draft, saves the previous version as a dated backup, and renews the approval marker.

Individual stages can also run for a partial rebuild. The ingest retains its curated inputs and intermediate outputs in a state directory, together with a build manifest. This supports resuming a build. The compare command builds with the port and compares its output with stock ktx on the same inputs. A comparison result concerns those inputs and versions, not all possible builds. Description enrichment, time investigation, and the challenge loop use the shared model gateway. Scanning, writing, diffing, and the stored time checks do not require model calls.

The ingest stage. The drafting stage in Figure 4.4 uses the ported ktx ingest to produce the generated schema file in Table 5.2. Writers added in this work extend the column entries with a category, a time-block placeholder, and disambiguation entries. For columns that can be confused, the disambiguation is recorded in both entries.

The time stage. The time-investigation agent has three tools. It can read the tables and their existing time blocks, run read-only SQL probes, and write a time block into the draft. Its instructions require evidence for proposed calendar rules and separate that evidence from observations about the current snapshot.

The investigation probes formats and ranges, calendar patterns, cadence and gaps, row grain, and relationships between date columns. For cross-table reconciliation, it aggregates the finer table over candidate time windows and compares the result with the coarser table. A matching window supports a proposed convention on the inspected data. It does not prove that the convention will hold after every refresh.

Each time block stores its proposed rules, dated evidence, and a check query. The separate `time check` command executes the stored queries without a model. A check passes when it returns zero. Missing queries, rejected SQL, query errors, and results without a numeric value are reported as failures. Failed checks require review of the rule or another investigation run. The existence of this command does not imply that it runs automatically after every data refresh.

The investigation function defaults to 60 loop iterations and an elapsed-time allowance of 1,800 seconds. A response can request several tool calls within one iteration. The elapsed-time check runs between iterations, so it does not interrupt a model call already in progress. Near the iteration limit, the prompt asks the model to write supported blocks and finish. The return text reports the summary or stop reason and the blocks written.

The challenge stage. Each ambiguity class has a frozen question set and a directory of run reports. The questions are executed against the draft context, and the judge identifies failures. Proposed corrections describe additions, edits, or deletions of semantic entries. They are stored in a trial overlay read by the query-writing steps during the challenge runs. The loop does not write directly to the approved context.

Failed questions are rerun with the overlay to check whether the proposed corrections resolve those failures. These reruns provide evidence about the questions tested. They do not establish that previously successful questions remain correct. A separate full-set regression is needed to check for such regressions. Questions used to revise the context are development questions, not held-out evaluation data.

Rounds stop when no new lessons are produced or the configured round limit is reached. The command defaults to three rounds. At the end of the build, the overlay is folded into the draft. Interpretation rules and glossary changes retain tags identifying the loop and date, so they appear in the diff reviewed before approval.

Table 5.3 Artifacts of the challenge loop.

Artifact	Role in the challenge loop
Frozen questions	Identify the questions reused across rounds
Run report	Stores the recorded results and review outcomes of a round
Trial overlay	Holds proposed semantic changes used during reruns
Revised draft	Collects the changes for whole-context review and approval

Challenge-loop artifacts. The overlay is separate from the approved context.

The write path. Corrections enter through the curated ingest inputs or the challenge overlay and follow the build, diff, and approve commands described above.

The value lookup. The lookup of Section 4.1.2 stores searchable business names in a DuckDB file separate from the warehouse. Name searches use these extracted values rather than querying the warehouse for distinct names on each request. It is a derived application store, not an index created inside Hadoop.

The indexer selects eligible categorical columns from the semantic context. Through the shared connector, it queries the source tables for distinct values and their frequencies. With the corporate backend, these queries run through Impala. The lookup retains the stored spellings alongside forms normalized by case and whitespace. A second table records the distinct count and indexing time for each indexed column. Ordinary builds skip columns already indexed, while a forced rebuild rereads eligible columns from the source.

At runtime, a literal from the question is matched first by its normalized form and then by string similarity within the database engine. The conversational prompts instruct the agent to ground literals through this lookup. The engine does not enforce that instruction, as noted in Section 4.3.6.

The lookup is deployed alongside the memory stores as a derived part of the shared foundation. Its size and column coverage depend on the source and semantic-context versions used for the build. **Refresh and scheduling.** The refresh command builds a new lookup file, checks eligible-column coverage and readability, and records data and context versions before replacing the current file. The previous file is retained. Failed validation or replacement leaves the current lookup in use with a failure record. An explicit warehouse update identifier supports duplicate-run detection. Without one, a summary fingerprint cannot identify every content change, so it does not justify skipping a rebuild.

The weekly-job entry point checks for the expected reporting-week rows, refreshes the lookup once, and then starts summary runs. Row presence is a readiness heuristic, not proof that a warehouse load is complete. Summaries can continue after a lookup-refresh failure because they do not read the lookup. The entry point is suitable for an external scheduler, with a cron example documented in the project. This does not establish that a schedule was installed in the company environment.

5.3 Executive-Summary Pipeline

This section follows the five stages of Section 4.2 from flagged deviations to a published weekly summary. Filtering and drill-down produce candidates, and Same-Effect Clustering combines duplicate views. Impact Scoring and Story Selection determine the reported findings. Story Localization adds their measured detail, and Composition and Output assemble and publish the report. The following subsections explain the data structures, calls, and checks that connect these stages.

The semantic context supplies most analytical parameters. Placement also uses code constants and defaults. Planning-action allowances are execution settings, and rendering constants remain in code. Table 5.4 lists the working configuration.

Initial floor and case settings were reviewed on the calibration week of 2026-02-15, informed by the probe. The share rule and child cap were revised in a later review. The execution limits and four-week reference window are working settings, not empirically optimal values.

The relevance floor is also used during selection. The leader threshold, share rule, child cap, and group requirement are reused when calculating cluster share. The strict merge threshold also supplies the containment tolerance used during selection.

Placement fixes `lift_power` and `context_overlap_min` in code. It reads the depth discount from `score_height_decay`, using 0.85 because the working context has no override. The context's `score_lift_power` key does not control placement. Localization reads its thresholds and limits from the context. Its inline-retention and overshoot thresholds and the carry band used in rendering remain code constants and are not listed in the table.

The pipeline command selects the drill driver, story-selection mode, phrasing mode, and whether to reuse a saved drill trace. Drill decisions can use a deterministic mock driver or the live model. Story selection can use deterministic placement, a mock driver that reproduces the baseline, or the live model. The report uses template wording unless guarded rewording is enabled.

Reusing a saved drill trace allows downstream evaluation without repeating the model-driven walk. The default configuration is fully deterministic and makes no model calls. Each run updates its status after every stage so the application can display progress. A failed division run does not stop the other divisions.

Data access and the node model. The pipeline reads the analytics layer through the shared connector introduced in Section 5.1. Its node operations use that interface rather than opening their own database connections. A node is one analytics row of a KPI, a route, and a series name for the reporting week, carried as a small model with its deviation, its value, and its depth. The children of a node on one dimension are the rows whose upper series is the node, so listing children is one query per dimension and needs no join. Each child row carries the signals the walk works with, the same four the methodology names: the deviation, the deviation share of the parent (with the value share it is computed against), the lift as their ratio, and the surprise from the stored bounds. Rows whose series is a placeholder value are marked from the placeholder prefixes declared in the semantic context.

Mirror routes. Mirror routes are unified before the walk sees them. The analytics layer stores some routes twice, rooted at the division and rooted at a nested dimension such as the business line. Because every business line belongs to exactly one division, the routes of the nested root are the same slices as the division-prefixed routes. The tree layer merges the mirror axes into the node's own axes and deduplicates them, so the walk works on the unified hierarchy of Figure 4.2 without knowing that mirrors exist.

5.3.1 Filtering and Concentration-Based Drill-Down

This subsection implements Section 4.2.2. It covers the filter and its calibration, the walk, the drill decision, and the trace the stage leaves behind.

Filtering and calibration. The initial scan retains flagged rows with a defined band width and deviations above the KPI-specific absolute minimum. Its output identifies the week and available KPI families and selects the walk's KPI when none is supplied. It is not the walk's input list. The walk reads the hierarchy directly and applies the relevance floor and placeholder rules. The floor is the larger of the absolute

Table 5.4 Working configuration of the executive-summary pipeline.

Parameter	Value	Meaning
Filtering and walk		
min_abs_deviation	1.0M	A flag below this amount is not relevant (divided by the KPI's priority)
share_floor	0.05	The relevance floor rises to this share of the scope movement
decision_budget · time_budget	100 · 900 s	Execution limits of the walk
Drill decisions		
leader_threshold	0.6	Case A when the largest child carries at least this share
share_rule	0.05	Minimum share for a named case-B child, subject to the child cap
max_case_b_children	8	Maximum number of named case-B children
group_requirement	0.5	Case B applies only when the named children together carry this share
counter_threshold	0.2	An opposing child is recorded from this share on
boundary_margin	0.05	A top share within this distance of leader_threshold is flagged for scrutiny
Reference window	4 weeks	Period for descriptive lift and surprise statistics, including the reporting week
Pair classification		
strict_merge_min	0.95	Both net ratios at or above this value merge a pair; also the tolerance for "sits inside"
ambiguity_min	0.85	Both ratios at or above this value, but not both strict, is ambiguous and kept separate
overlap_min	0.20	Either ratio at or above this value records a partial overlap
Story selection		
context_overlap_min	0.5	A cluster with at least this share of its movement inside a story is context of it
lift_power	0.3	Exponent that softens lift in the impact score
depth_discount	0.85	Score factor per description dimension beyond the first
story_budget	8	Maximum number of leading stories in a plan
Planning-action allowance	workload-based	Driver attempts allowed, including refused proposals (Section 5.3.3)
score_unit	1.0M	The fixed unit in which the impact score is reported
Localization		
location_share	0.95	A dimension whose top value carries this share of a slice locates the story
lead_share	0.60	Below the location share, one value at or above this share is followed deeper
name_share	0.25	In a shared split every value at or above this share is named, up to three
carry_coverage	0.95	Directional coverage from which a route is worded "almost entirely in" rather than "mostly in"
floor_abs · floor_rel	1.0M · 0.10 of the owned amount	Below the larger of the two a slice is not explained further
max_depth	3	Maximum number of explanation levels inside a story

minimum, divided by the KPI's priority, and 0.05 of the absolute scope movement. The working absolute minimum is one million of the measure. No minimum surprise is applied.

The calibration probe. The probe measures and reports the distributions of deviations and child shares for a calibration week. These measurements support the review of the working thresholds. The selected values are written to the drill block of the application parameters in the semantic context. They are review decisions rather than thresholds derived automatically from the distributions. The pipeline reads this block at startup. The same block stores the walk's decision and time limits.

The low child-share threshold follows the collection policy of Section 4.2.2. The drill retains candidates for later comparison and selection, so the threshold is not a final story-acceptance rule. Keeping the settings in the semantic context allows them to be reviewed without changing the walk implementation.

The working share rule is 0.05, recorded in the semantic context as a review decision. The calibration probe still contains a built-in default of 0.15. That default does not replace the loaded value during an ordinary pipeline run. A fresh block written by the probe therefore needs review before it replaces the working configuration.

The walk. Stage 2 keeps pending nodes in a queue, beginning with the scope root. It processes larger absolute deviations first, giving them earlier examination when execution time is limited. At each node, it examines the available dimensions, obtains a decision, applies the guards, and records the effective action. Algorithm 1 summarizes the walk.

A drill action adds its target to the queue. A pick action records its targets as candidates and also allows the walk to continue below them. The walk records counters and retains a node itself when no dimension continues. If an execution limit stops the walk, pending nodes above the floor are kept at their current level and marked as provisional candidates. The trace distinguishes them from candidates selected after a completed decision. Queue order is an implementation detail, not a guarantee that changing the order preserves the candidates.

Per-dimension accounting. Each drill decision records the dimension's coverage, the deviation followed through drilled or picked children, and the rest of the parent. It also records the reconciliation residual between the parent and the sum of all children on that dimension. This makes gaps in the analytics layer visible in the trace.

The case rule. The case rule is one deterministic function. It receives the children of one dimension with their deviations and shares and returns the case, the recommended action with its targets, the counters, the top share, and the list of admissible actions. A time dimension and a dimension without any child above the floor have exactly one admissible action, a skip, so the agent cannot drill them. The thresholds are the four values of Section 4.2.2, read from the calibration block.

When the parent's deviation is zero or its absolute value is below the configured floor, the case rule uses the flat-parent branch. It sums the absolute deviations of the non-placeholder children and measures each child's share against that gross movement. Children moving in either direction can be selected, and no counter is assigned relative to the flat parent. If the gross movement is zero, the rule returns no targets or counters.

Decision counting. Every node–dimension decision counts toward the walk's decision limit, including dimensions with only one permitted action. A live decision can require several model calls, while a mock decision makes no model call. The trace records these counts separately.

The drill decision. The agent seat of Section 4.2.2 is one drill decision per node and dimension, implemented in the driver module. A drill decision is a bounded episode of the agent: it starts with fresh context, holds one node and one dimension, and ends with the decision for that dimension. Figure 5.3 shows what this means for one walk: the decisions fan out with the tree. The root node opens one decision per dimension. Every drilled or picked child opens its own decisions one level deeper, and the walk ends where no decision continues. A decision receives its current node and dimension, their signals, and the declared reference statistics. It does not inherit earlier agent decisions. This implementation executes them one after another, which keeps the trace strictly ordered; nothing in the design prevents running the decisions of one node in parallel. The figure also shows why stage 3 exists: the same customer is reached over three routes and becomes three candidates.

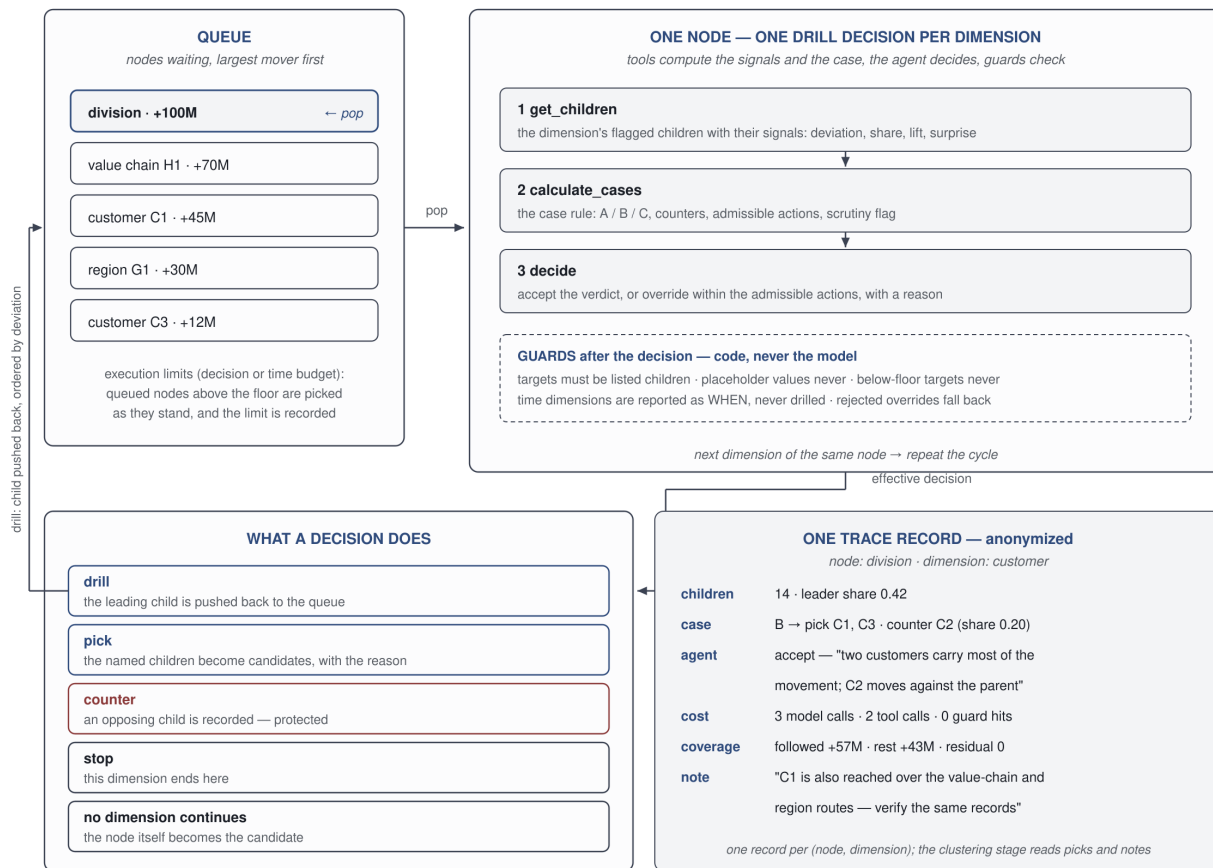


Figure 5.2 The drill walk as implemented. Nodes wait in a queue ordered by absolute deviation. Each popped node runs one drill decision per dimension, in which deterministic tools compute the signals and the case, the agent decides, and guards check the decision. A drill decision pushes the leading child back to the queue; picks and counters become candidates; every drill decision writes one trace record. The example record is anonymized.

The decision episode offers three actions in a fixed cycle, shown in Figure 5.2. The first returns the children with their deviation, deviation share, lift, and two surprise values. When enabled, it also loads the computed reference statistics. The second runs the case rule and returns the recommendation, admissible actions, and scrutiny flag. For eligible excluded children, it shows the distance from the naming threshold and checks whether inclusion is permitted. The third records acceptance or an override with a reason and an optional note. The episode refuses a decision before the case has been computed.

Reference statistics. The live driver obtains a derived reference for the same KPI and exact child route across parents. The semantic context enables this comparison and sets its window, currently four weeks including the reporting week. This is a working setting, not an empirically optimal window. The provider computes the median, 90th percentile, and count from finite same-direction observations. It excludes placeholders, nonpositive value bases, invalid bands, and parents below the current run's floor. References are computed once per route within a walk and saved with the period, exclusions, and data and context versions. A later walk recomputes them rather than trusting a saved snapshot. Fewer than two usable observations, duplicate rows, incomplete queries, or failures make the comparison unavailable. The trace records that status, and the original signals and guards remain available.

The action schema and the live driver. The driver uses a structured-output schema with a closed list of tools. A decision must include a reason and cannot be combined with another tool action. The live driver sends this schema, the system prompt, and the decision brief to the gateway model. Schema validation checks the response format. The guards then check whether the action and its targets are permitted.

Algorithm 1 The drill walk

Require: Scope, week, working limits, and selected decision mode

Ensure: Candidates and per-decision trace

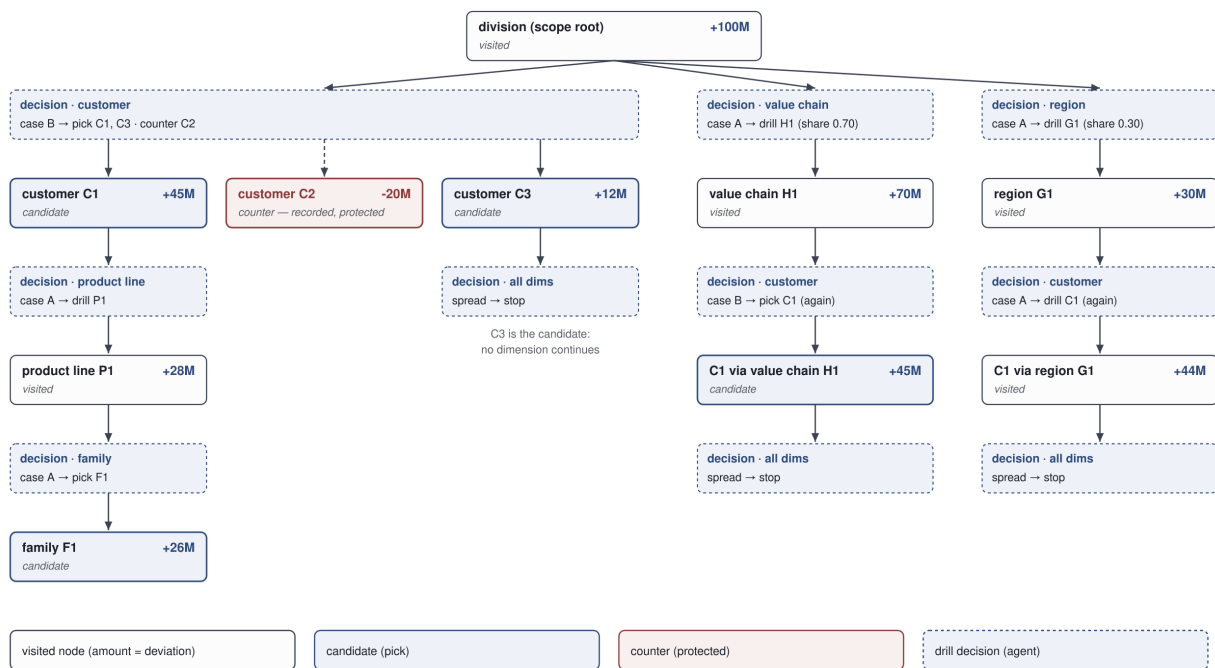
```
1: procedure WALK(scope, week)
2:   queue  $\leftarrow$  [root node of the scope], ordered by |deviation|
3:   candidates  $\leftarrow$  []
4:   seen  $\leftarrow$  {root}
5:   while queue is not empty and no execution limit has fired do
6:     node  $\leftarrow$  pop the largest mover
7:     continued  $\leftarrow$  false
8:     for each dimension of node (mirror axes unified) do
9:       children  $\leftarrow$  children of node on this dimension
10:      episode  $\leftarrow$  new drill decision(node, dimension, children)
11:      decision  $\leftarrow$  run the agent cycle on episode
12:      effective  $\leftarrow$  apply guards(decision, case of episode)
13:      record trace(node, dimension, children, case, decision, effective)
14:      if effective is drill then
15:        enqueue unseen target and mark it seen
16:        continued  $\leftarrow$  true
17:      end if
18:      if effective is pick then
19:        add named children to candidates
20:        enqueue unseen picked children and mark them seen
21:        continued  $\leftarrow$  true
22:      end if
23:      add the counters of this dimension to candidates
24:    end for
25:    if not continued then
26:      add node itself to candidates
27:    end if
28:  end while
29:  if an execution limit fired then
30:    retain queued nodes above the floor as provisional candidates
31:  end if
32:  return candidates, trace
33: end procedure
```

The system prompt includes the KPI definition and the applicable constraints. Each decision brief adds the meaning of the dimension under examination, avoiding repetition of the full semantic context.

The prompt defines lift and surprise with numerical examples. It asks the agent to follow clear recommendations and inspect eligible excluded children near a flagged boundary. The agent retains or revises the selection with an evidence-based reason, subject to the guards. Reference percentiles are not mandatory cutoffs. If the model does not answer or exceeds the decision episode's action budget, the driver accepts the computed verdict and records the fallback.

At startup, the driver checks that the configured model is available on the gateway. If it is missing, startup fails and lists the available model names. It does not silently substitute the deterministic path.

Guards and test drivers. Guards run after every decision, in code. An override is applied only when its targets are listed children, contain no placeholder value, and lie above the floor. A dimension with one admissible action rejects every override. Time dimensions are reported as the WHEN of a movement and are not drilled. A rejected override falls back to the computed verdict, and every intervention is written to the trace as a guard note. Two further drivers exist for testing: a mock driver that accepts every verdict, which reproduces the deterministic walk and serves as the parity baseline, and a scripted driver that replays canned actions for the guard tests. The separation between the decision driver and the execution tools allows rule-based and agent-guided operation to use the same calculations and guards.



this example: 8 visited nodes · 5 candidates · 1 counter · the same customer is reached over three routes
arrows: a decision creates the node below it (drill or pick); dashed = counter recorded, not drilled; "all dims" = the remaining dimensions of that node, one decision each

Figure 5.3 The drill decisions of one walk on the example week. Every visited node opens one drill decision per dimension; a decision creates the node below it. Decisions are independent by construction and executed sequentially in this implementation. Eight visited nodes, five candidates, and one protected counter; the same customer is reached over three routes. A larger landscape version is provided in Appendix C.

Trace and regression. Each walk writes a trace document and a readable report. Candidate records contain the route, filters, deviation, kind, depth, and selection reason. Per-dimension records contain the children and their signals, the case, the proposed decision, the effective decision after guards, and the coverage accounts. When reference context is enabled, they also retain its availability, snapshot identifier, and the boundary evidence shown to the agent.

Run statistics include visited nodes, agent and rule-based decisions, model and tool calls, tokens, fallbacks, overrides, guard interventions, and any execution limit that stopped the walk. Clustering reads only the candidates and notes. A saved trace can therefore supply fixed candidates for downstream evaluation without repeating the drill.

The regression suite protects the walk in three ways. A parity test walks a frozen week with the mock driver and with the deterministic baseline and requires the same visited nodes and the same candidates. Guard tests script overrides that violate each rule and check that the guard rejects them with a note. Unit tests cover the case rule at its thresholds, the single-admissible dimensions, the boundary flag, and the refusal of a premature decision.

5.3.2 Same-Effect Clustering

This subsection implements Section 4.2.3. It covers the record attribution that gives every candidate a measured deviation, the pair measurement, the classifier, and the strict merge that turns pairwise verdicts into clusters.

Input. Clustering reads candidates from the walk trace. Picks and counters retain their respective roles. If an execution limit produced provisional candidates, they can enter a story only when no better candidate owns their records.

Each candidate is represented by a predicate combining dimension values within the scope. Mirror-route predicates are first normalized to the scope's route representation. Candidates that resolve to the same predicate are combined, retaining the metadata of both. The pair loop therefore does not compare a slice with itself.

The attribution frame. A frame is the set of raw cells inside one anchor slice, grouped by the dimensions the comparison needs. Each cell carries its current value for the week and its history, the sum of the same cell over all prior weeks in the extract. Section 7.4.1 assesses the raw-query window. The expected value of a cell is the anchor's stored expected value multiplied by the cell's historical share of the anchor, and the cell's deviation is its current value minus that expected value. This is the allocation of Section 4.2.3 as code: the history decides the shares, the analytics layer supplies the level, and a complete partition of the anchor reconciles with the anchor's own deviation by construction. A cell without history receives an expected value of zero. If the entire frame has zero total history, all its expected values are set to zero. This fallback treats current values as deviations but does not reconcile a nonzero anchor expectation by construction.

The anchor of a frame is the nearest analytics ancestor of the slices being compared. The routine walks upward from the shared part of two candidates, dropping the most detailed dimension until an analytics row exists for the week. Every candidate is tagged with an evidence tier from this search: tier A when the candidate itself is an exact analytics series, tier B when its numbers are allocated below an exact ancestor, and tier C when one of its dimensions is unknown to the analytics layer or no ancestor exists at all. The tier travels with the candidate into the trace, because it states how much of its number is measured and how much is allocated.

The pair measurement. Every unique candidate pair is measured once, on one shared frame. The frame is keyed by the anchor, the union of the two candidates' dimensions beyond the anchor, the week, and the anchor's expected value, and a cache returns the same frame for every pair that shares this key, so a frame is reused only for matching cache keys. Different requested dimensions require a separate frame, even under the same anchor. The measurement, Algorithm 4 in Appendix E, takes the deviation mass of each candidate and of their intersection from the frame and forms the two net-contribution ratios of Section 4.2.3. The ratios keep their sign and are not clamped: a value above one means the intersection carries more net movement than the whole candidate, because records outside it move the other way, and a negative value means the intersection moves against the candidate's own net direction. The bounded directional coverage is computed on the same frame and recorded beside the ratios for analysis; it takes no part in the decision. The record of a pair also stores each candidate's official analytics deviation next to its frame mass and their difference, so that an allocated number cannot pass for a measured one in the audit. When either candidate's measured deviation is zero, no ratio is formed. The record keeps the measurements and marks the pair as not measurable, the classifier records the support relation as not measurable, and the pair stays separate. Such a pair is never read as disjoint, because a zero net deviation says nothing about shared records.

The record retains numeric zero placeholders where a ratio is undefined. Its `ratio_available` flag controls interpretation. If either candidate has zero measured deviation, the classifier returns not measurable before applying ratio thresholds. If no analytics ancestor exists, pair measurement raises an error rather than inventing an expected value. Frame-cache keys use the anchor, sorted dimensions, week, and expected value rounded to two decimal places.

The classifier. The classifier is a deterministic function of one pair record and the three thresholds of Table 5.4, `strict_merge_min`, `ambiguity_min`, and `overlap_min`, read from the clustering block of the semantic context. Algorithm 2 states it. It answers three questions in a fixed order. The direction relation compares the signs: opposed when the two candidates' net masses have opposite signs, mixed when the intersection moves against one candidate's net direction, aligned otherwise. The support relation reads the magnitudes of the two ratios with a gap-free precedence, so that every combination lands in one relation. The merge decision combines both. A merge is issued only for an aligned pair whose support is equivalent in the strict zone. An aligned pair in the ambiguity zone is marked ambiguous and stays

Algorithm 2 The pair classifier

Require: Pair record and working thresholds

Ensure: Support relation, direction, and merge decision

```
1: procedure CLASSIFY(dossier, thresholds)
2:   direction  $\leftarrow$  aligned
3:   if mass_A  $\times$  mass_B  $< 0$  then
4:     direction  $\leftarrow$  opposed
5:   else if min(ratio_A, ratio_B)  $< 0$  then
6:     direction  $\leftarrow$  mixed
7:   end if
8:   if the dossier marks its ratios as not measurable then
9:     return not_measurable, direction, separate, reason
10:  end if
11:  a, b  $\leftarrow$  |ratio_A|, |ratio_B|
12:  support  $\leftarrow$  disjoint
13:  if a  $\geq$  ambiguity_min and b  $\geq$  ambiguity_min then
14:    support  $\leftarrow$  equivalent
15:  else if a  $\geq$  ambiguity_min then
16:    support  $\leftarrow$  containment_A_in_B
17:  else if b  $\geq$  ambiguity_min then
18:    support  $\leftarrow$  containment_B_in_A
19:  else if max(a, b)  $\geq$  overlap_min then
20:    support  $\leftarrow$  partial_overlap
21:  end if
22:  decision  $\leftarrow$  separate
23:  if direction = aligned and support = equivalent then
24:    decision  $\leftarrow$  ambiguous
25:    if a  $\geq$  strict_merge_min and b  $\geq$  strict_merge_min then
26:      decision  $\leftarrow$  merge
27:    end if
28:  end if
29:  return support, direction, decision
30: end procedure
```

separate by default, and every other pair stays separate, with its support relation kept for the selection stage.

The strict merge. Candidate pairs with a merge decision form edges in a graph. Each connected component is accepted as one cluster only if every pair within it has a merge decision. Such a component is a clique.

For example, A may merge with B and B with C even though A does not merge with C. The implementation splits this group by complete linkage. It visits candidates in sorted label order and adds each to the first group whose members all merge with it. If no group meets this condition, the candidate starts a new group. Fixed inputs and labels therefore give a reproducible split, but not necessarily a unique valid partition.

The run artifacts retain the original component, its members, the pairwise verdicts, and the resulting groups in a review record. Each resulting cluster carries a warning that identifies this record. The weekly run continues automatically and does not wait for human review.

The cluster. A cluster keeps all of its views, the routes on which the drill reached it, the pick reasons and the notes the agent wrote, and the counter role if any member was a counter. Its representative is the widest member, the view with the fewest dimensions, which names the cluster and provides the slice on which its own deviation is measured in the next stage. The output of the stage is the list of clusters and the pairwise relation records of the week; pair counts grow quadratically with the candidate count. The frame cache reuses raw scans for matching anchor, dimension, week, and expected-value keys, not for every pair with the same anchor alone.

5.3.3 Impact Scoring and Story Selection

This subsection describes the implementation of impact scoring and story selection from Section 4.2.4. The story seat is the implementation of the agent-guided story-selection procedure. Its planning episode holds the current plan and applies the agent's actions under guards. The subsection covers the shared scope frame, impact evidence, ownership checks, and deterministic placement used as the baseline and fallback.

The scope frame. The stage builds one attribution frame for the whole scope and reporting week, grouped by the dimensions used to describe stories. Scoring, ownership, and the remainder scan share this frame. The cluster's representative, the widest member defined in Section 5.3.2, defines its reporting slice. The current value and deviation are measured on the frame cells matching that representative. Each matching cell is counted once. The routine does not form a union of all member predicates or add their deviations. Three quantities of a story are therefore kept apart in the artifacts: the measured deviation of its cluster on the scope frame, the owned amount after record ownership, which the summary reports, and the deviation of the representative's own analytics series, on which the explanation of Section 5.3.4 is measured. The composed output records all three under the story.

Relations between clusters are measured on the same frame. One cluster contains another when the contained cluster's movement lies inside the container at the ambiguity tolerance, but the reverse does not hold. A contained cluster is a refinement when it extends the container's dimension values with additional filters. Other containment relations describe distinct findings within a broader slice.

Impact evidence. Algorithm 5 in Appendix E calculates the deviation, lift, cluster share, and depth discount used in Section 4.2.4. Lift compares the cluster's deviation share with its current-value share, both measured against the scope. The exponent reduces the influence of lift on the score. It does not place an upper bound on that influence. On a flat scope, the same flag that the case rule and the placement use, lift is set to one, so the score keeps the deviation, the cluster share, and the depth discount.

The cluster-share calculation considers distinct contained clusters. It excludes refinements, clusters whose directional coverage inside the container is below `strict_merge_min`, and clusters described by more dimensions than the container. These exclusions distinguish competing reporting levels from narrower supporting detail. The remaining shares are sorted from largest to smallest. If the largest reaches `leader_threshold`, the container receives the remaining share. Otherwise, the routine considers at most `max_case_b_children` shares and keeps those reaching `share_rule`. If their sum reaches `group_requirement`, the container receives the remainder of that sum. The cluster share is otherwise 1, and any negative remainder is clamped to zero.

Numerical fallbacks. A zero current-value share gives a neutral lift of 1, and so does a flat scope. When the scope's current value is exactly zero, the routine substitutes a denominator of 1 for the value-share calculation. These are computational fallbacks, not measured shares for those undefined cases. A cluster with zero deviation contributes zero shares in the cluster-share calculation. A minimum lift of $1e-9$ before the exponent remains as a numerical guard. The depth factor is applied only for description dimensions beyond the first, excluding the scope dimension.

The story seat. Figure 5.4 shows the planning state and its driver. The planning episode assigns each cluster a short identifier and stores its selected role. Roles include story, counter, branch, and context. Context is recorded as either a container or a dependent. The episode also stores the selected remainder movers.

Before each action, the episode renders this state as a notebook. It begins with the scope movement, relevance floor, and story budget. Each cluster entry contains its identifier, name, measured deviation, impact factors, relations, and current role. It also has a localization preview computed from its exact analytics row. A cluster without such a row has no preview.

The notebook lists the currently eligible remainder movers and up to twelve drill notes. After an accepted action, the mover list is recomputed against the current owners. Identifiers remain stable, but movements that disappear or fall below the floor become unavailable. Selected movers retain their amounts at selection. The live driver receives this notebook together with its instructions and the recorded action history, including refused proposals and their reasons.

The planning episode sets `flat_scope` when the absolute scope deviation is below the relevance floor. For tool-listed remainder movements, a flat scope does not provide a direction for assigning a counter role. Selected movements of either sign therefore count toward the story budget. For a non-flat scope, an opposing remainder movement is treated as a counter and does not consume that budget. Existing cluster counter roles remain subject to their protection rules.

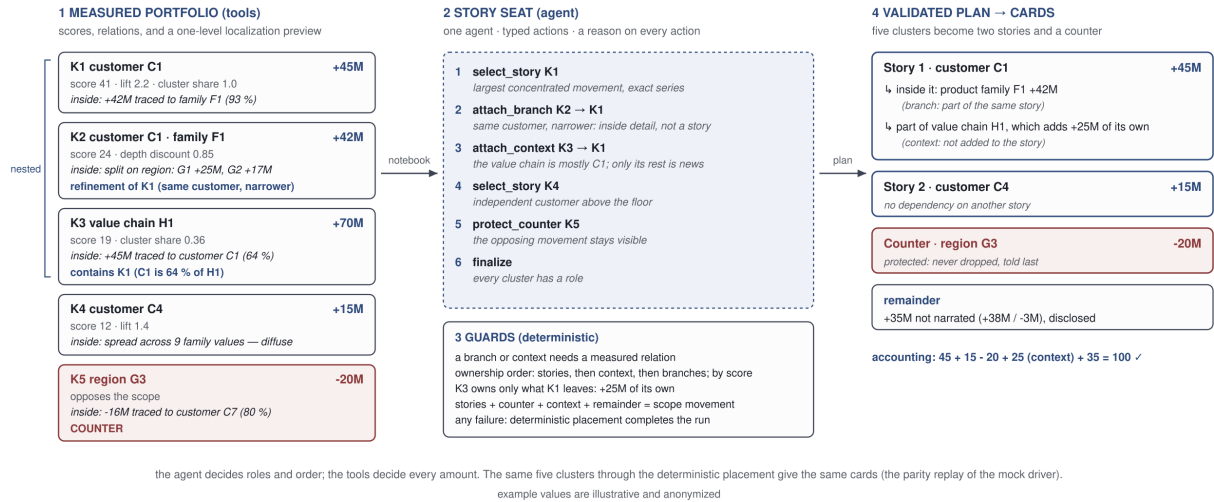


Figure 5.4 The story seat on one week. Left, the measured portfolio the agent receives: five clusters with their deviations, impact evidence, and relations, three of them nested. Middle, the agent's typed actions with their reasons, and the deterministic guards. Right, the validated plan as report cards: the refinement became a branch inside its story, the containing value-chain view became context with only its own remainder, the counter is protected, and five clusters are told as two stories and a counter with nothing dropped. Values are illustrative and anonymized.

Actions and guards. The action schema records the action and target identifier, the story identifier for attachment actions, and a required non-empty reason. Each handler either updates the plan or returns a refusal explaining why the action was rejected. The driver receives that refusal on its next turn and can correct the action within the remaining budget.

Story selection rejects unknown or already planned clusters. It also rejects a non-opposing selection when the story budget is exhausted. This budget counts leading stories and selected non-opposing remainder movements. Counters do not consume it. Counter protection rejects a cluster that does not oppose the scope.

A branch can be attached only when the pair has a measured refinement relation. Context requires measured containment in either direction or an overlap reaching `context_overlap_min`. The overlap is measured as the share of the attached cluster's movement inside the story. A remainder movement can be selected only if it remains eligible in a fresh scan of the unowned records.

A non-counter cluster may be marked `left_in_remainder`. It then owns no records and consumes no story budget. Finalization is rejected until every cluster has a recorded decision. Each proposed action is saved with its reason and outcome.

Planning-action allowance. The default allowance covers one action per cluster, one finalization action, and reserves for remainder selections and corrections. For N clusters and story budget B , it is $\min(200, N + 1 + B + 10)$. The correction reserve of 10 and ceiling of 200 are configurable working settings, not empirically optimal values. Counter selections may exceed the reserve because they do not consume the story budget. The pipeline accepts an explicit action allowance and ceiling. If the basic workload already exceeds the ceiling, it falls back before calling the driver. Otherwise, driver attempts, including refused proposals, consume the allowance. Exhaustion without a finalized plan invokes deterministic placement.

Drivers. Three drivers implement the same interface. The live driver sends the notebook and action history to the gateway with structured output bound to the action model. Its instructions define the seven

actions and prohibit changes to measured numbers or relations. A scripted driver replays a fixed action list for guard tests. The mock driver derives its actions from deterministic placement and passes them through the same guards. This supports parity checks without model calls. A completed mock or scripted plan is not recorded as model-driven story selection.

Ownership and closure. Algorithm 3 turns a finalized plan into owned amounts. The implementation orders entries by role priority, with stories before context and branches last. Within a priority, it uses the ranking returned by the scoring routine. Each frame cell is provisionally assigned to the first matching entry. Giving stories priority prevents context from taking records away from a selected story.

The current implementation includes branches in this provisional partition. It then removes every branch and every owner below the relevance floor, records them as evidence, and repeats the partition. Branches therefore do not own records in the final result. Removing an entry can release cells for another owner. The loop stops when no entry is removed and raises an error if its iteration limit is exhausted.

After convergence, unowned cells form the positive and negative remainder. The sum of the owned amounts and both remainder parts must match the scope deviation within a numerical tolerance. This tolerance is the larger of one unit and one billionth of the absolute scope deviation. Another guard checks that every protected counter remains in the output or evidence. A failed guard raises an exception and invokes the deterministic fallback. Material remainder movers not selected by the plan are recorded for disclosure.

The output retains the placement fields used by the deterministic baseline and adds the agent's reasons and telling order. The telling order follows selected stories and remainder movers. It is separate from the deterministic order used to assign records. The pipeline saves the planning record before composition and includes it in the final JSON artifact. The record distinguishes model-driven completion, mock or scripted completion, and deterministic fallback.

The deterministic placement. Algorithm 6 in Appendix E implements the baseline and fallback of Section 4.2.4. It assigns roles in score order and uses the ownership procedure of Algorithm 3. Removing an owner or selecting a remainder movement restarts the placement because the available records have changed. Placement stops when no eligible remainder movement remains under the story-budget rule, which permits an exception for the first opposing mover. Failure to converge within the iteration limit raises an error. The trace records whether no mover qualifies because the budget is reached or the eligible movements are exhausted. The output has the same structure as the story seat's output, without agent reasons.

Trace and parity. The stage writes the placement rows with their evidence and role, the demoted clusters with their last owned amount and reason, the selected remainder movers with their net at selection time, the remainder owners, the iteration trace, and the seat block with the driver name, the fallback flag, the action trace, and the guard log. The regression suite scripts each refusal, runs the accounting and counter guards against plans built to violate them, checks the budget with selected movers included, and replays the validated weeks through the mock driver against the deterministic baseline.

5.3.4 Story Localization

This subsection implements Section 4.2.5. It covers the localization routine that the composition calls for every selected story, the three kinds of finding it produces, the wording tiers, and the provenance every line carries.

Input and floor. Localization measures the representative's series. Ownership may assign only part of that series to the story. When these populations differ, the output marks the localization as evidence about the broader series, not as a breakdown of the owned amount. The root deviation of the descent is the representative's series deviation from its analytics row, because the children are measured against that series' expected value; every share on a line therefore divides by the deviation of the slice above it, and each line records that denominator and the word it uses for it, the story at the root, this route below a located route, that slice below a lead. The owned amount sets the floor, the larger of floor_abs and floor_rel times the owned amount, and is the amount the card reports; when the series deviation and the

Algorithm 3 Ownership and closure over a story plan

Require: Plan, selected movers, frame, scope, and floor

Ensure: Final owners, evidence, remainder, and telling order

```
1: procedure VALIDATE_PLAN(plan, selected_movers, frame, scope, floor)
2:   active  $\leftarrow$  planned entries except left_in_remainder, plus selected movers
3:   converged  $\leftarrow$  false
4:   for iteration in 1..MAX_ITERATIONS do
5:     active  $\leftarrow$  sort by role priority, then scoring rank
6:     partition  $\leftarrow$  assign cells to first matching active entry
7:     owned  $\leftarrow$  deviation sums from partition
8:     drop  $\leftarrow$  branches and entries with |owned| < floor
9:     if drop is empty then
10:       converged  $\leftarrow$  true
11:       break
12:     end if
13:     record drop as evidence with reasons
14:     active  $\leftarrow$  active without drop
15:   end for
16:   if not converged then
17:     raise ownership non-convergence error
18:   end if
19:   recompute partition over final active entries
20:   remainder_positive, remainder_negative  $\leftarrow$  unassigned deviation sums by sign
21:   tolerance  $\leftarrow$  max(1.0, |scope_deviation| x 1e-9)
22:   remainder_net  $\leftarrow$  remainder_positive + remainder_negative
23:   difference  $\leftarrow$  sum(owned) + remainder_net - scope_deviation
24:   if |difference| > tolerance then
25:     raise accounting guard
26:   end if
27:   if a protected counter is absent from active and evidence then
28:     raise counter guard
29:   end if
30:   record unselected material remainder movers
31:   return active, evidence, remainder, telling order
32: end procedure
```

owned amount differ, the card states on which the explanation was measured. The tiers, the floor, and the depth limit are read from the localization block of the semantic context.

The routine receives the story predicate, the representative's expected value and series deviation, the owned amount for the floor, and the remaining explanatory dimensions. Time dimensions, the scope dimension, and dimensions already fixed in the predicate are excluded. Each recursive call explains one slice. It stops below the localization floor, at `max_depth`, or when no dimension remains. The floor combines an absolute allowance with a share of the owned amount. Algorithm 7 in Appendix E describes the recursive procedure; the root call passes the parent word the story, and the recursion passes this route below a located route and that slice below a lead.

The split. For each remaining dimension, the routine builds an attribution frame for the current slice, grouped by that dimension. The slice's expected value supplies the anchor level. The allocation described in Section 5.3.2 then gives an expected value and a deviation for each group.

After removing placeholders, the routine sorts the groups by absolute deviation. The top group's share of the slice deviation measures concentration for that dimension. Dimensions are ranked by this share. The walk's concentration logic then determines the finding, using the localization thresholds in Table 5.4.

Location. When one or more dimensions have a top value at or above `location_share`, the story sits essentially entirely in one value of each of them, and together these values are the story's location. The routine combines them into one narrower slice and measures the explanatory path through them on one

shared frame. Starting from the whole slice, it takes at each step the dimension whose top value keeps the most of the direction-aligned movement, the declared dimension order breaking ties, so the path is one measured ordering, not a claimed hierarchy. The final cell yields the route's own deviation and its directional coverage, the share of the slice's aligned movement that the route retains. This coverage chooses the word: almost entirely in from carry_coverage upward, mostly in below it, with the retained share stated in the line. The top tier does not say entirely, because a coverage of 95 percent leaves up to five percent of the movement elsewhere, and the wording must not claim more than the records show. When the route's own deviation differs from the slice's by more than two percent, a second line states the offset, either as movement elsewhere in the same direction or, when it opposes the story, as the amount that brought the story to its reported value. The full measured ladder, one line per step with the retained share and the step's net, is written as audit lines that the report keeps behind the fold. The routine then recurses inside the located slice with the remaining dimensions, so a location is not restated at a deeper level.

Lead and shared split. If no location is found, the routine uses the most concentrated dimension. A top value reaching lead_share produces a lead finding. The output states its amount, value, and share, and the routine continues within that value.

Otherwise, values that reach name_share of the slice deviation and lie above the floor qualify as named carriers. Two or more carriers produce a shared split. The output contains one split line and up to three leaf lines with their amounts. If neither condition holds, a diffuse finding states how many values share the deviation, and the descent stops.

Section 6.3.5 shows the measured localization outputs for the Week A example introduced in Section 4.2.5.



Figure 5.5 Story localization on one story. At each level the slice is split by every remaining dimension, the most concentrated dimension is read against the tiers of Table 5.4, and the tier chooses the finding and its wording: a location, a lead to follow deeper, a shared split that ends the descent, or a diffuse stop. A lead at 93 percent is not rounded up to a location. Values are illustrative and anonymized.

Provenance and checks. Every line carries the predicate it was measured on, the exact filter that reproduces its numbers with one raw query. A leaf of a shared split additionally carries a measurement record with the week, the KPI, the path of dimension values from the story down to the leaf in display order, the current value, the expected value, the deviation, the parent's deviation, the share, and the analytics rows in which the same value appears within the story's filter chain, listed as evidence of where the value lives in the analytics layer without ever being added to the raw number. Two checks protect nested findings. A shared split rendered inside a narrowed slice is re-measured on the whole story population, and the audit lines say whether the two splits match or diverge, value by value. A story whose owned population is only part of its representative's series has every route line marked as evidence about the

broader series, because the localization frames cover that series, not the clipped ownership. The notes the drill agent wrote for a story are carried into the report as drill notes in the audit part. They are shown as the agent's observations and not as measured lines.

5.3.5 Composition and Output

This subsection describes the implementation of composition and output from Section 4.2.6. The composer receives the validated plan from Section 5.3.3, including story roles, attachments, and telling order. It renders that plan rather than deciding the arrangement again. The following paragraphs describe report construction, narrative and composition checks, optional phrasing, and publication.

Input and order. The composer receives the validated story plan of Section 5.3.3 and the scope frame. The scope movement is read from the analytics root row, and the components of the headline KPI declared in the semantic context, confirmed and unconfirmed orders in this work, are read from their own root rows, so the division sentence can state the split behind the net. Stories are told in the plan's telling order when the story seat produced one, and in score order from the deterministic placement otherwise; counters always come last. Context clusters are not told as stories; they appear as one line under the story they were attached to.

The composer also checks whether the absolute scope deviation is below the relevance floor. For a flat scope, it uses a separate headline template that reports the signed net amount as flat against the expected level. It does not use that small net movement to check whether a story's owned amount agrees with its contributor or counter role.

The narrative gate. Before rendering a story, the composer checks its owned amount and direction. A story whose absolute owned deviation falls below the relevance floor moves to context with its amount and a recorded reason. For a non-flat scope, a contributor is expected to move in the scope's direction and a counter in the opposite direction. A story whose owned amount conflicts with that role also moves to context and is recorded as an open case in the run. The direction check is skipped for a flat scope. These adjustments preserve the reported amounts rather than silently changing their signs or roles.

The layers. The template output has a headline, story cards, a closing account, and audit detail. The headline names the scope movement and the two headline components. Each story card begins with its name, owned amount, direction, and first localization line.

Below that sentence, the card shows component signals and any interpretation of opposing components. It then presents the localization tree or a stop line, followed by attached context and corroborating KPI findings. Corroborating amounts are not added to the story total. Provenance identifies the drill routes or remainder scan that found the story. Counter cards use the same structure with an explicit counter label.

The closing account reports the amount outside the stories, the signed explained-share ratio, and the positive and negative remainder. Scores, boundary warnings, agent reasons, raw checks, reconciliation details, and drill notes remain in the audit output. They are not all repeated in the executive text.

Composition guards. After assembling the report layers and before rendering, the composer checks that protected counters remain among the cards. Each card must also contain at least one component line, localization line, or stop line. Finally, card ownership, context movements, and the net remainder must sum to the scope movement within one unit of the measure.

A failed assertion stops that division's run. Its message is written to the status file, while runs for the other divisions continue.

The guarded phrasing pass. Algorithm 8 in Appendix E implements the optional rewording of executive sentences shown in Figure 5.6. The composition pass can reword the division headline, story sentences, and interpretation lines. It does not change the findings tree or audit records. Before a sentence reaches the model, names are replaced longest-first on whole-token matches. Longer names are replaced first so that a shorter name does not interrupt a longer match. Amounts outside those placeholders are then replaced. The implementation retains the mapping for restoration.

The verifier compares the ordered placeholders and recognized fact classes in the original and proposed sentences. These classes include direction, coverage qualifiers, and headline-component names. It also

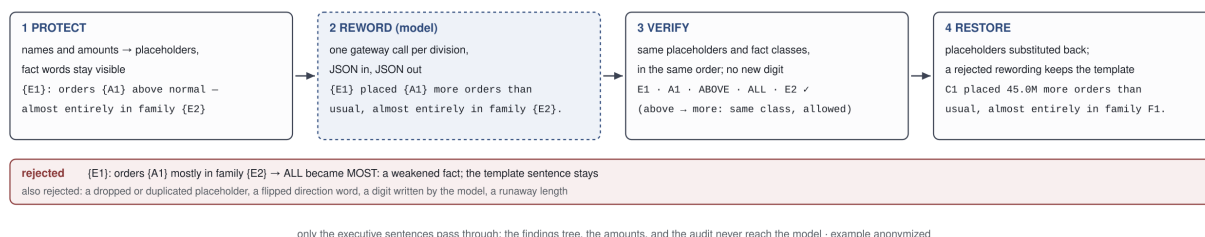


Figure 5.6 The guarded phrasing pass on one sentence. Names and amounts travel as placeholders, the model rewords, the verifier compares the sequence of placeholders and fact classes, and the accepted text is restored; a weakened fact is rejected and the template sentence stays. Example anonymized.

rejects digits outside placeholders and checks the permitted length. Matching these features can catch altered amounts, direction flips, weakened recognized qualifiers, and reordered protected elements. It does not establish that every possible change in meaning is detected.

One gateway call carries the sentences as a JSON object keyed by sentence identifier. Each accepted sentence has its placeholders restored. A missing or rejected sentence keeps its template wording. An unusable response or call failure leaves the affected output on the deterministic path. Both template and delivered wording are retained for comparison.

Publishing. Each division-week writes a JSON document, a readable text tree, and a status file. The JSON retains the report layers, audit records, and both template and delivered wording. The application polls the status file while the run is active.

The weekly publisher collects the division outputs and builds an executive view for the web cards and the formatted Portable Document Format (PDF). Each card uses its story dimensions as the title and shows the analytics route beside it. Reported contribution is kept separate from the breakdown of the full group. The closing reports context and the positive and negative movement outside the selected stories.

The executive-view phrasing pass can reword breakdown lines, while titles, amount labels, headline, and closing remain template text. This is separate from the composition pass described above. The publisher renders the executive view as Hypertext Markup Language (HTML) and prints it through a headless browser. If that rendering is unavailable or fails, the text-tree PDF remains the fallback. The published products are registered in Shared Memory for retrieval through the conversational digest tool. Neither rendering nor rewording changes story roles or owned amounts.

5.4 Conversational Agent

Section 4.3 described the Context-Grounded Conversational Agent and its ladder control rules. This section follows the implementation from dispatch through the answering steps, their typed outcomes, and memory. Table 5.5 maps the thesis terminology to class and module names. Figure 5.7 shows the calls and returns for one question.

LangGraph is a framework for stateful agent workflows [57]. The implementation uses its ReAct executor for the model–tool loop in the Predefined Tool Step.

5.4.1 The Ladder Orchestrator

One policy. The orchestrator owns the answering steps, creates each lazily on first use, and hands them to one dispatch policy. During the design phase the policy was a seam behind which several control strategies were compared, among them a router-first and a plan-execute-verify variant. When the ladder was settled as the production design, the alternatives were removed from the code base, so the orchestrator now runs one policy, the ladder, and the settings default to it. The retired variants remain in the repository history for the case that a baseline is needed again.

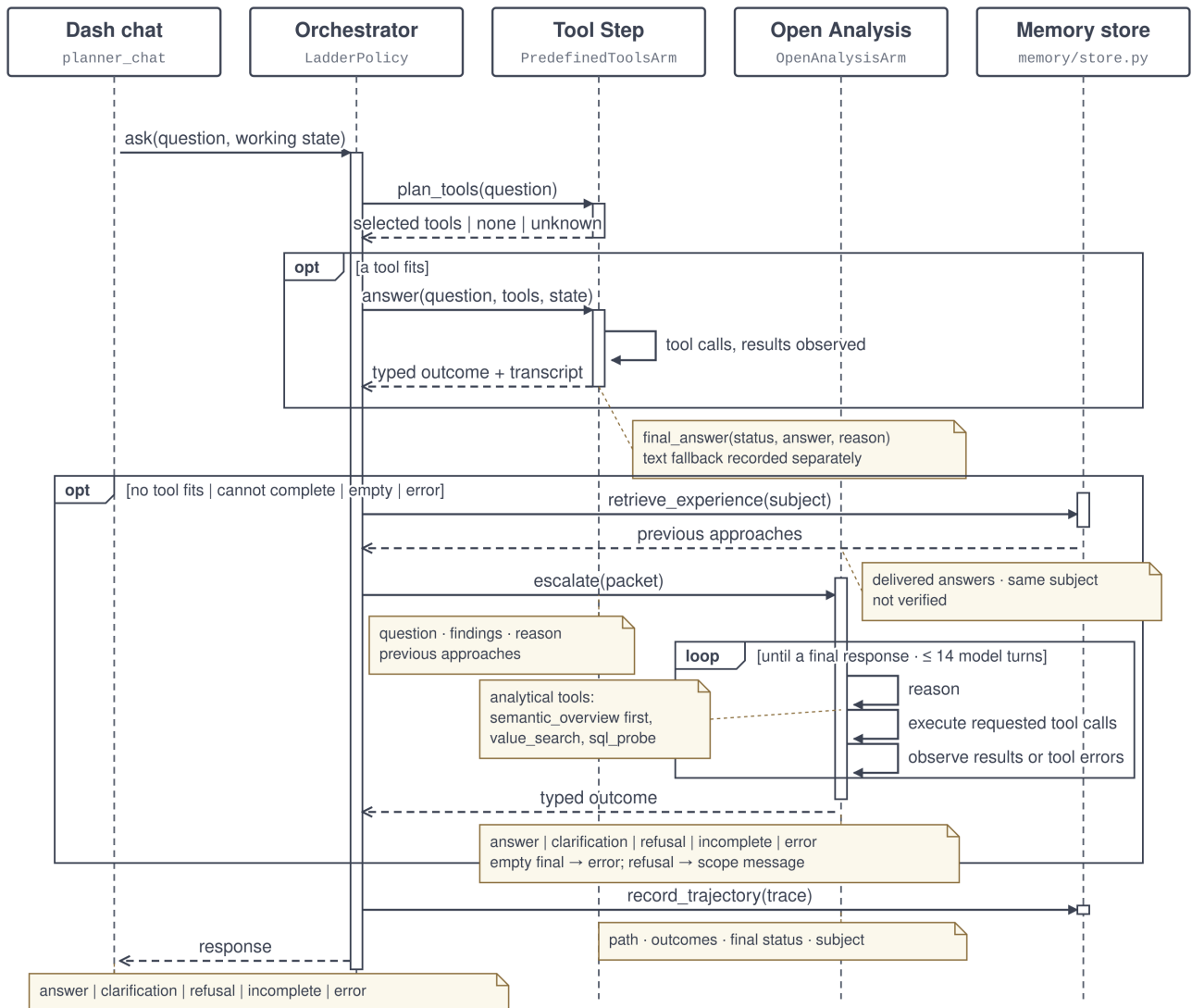


Figure 5.7 One question through the code. The Ladder Orchestrator starts the catalog check and runs the Predefined Tool Step when applicable. A missing tool match or an unsuccessful first attempt leads to the single escalation to the Open Analysis Step. Final step results return to the orchestrator as typed outcomes. Solid arrows show calls, and dashed arrows show returns. The run is recorded before the response is delivered.

Table 5.5 Thesis names and code names of the conversational agent.

Thesis name (Section 4.3)	Code name	Module
Ladder Orchestrator	Orchestrator with the ladder policy, LadderPolicy	agent/orchestrator.py, agent/dispatch.py
Predefined Tool Step	PredefinedToolsArm with Catalog Selector and the LangGraph executor	arms/predefined_arm.py, tools/tool_selector.py
the 20 predefined tools	predefined_registry	tools/analytics_tools.py
Open Analysis Step	OpenAnalysisArm around run_semantic_agent	arms/open_analysis_arm.py, text_to_sql/agent.py
the bounded ReAct agent	the tool loop of run_semantic_agent	text_to_sql/agent.py
sampling-and-selection method	run_soma	text_to_sql/soma/pipeline.py
typed outcome protocol	FinalOutcome, final_answer, and classify_final	backend/answer_outcome.py
Trajectory Recording	record_trace, record_trajectory	memory/trace_store.py, memory/store.py
honest refusal	CantAnswerArm	arms/cant_answer_arm.py
step identities in traces	predefined_tools, open_analysis, cant_answer	arms/base.py
memory stores	store.py over one DuckDB file	memory/store.py

The run. The orchestrator first asks the Predefined Tool Step whether its tools fit the question. The catalog check returns selected tools, an explicit no-fit result, or a failure to produce a usable selection.

A failed check leaves tool suitability unknown. It does not establish that no tool can answer the question. In this case, the Predefined Tool Step runs with all tools exposed. After a successful selection, it runs with the selected tools and the three core tools, with eight tools in total at most. The step returns its answer and the transcript of its tool calls.

Outcome rules. The Predefined Tool Step returns text and a status handled by the final-response protocol of Section 5.4.4. This status identifies how the response should be handled, not whether its content is correct.

The policy then applies fixed rules. A clarification returns to the user. A cannot-complete status, an empty response, or an error reported by the step escalates. Every other answer is delivered as the step produced it, including an answer that states an assumption or reports an empty result. An empty query result differs from an empty response. It can describe a quiet week or a slice without records, so it is delivered as a finding.

The escalation packet. The Open Analysis Step does not receive a bare question after a failed first step. It receives the packet of Section 4.3.2 as plain text, which carries the original question, the partial findings of the earlier attempt, which include the step's own statement of what it could not do, and an explicit instruction to ignore any claim that the question cannot be answered. The instruction tells the escalated step to assess the original question using its own available evidence, rather than accept the earlier step's conclusion. When eligible experience on the same subject exists, the packet also carries it, as Section 5.4.5 describes.

Typed outcomes. The Open Analysis Step returns its answer with a typed status, which is one of answer, clarification, refusal, incomplete, and error. The policy honors the status before anything else. A clarification returns to the user without escalation, because a clarifying question is a designed outcome of Section 4.3.5. An incomplete attempt returns the fixed execution-limit message without entering the refusal

step. A reported refusal goes to the fixed refusal message at the bottom of the ladder. A technical error returns a separate error message without entering that refusal step. An answer is delivered as the step produced it. The ladder allows one analytical escalation, so no run visits the Open Analysis Step twice.

Recording. Every run ends in the same place regardless of its path. The orchestrator writes the trace, which is the question, the policy name, every attempt with its step, its status, its execution counters, and its escalation, the final answer, and the final status, first to the trace log and then to the memory store as a trajectory. The write is best effort, so a memory problem cannot block an answer. The trace is also what the chat shows in its step panel and what the evaluation reads.

5.4.2 The Predefined Tool Step

The tool layer. The 20 tools of Appendix A are LangChain tool objects registered in one registry. Each tool is a Python function over the read-only connector, and the same registry serves three consumers without duplication, which are the tool step, the tools service that exposes the tools to external clients, and the developer view of the web application that runs any tool by hand. Table 5.6 groups the tools by the question shape they answer; the one-line definitions that the selector reads are the first line of each tool's docstring, which is why the appendix is generated from the registry.

Table 5.6 The predefined tools grouped by the question shape they answer.

Family	Tools	Question shape
Orientation (core, always exposed)	describe_semantics, resolve_entity, list_entities	what exists, and which stored name a business name means
Point lookup	get_value, kpi_split, entity_profile	one value, one slice across its KPIs, one entity in full
Ranking and movers	rank_series, movers, scan_level	who is largest, who moved most, what moved across one level
Windows and periods	get_trend, compare, compare_periods	one series over time, two things side by side, two periods
Flags and bounds	get_outliers, get_bound_breaches, flag_history, explain_flag	what is flagged, what crossed a bound, flags over time, why a flag
Hierarchy and novelty	drill_down, new_series	where a movement comes from, what appeared for the first time
Relations and overview	kpi_relation, weekly_digest	do two KPIs move together, what the week's critical stories are

Progressive disclosure in code. The catalog selector implements the two-level exposure described in Section 4.3.3. It first shows the model one-line descriptions of the 17 question-specific tools and requests a structured selection with a reason. The three core tools are added automatically and are not listed in that catalog.

After successful selection, the LangGraph executor receives at most eight analytical tools, including the core tools, plus the `final_answer` outcome tool. It handles the exchange between the model and these tools. Full schemas for unselected tools are excluded from its context. The selection reason is retained in the trace.

The executor wrapper does not set a separate application-level recursion limit in its `graph.invoke` call. It therefore uses the configured framework behavior. An exception from the answering step is caught by dispatch and treated as a failed attempt, allowing the ladder to escalate. This is separate from the explicit model-turn limit of the Open Analysis Step.

Instructions generated from definitions. The executor's system prompt is composed from the definitions of the tools it received, which keeps the descriptions used during tool selection consistent with those shown during answering. The tools also apply their own discipline. A request outside a tool's declared shape is rejected with a short message naming the tool that covers it. A period expression is resolved by one shared grammar that accepts the newest week, an explicit week, a trailing window of weeks, an

explicit range, and fiscal quarters. Quarter words resolve through the fiscal calendar declared in the semantic context, not through the calendar year. A tool that reads one reporting period rejects an expression that spans several, and every tool caps its rows. A rejection travels back as an ordinary tool result, so the executor can choose another tool or report the finding, and the orchestrator can escalate when the step reports that it cannot complete the request.

The shared-memory read. One tool of the family table is also the agent's read path into the shared memory. When a question asks about the week as a whole, the selector picks `weekly_digest`, and that tool first looks for the summary the pipeline published for the requested week. When one exists, it returns the published stories and their source. A missing or unreadable artifact returns an explicit tool error. It never recomputes a substitute and presents it as the published report. The read therefore happens only when a question asks for it, which is the behavior Section 4.3.7 describes and Figure 5.1 draws.

5.4.3 The Open Analysis Step

One session and its tools. The Open Analysis Step wraps a function that runs a bounded ReAct loop through the LangChain model interface and the gateway. It uses a custom loop rather than a framework agent to control the turn limit and stopping conditions described in Section 4.3.4.

The model has three analytical tools plus the final-answer outcome tool. The semantic overview returns the composed context from Section 5.2.1, capped at 46,000 characters. The prompt instructs the model to read it first for schema meanings, glossary entries, interpretation rules, and time semantics. Value search uses the value lookup to find stored spellings of business names. The guarded query probe executes read-only SQL through the shared connector and returns at most 50 rows. Queries must therefore use aggregation and limits rather than return whole tables.

The model-turn limit. The agent loop reads `open_analysis_turn_limit`, which defaults to 14. The counter advances once per loop iteration. Each model response may request several tool calls, which are executed before the next iteration. A retried model request remains within the same iteration. Near the limit, the loop adds a reminder asking the model to finish using the available evidence.

At the limit, an attempt without a final answer returns empty text and the status `incomplete`. The result also contains `num_turns`, `tool_calls`, and the partial trace. `OpenAnalysisArm` passes this outcome to `LadderPolicy`, which returns a fixed message without entering the refusal step. The attempt remains recorded as incomplete rather than as a supported answer. A final model response with empty text is handled separately as an error with the reason `empty model response`.

Typed status by contract. The loop validates final-answer tool arguments and records the outcome channel. Invalid final submissions return a validation observation and consume further iterations within the same limit. The text fallback and its limitations are described in Section 5.4.4.

Conversation and clarification policy. The chat composes earlier turns into conversation text for the Open Analysis Step. When a clarification is open, it combines the user's reply with the original question and instructs the model not to ask again.

A policy setting allows clarification in production and lets the evaluation suppress it for comparison. When clarification is suppressed, the prompt instructs the model to use the interpretation favored by the semantic context's defaults and state the alternative briefly.

Model calls use the corporate gateway. If a model accepts only a fixed temperature, the loop rebuilds the model interface once with that temperature and continues. This handles a gateway restriction rather than selecting a new analytical method.

The sampling-and-selection method. The SOMA-inspired method of Section 4.3.4 is implemented as a separate function that samples several candidate queries with a requested higher temperature, executes them read-only, requests a lower temperature for selection, probes the disagreements against the database, and repairs the selected candidate when needed. It shares the semantic context, the value lookup, and the read-only guard with the ladder, and the evaluation harness runs it beside the ladder

on the same questions. It is not part of the production ladder and has no arm of its own, which is the standalone position Figure 4.12 shows.

5.4.4 Guards and Typed Outcomes

Guards at the execution points. Predefined tools check arguments against their supported request forms. The shared period grammar rejects a multi-period expression when a tool requires a single reporting period. The Open Analysis query probe checks generated SQL with the read-only guard before execution and returns at most 50 rows. Reported rejections return to the agent as tool results, allowing another request within the step's execution limits.

The final-response protocol. Both steps expose a `final_answer` tool alongside their analytical tools. Its arguments are `status`, `answer`, and `reason`. The shared `FinalOutcome` model requires nonempty text, rejects extra fields, and accepts only `answer`, `clarification`, `cannot-complete`, and `refusal` statuses. The tool step maps refusal to `cannot-complete` so that it cannot end the data analysis on its own. Open Analysis maps `cannot-complete` to `refusal`.

An invalid final-tool call produces validation feedback. The LangGraph wrapper permits at most two continuations after a rejected final response. Open Analysis can retry within its existing model-turn limit. It also defers a final submission made together with analytical calls until their results can be observed. The tool-step wrapper instead records such a mixed final channel. These controls concern response structure, not answer verification.

The shared parser also accepts a validated JSON outcome in final text. If nonempty text does not parse, the implementation retains an `unstructured` fallback and delivers it as `answer-shaped` text. The trace preserves the original response, parser diagnostic, outcome channel, and invalid-final count. A missing structured outcome can therefore still conceal an intended clarification or refusal.

Execution outcomes and dispatch. The Open Analysis loop sets `incomplete` when its model-turn limit is reached without a final response. It sets `error` when a model request fails or its final response contains no text. These outcomes are assigned by execution code, not declared by the model. Dispatch uses status and structural failure checks. For an `answer-shaped` response, it also checks for empty text or a leading error marker. Section 5.4.1 describes the resulting routes.

Observations and limitations. A pre-model hook bounds each tool observation in the Predefined Tool Step to 12,000 characters without overwriting the recorded full message. The query probe also bounds returned rows. Shortening preserves a manageable context but can hide detail needed for the answer. The model must distinguish a preview from a complete population. The guards enforce their defined constraints, not semantic correctness. Chapter 7 assesses full answers against reference queries and the recorded evidence.

5.4.5 Memory Stores

One file, three tables. The memory store keeps session working state, run trajectories, and registered pipeline products in one DuckDB file. These records use the embedded storage introduced in Section 5.1. Figure 5.8 shows the read and write paths. Trajectory and product rows carry the semantic-context version recorded at write time. Working-state rows instead carry their session, turn, update time, and expiry. The memory interface also serves the semantic context read-only. It does not provide an agent tool for changing that context.

Working memory. The chat stores working state by session. Each turn loads that state, runs the question, appends the new turn, and saves the updated state. It contains recent turns and any open clarification. The state expires after 24 hours or is cleared by the user.

Both answering steps receive conversation text from this state. This supports follow-ups that change the period or scope without restating the whole question, but does not guarantee that the model preserves the intended interpretation.

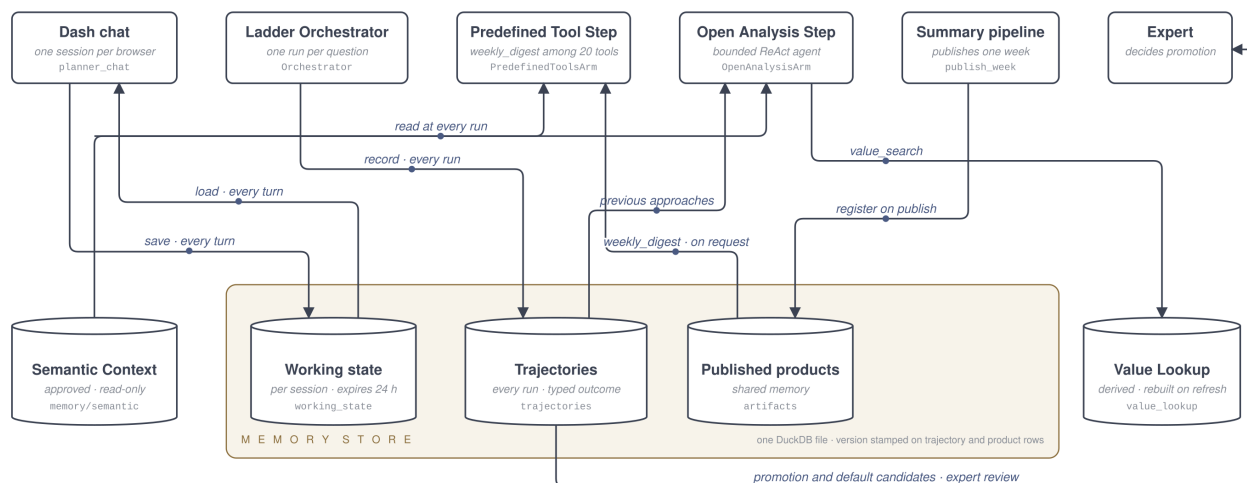


Figure 5.8 Memory access in the conversational system. Working state is loaded and saved each turn and expires independently. Trajectories record the final status and are retrieved only on escalation, for delivered answers with the same subject and current semantic version. These records are previous approaches, not verified answers. The pipeline registers published products, which the digest tool can read for a question about the week. The agent can search the value lookup to resolve names. Long-term memory is an expert-governed process rather than a separate store.

Experiential memory. At the end of each run, the orchestrator records a trajectory as described in Section 5.4.1. It includes a subject signature derived from the question, the answering step, escalation and clarification indicators, and the final status. The long-term process uses these fields.

On escalation, the orchestrator retrieves up to three recent trajectories with the same subject signature and current semantic version. Only trajectories whose final status is an answer are eligible. Rows recorded before the status field existed carry no status and are not eligible. Eligibility does not mean the earlier answer was confirmed against a reference. The retrieved records are previous approaches.

The escalation packet includes the retrieved questions, answering steps, and answer excerpts. Its instructions require the agent to remeasure every number when reusing an earlier approach or query. The clean tool path does not retrieve experience, and a failed memory read does not block an answer. The current store is shared across conversations. A per-user partition would require a corresponding table key.

Version matching. The memory store computes a version identifier from the paths and contents of semantic YAML files. Trajectory retrieval normally requires that identifier to match the current one. A YAML edit therefore changes retrieval eligibility, but does not prove that the new content was approved. Non-YAML files are outside this hash. Working-state expiry and whole-context approval are separate mechanisms.

Shared memory. The products table registers each published week's summary and PDF on a best-effort basis. The `weekly_digest` tool reads the newest registered summary for the requested week and includes the product handle in its answer. Its retrieval and missing-artifact behavior is described in Section 5.4.2. Published summaries are not automatically attached to every question.

The long-term process. Two queries over the trajectories make the governed process of Section 4.3.7 visible without adding a store. The first lists subjects that were repeatedly answered beyond the predefined tools with a delivered answer, the candidates for a new predefined tool. A candidate awaits expert review and is not a demonstrated solution. The second lists subjects that repeatedly needed a clarification, the candidates for a new default in the semantic context. Both lists appear in the memory view of the web application, and both stop at the expert. An approved candidate reaches the system only through the builder of Section 5.2 or through a new tool in the registry, and not through a write by an agent.

6 Results

The recorded runs show how the system connected business questions and weekly reporting to the same data foundation. The semantic investigation established how to read the data. The summary pipeline combined overlapping views into reports, and the conversational agent used tools and generated queries to answer questions. The examples also expose a limitation. An answer can agree with its query result while changing the measure that the user intended.

This chapter presents those findings through selected records and a constructed numerical example. The example makes the reporting calculations visible without disclosing company figures. Chapter 7 evaluates correctness, agent behavior, and model usage across the test cases.

6.1 Data and Execution Conditions

Company names and business figures are withheld. The numerical examples use fictional entities and constructed records for a reporting period called Week A. Their amounts are expressed in millions of euros. They illustrate the calculations but are not company measurements or altered transcripts of live runs.

The evidence has different origins, which determine what each example can show.

Table 6.1 Evidence sources and the claims they support.

Evidence	What was executed	What it supports
Semantic construction records	Investigations, stored rules, and recorded decisions on proposed corrections	How specific interpretations were established
Recorded weekly reports	Deterministic drill decisions, story selection, and template composition	The operation of the reporting procedure, not an agent advantage
Constructed numerical example	Selected production functions with prescribed candidates and planning actions	Clustering, scoring, guards, accounting, and recursive localization on disclosed inputs
Recorded synthetic drill decision	A live agent reviewed a constructed boundary case through the guarded decision interface	A permitted departure from the rule, not improved report quality
Recorded conversational runs	Live tool selection, tool execution, and Text-to-SQL analysis	The paths taken and the interpretations expressed in those attempts

The original run records retain their data, semantic-context, and lookup versions. Some conversational examples preceded a lookup refresh. They remain observations of that earlier configuration, not evidence of identical behavior after the refresh.

The chapter does not treat these examples as a representative sample of all questions or reporting periods. Their purpose is to explain the observed behavior and make the resulting report understandable.

6.2 Semantic-Context Construction

The construction records show that column descriptions alone did not settle how the data should be used. Reporting dates needed reconciliation against raw records, and familiar business terms could refer to different measures or account levels. The resulting rules supplied the interpretations used by both tasks.

6.2.1 Establishing Reporting Weeks and Fiscal Quarters

The time investigation established that the reporting label covered the full week, including its final day. This was not inferred from the weekday of the label alone. The investigation compared analytics values with sums over different raw-record windows.

The full-week window matched the recorded comparisons. A window restricted to working days did not match in some periods with weekend postings. Excluding those records could therefore change an answer even when the selected KPI and division were correct.

The resulting interpretation can be summarized as follows.

Reporting label	The final day of the reporting week
Included dates	The label and the six preceding days
Day-level question	Read the raw records
Missing weekly row	Do not assume a value of zero

This block summarizes the stored rule rather than reproducing its configuration syntax. A later check on refreshed data reported that the time invariants still held.

The result matters to both tasks. The summary must attribute a movement to the intended week, and a conversational query must use the same period when answering a follow-up. Agreement on a date label alone would not establish that consistency.

The time agent's investigation suggested that Q3 covered April to June, implying a fiscal year beginning in October. Expert review confirmed this interpretation and set the default for unqualified quarter expressions. The mapping and interpretation policy were then recorded in the semantic context. The following block is a readable summary of the approved rule.

Fiscal year starts	October 1
Fiscal Q1	October to December
Fiscal Q2	January to March
Fiscal Q3	April to June
Fiscal Q4	July to September
Unqualified quarter	Use the fiscal calendar
Explicit calendar	Use the calendar-year interpretation
Date-based tables	Translate the selected quarter into a date range
Answer	State which interpretation was used

Under this rule, “orders in Q3” refers to April to June, while “orders in calendar Q3” refers to July to September. If an unqualified quarter is paired with months that contradict the fiscal mapping, the policy requires the mismatch to be pointed out.

The raw cube's CRD labels use fiscal quarters as well, but describe requested delivery timing rather than when an order was booked.

6.2.2 Resolving Ambiguity During Construction

The construction process recorded corrections to business interpretations and also rejected an incorrect proposed correction. These outcomes show both the use of the semantic context and the need to check what is added to it.

Table 6.2 Examples of ambiguity handling during semantic-context construction.

Class	Problem found	Recorded decision
Question-level	“Orders” did not identify a measure or a particular order KPI	Store the inclusive order-intake default and require the answer to state the interpretation
Schema-level	“Customer” could refer to a top-level account or a billed entity	Record the default account level and distinguish the competing fields
Construction-level	A proposed edit confused booking time with requested delivery	Reject the edit using the earlier time evidence and keep the concepts separate

The orders and customer entries were adopted from question-level and schema-level challenge lessons, respectively. The following block is a readable summary of the approved glossary rules.

Orders, unspecified	Combined order-intake KPI (confirmed plus unconfirmed)
Alternative readings	Confirmed-only and unconfirmed-only
Answer	State the chosen basis and name the alternatives
Customer, executive	Top-level named-account grain
Answer	State the account level used
Billed-entity grain	Use when explicitly requested

These entries determine which measure and customer level the analytical request uses, rather than merely changing the wording of the answer.

The proposed column edit would have contradicted the earlier time investigation. Review rejected the incorrect definition and adopted a corrected rule for questions about when an order came in. The rule uses booking time and keeps requested delivery separate. A proposed lesson was therefore not accepted merely because a model produced it.

These records support specific corrections, not a general improvement rate. There is no matched before-and-after run for the full construction process. Section 6.4 follows the same interpretation choices into answering, where defaults, clarification, and query construction become visible.

6.3 Executive-Summary Pipeline

The recorded weekly reports combined several views of a movement without counting their shared records repeatedly. They also retained opposing movements and disclosed what remained outside the selected stories. Those reports used deterministic decisions. They illustrate the reporting procedure, not the contribution of a live drill or story-selection agent.

The constructed example makes the same reporting questions visible with public figures. Division Example has a current value of 275.0M against an expected value of 213.4M. Its net movement is therefore +61.6M. Confirmed orders contribute -60.0M and unconfirmed orders contribute +121.6M. The positive headline alone would hide that difference.

The following sections trace the example from concentration decisions to a short report. Candidate views and planning actions are prescribed for this illustration. The calculations are checked with the corresponding implementation functions, but the example is not a complete pipeline execution. Section 7.4.5 reports complete pipeline runs with live agent decisions on company weeks and compares them with their deterministic references.

6.3.1 Filtering and Concentration-Based Drill-Down

The concentration cases lead to different reporting choices on the constructed records. The amount floor is 3.08M, obtained from the larger of the fixed floor and five percent of the scope movement. Surprise is supporting evidence, not an additional hard filter.

Table 6.3 Concentration cases checked on constructed records, without a model.

Examination	Measured movement	Case result
Division Example by business line	Alpha +8.6M, Beta +65.0M, Gamma -12.0M	Case A identifies Beta as the leading child
Beta by customer	Pulsar +32.0M, Vega +27.0M, Aster +6.0M	Case B identifies a group of contributing children
Pulsar by process group	Thirty groups contribute equally to +32.0M	Case C finds no child meeting the share rule or amount floor

Beta exceeds the division's net movement because another business line moves in the opposite direction. Its share above one is therefore not an arithmetic error. Within Beta, no customer carries the leader share. Within Pulsar, each process group contributes about 1.07M, below the amount floor.

These examinations show why the report should not descend to the same depth everywhere. A leading child, a concentrated group, and a dispersed movement provide different evidence about the level at which to report. The case function produced the results in the table. No model selected an alternative in this check.

In recorded case Q036, gpt-5.6-luna added a child that narrowly missed the naming threshold. These are synthetic inputs from the controlled boundary experiment, not the Week A records above.

Child	Movement share	Lift	Surprise / full band width
Region 1	0.571	1.00	0.4
Region 2	0.384	1.00	0.4
Region 3	0.045	1.00	4.0
Naming threshold	0.05		
Region 3 amount	+3.6M, above the hard floor of 1.0M		
Rule recommendation	Pick Regions 1 and 2		
Agent proposal	Pick Regions 1, 2, and 3		
Guard outcome	Accepted unchanged, without fallback		

The agent's recorded reason cited Region 3's near-threshold share and its movement beyond the normal band. The supplied evidence placed it 3.5 full band widths beyond the nearest bound. Its lift was neutral. The agent therefore retained the child on surprise evidence without treating the naming threshold as a hard exclusion.

For RQ1, this record shows a permitted choice that differs from the rule. It does not establish that the added child improves the report. Section 7.4.2 reports the full controlled experiment, while Section 7.4.3 examines choices across real-data walks.

6.3.2 Same-Effect Clustering

Clustering combines compatible views, not just pairs of candidates. The constructed input contains twelve prescribed views. All 66 unordered pairs are measured, and complete-link grouping produces seven clusters. Three clusters have several members, while four remain single views.

One cluster contains Customer Comet, Comet within Region A, Product Family Amber, and Business Line Alpha. All six internal pairs pass the strict merge rule. A connected Value Chain A view remains outside this cluster because it does not pass that rule with every member.

Table 6.4 Selected pair measurements in the constructed clustering example.

Views compared	First ratio	Second ratio	Decision
Comet and Family Amber	1.0000	0.9511	Merge
Family Amber and Value Chain A	0.9511	1.0000	Merge
Comet and Value Chain A	0.9486	0.9486	Ambiguous, keep separate
Comet and Business Line Alpha	1.0000	2.9419	Merge
Pulsar and Business Line Beta	1.0000	0.4923	Containment, keep separate

The first three rows show why a chain of merging pairs is not enough. Family Amber passes with both Comet and Value Chain A, but those two views do not pass with each other. The final cluster therefore does not inherit a merge through an intermediate member.

The Comet–Alpha comparison also limits what the ratios mean. Alpha includes negative movement outside Comet, which reduces its net deviation. Its ratio can exceed one even though no record is counted twice in the pair measurement. These are signed-deviation ratios, not percentages of identical record sets.

Pulsar’s movement is contained within Beta, but accounts for only part of Beta’s movement. The classifier records containment without merging the two views. The relation remains available when the plan assigns their reporting roles.

Comet is the representative chosen among the equally wide views in this illustration. The remaining clusters represent Value Chain A, Pulsar, Vega, Nova, Lumen, and Beta. Seven clusters do not imply seven leading stories. Story selection still determines which findings lead and which provide context or remain unselected.

6.3.3 Impact Scoring and Story Selection

The broadest movement is not necessarily the strongest leading story. Beta has the largest measured deviation in the constructed example, but much of it is already represented by Pulsar and Vega. Its score reflects that concentration.

Table 6.5 Selected cluster scores in the constructed example. Scores are not reported business amounts.

Finding	Measured deviation	Lift	Cluster share	Score, divided by one million
Pulsar	+32.0M	2.30	1.000	41.11
Vega	+27.0M	3.44	1.000	39.13
Comet	+25.3M	1.49	1.000	28.49
Beta	+65.0M	2.48	0.092	7.88

The table shows selected findings, not the complete priority order. The score is a priority measure, not an amount reported to management. The prescribed plan selects Pulsar, Vega, and Comet as stories, retains Nova and Lumen as counters, and attaches Beta as context. A fourth leading story is selected from the remainder in the next section.

The same planning interface also checks an intentionally invalid action. An instruction to leave the protected Nova counter unreported is refused. A subsequent instruction to protect it is accepted.

Scripted proposal	Leave the Nova counter in the remainder
Guard outcome	Refused because protected counters must be shown
Next proposal	Protect Nova as a counter
Executed outcome	Counter retained in the plan

This is a scripted guard check, not a model mistake or an example of agent reasoning. It shows the boundary within which a story-selection agent must work. Section 7.4.4 compares recorded live and deterministic selections on the same candidate inputs.

6.3.4 Remainder and Accounting

The remainder check can expose a movement that the candidate list did not include. The recorded company example contained such a finding. In the constructed illustration, Customer Meteor is deliberately absent from the candidate input but contributes +12.0M in the records left after the initial plan.

The tool lists Meteor and an opposing Eclipse movement. The prescribed plan selects Meteor as its fourth leading story. Eclipse remains disclosed within the negative remainder. This tests selection from a measured remainder, not whether a live drill would miss or discover Meteor.

Ownership then separates the amounts that may be added in the report. Beta measures +65.0M before ownership, but Pulsar and Vega already account for +59.0M within it. Beta therefore contributes +6.0M of additional context, not another +65.0M on top of those stories.

Table 6.6 Assigned components and remainder in the constructed example.

Report component	Assigned deviation
Pulsar	+32.0M
Vega	+27.0M
Comet	+25.3M
Meteor, selected from the remainder	+12.0M
Nova counter	-18.0M
Lumen counter	-14.0M
Additional Beta context	+6.0M
Total assigned to reported components	+70.3M
Positive remainder	+1.3M
Negative remainder	-10.0M
Scope movement	+61.6M

The assigned movement and the signed remainder reconcile to the scope movement. In this example, $70.3M + 1.3M - 10.0M$ equals 61.6M. The assigned total exceeds the net scope movement because an opposing amount remains outside the report components. It should not be described as more than complete coverage.

The check sums the deviations of assigned cells and compares them with the reported components. It uses the production ownership function, so it tests the resulting assignments and arithmetic rather than providing an independent implementation of the ownership method.

6.3.5 Story Localization

The recursive localization routine produces different explanations for Comet, Pulsar, and Beta. It was run unchanged on the constructed Week A records through an in-memory data provider. The production attribution function allocated the supplied expected amounts before each split. Those amounts and historical allocation weights are constructed, not estimated from a company time series.

Selected returned lines are shown below. The audit ladder is omitted, while the wording and indentation of these explanation lines are retained.

```

Comet
+24.0M traced to Value Chain A (95% of this route)

Pulsar
almost entirely in Business Line Beta / Product Family Copper / Value Chain C /
Region B
spread across 30 Process Group values -- diffuse from here

Beta
+59.0M traced to Value Chain C (91% of the story)
split on Customer:
  Pulsar +32.0M
  Vega +27.0M

```

Comet's route retains its full +25.3M movement. Value Chain A contributes +24.0M, or 94.9 percent, so the routine treats it as a lead rather than a location meeting the 0.95 threshold. The displayed percentage rounds to 95 percent, but the decision uses the unrounded share. Pulsar is located within one combined route. Its movement is then spread across thirty process groups, and the routine stops rather than naming a leading group.

Beta shows why the explanation basis differs from the assigned amount. Its full series moves by +65.0M, while only +6.0M remains assigned as context after ownership. Localization uses the former as its root denominator and the latter to set the amount floor, here 1.0M. Value Chain C carries +59.0M, or 90.8 percent of the full series. Inside that slice, Pulsar and Vega contribute +32.0M and +27.0M. Pulsar therefore accounts for 54.2 percent of the slice but 49.2 percent of the full Beta series. Neither percentage uses the assigned +6.0M as its denominator.

The returned records retain the parent deviations and confirm these distinctions. This check exercises recursive selection and wording, but not warehouse extraction or the optional analytics-layer references appended to explanation leaves.

6.3.6 Composition and Reported Output

The checked amounts can be presented as a short report without exposing each intermediate candidate. The formatted excerpt below retains the headline, two selected stories, and related context from the constructed Week A report. Its explanation paths, like those in Appendix B, were prescribed for the rendering illustration. Section 6.3.5 separately shows output from the localization routine. Neither illustration is a live agent-generated report.

Division Example · Week A — report excerpt

Order intake is 275.0M against an expected 213.4M, a deviation of +61.6M. Confirmed orders are 60.0M below expectation, while unconfirmed orders are 121.6M above.

Customer Pulsar — +32.0M above expected

- The movement is within Business Line Beta, Product Family Copper, and Value Chain C.
 - It is spread evenly across 30 process groups. No single group stands out.
- **Related context.** Business Line Beta contributes an additional +6.0M. Its full series movement is +65.0M, of which Pulsar contributes +32.0M (49.2 percent).

Customer Comet — +25.3M above expected

- The full movement is within Business Line Alpha, Product Family Amber, and Region A.
 - Value Chain A contributes +24.0M, or 94.9 percent of Comet's +25.3M movement.

The context note keeps Beta’s additional assigned movement separate from the full series used for explanation. Neither explanation establishes a business cause. Appendix B provides the complete rendered Week A report and two further constructed examples with business-line, product-family, regional, and combined story definitions.

The weekly examples support the reporting part of the main research question by showing how overlapping views become an accountable summary. For RQ1, Section 7.4.3 compares agent and rule-based drill decisions, and Section 7.4.4 compares story selections. Those comparisons show where choices changed, while improvement in management usefulness remains unassessed.

6.4 Context-Grounded Conversational Agent

The live conversational records show the system choosing between predefined analysis and a generated query. They also show the interpretation work inside those paths. The model selected tools, resolved names, applied context rules, and sometimes asked for clarification or refused a request. The examples below describe recorded behavior without disclosing the original business figures.

Adapted questions use the fictional names and period introduced earlier. Any numerical illustration is calculated separately on the constructed records and is not presented as a live answer to those records.

Chapter 7 compares the SOMA-inspired sampling-and-selection method with the ladder and direct ReAct method, while distinguishing its earlier implementation version.

6.4.1 Progressive Disclosure and a Predefined Tool

For a question requesting one weekly KPI value, the selector chose `get_value`. The answering agent received that tool together with the three core tools. It resolved the requested entity before making the value lookup.

Adapted question — What were total confirmed orders for Division Example in Week A?

Selected analytical tool	<code>get_value</code>
Core tools also exposed	<code>describe_semantics, resolve_entity, list_entities</code>
Recorded execution order	<code>resolve_entity -> get_value -> final answer</code>

Selection, exposure, and the executed calls were retained in the record. The catalog wording was re-constructed from the corresponding code because the run did not retain it. The answering agent therefore received the selected tool definitions rather than the full analytical tool set, but this example does not measure the saving from that choice.

The constructed records give 76.5M for the same type of confirmed-orders lookup. This figure illustrates the requested measure only. It is not the amount returned in the historical live run.

For RQ3, the trace identifies the work delegated to a predefined tool. The model still chose the tool and its inputs, while the tool supplied the result. Whether this reduces model usage without reducing correctness requires the matched comparison in Chapter 7.

6.4.2 Open Analysis Through a ReAct Loop

A quantity question required analysis outside the predefined tool definitions. The selector found no suitable tool, and the orchestrator entered the Open Analysis Step directly. This differed from escalating after an unsuccessful tool attempt.

Adapted question — How many pieces of confirmed orders did Division Example book in Week A?

The recorded agent first read the semantic overview, then searched for the division value, and then executed a query. It used the observations from those calls before producing an answer.

Table 6.7 The recorded ReAct sequence, with confidential observations withheld.

Recorded step	Evidence used or produced
Read the semantic overview	Quantity field, KPI filter, and time interpretation
Search the stored division value	Matching literal and field
Execute a read-only query	Quantity and supporting row count
Answer	Requested measure, reporting period, and interpretation

This was a ReAct-style execution, not a fixed pipeline imposed on every question. The agent selected its next action after receiving the preceding observation. No separate runtime judge reviewed the answer.

The following author-written query illustrates the calculation on the constructed dataset. It was executed in an in-memory database, not generated by the live agent. The dates are fictional and define Week A for this example.

```
SELECT SUM(pieces) AS pieces, COUNT(*) AS rows
FROM example_orders
WHERE division = 'Example' AND component = 'confirmed'
AND CAST(booked_on AS DATE)
BETWEEN DATE '2030-02-04' AND DATE '2030-02-10'
```

The query returns 1,650,000 pieces across 39 constructed rows. An independent sum over those same input records matches the quantity. That checks this illustration's calculation, not the correctness of the historical model-generated query.

The live record supports a narrower finding. The agent connected the business measure to a quantity field, grounded the division, and used the booking period rather than the requested-delivery period. For RQ2, those interpretation choices are part of answering the question, not merely formatting the returned number.

6.4.3 Applying the Ambiguity Rules

The recorded questions used the interpretations established during construction. Their main result is the meaning selected before the calculation, rather than the size of the returned amount.

Table 6.8 Adapted questions and the interpretations expressed in recorded answers.

Adapted question	Ambiguity	Recorded interpretation
How many orders did Division Example receive in Week A?	"Orders" does not identify the measure	Inclusive order intake in euros, with the default stated and other readings distinguished
What did Customer Orbit order from Division Example in Week A?	A customer name can refer to different account levels	The top-level named account, with that basis stated
How did Division Example's confirmed orders change in the fiscal quarter?	Quarter boundaries and the relevant date field must be chosen	The context-defined fiscal period and booking-time series, not the requested-delivery bucket

The first question was answered through a disclosed default rather than clarification. The schema example resolved a business name to an account level. The quarter example combined the calendar rule with the time meaning needed to construct the request.

These cases connect Section 6.2 to RQ2. Stored context influenced which question was actually computed. Agreement between the answer and the returned observation remains a separate check from whether the selected interpretation was appropriate.

6.4.4 Clarification and Continued Conversation

The tool step did not answer every ambiguous question with a default. In one recorded comparison, it asked the user to specify the metric and period before performing the analysis. The adapted exchange below preserves that decision without reproducing the original entities or amounts.

User	Compare Customer Comet and Customer Orbit.
Agent	Which metric should be compared, and for which period?
User	Confirmed orders in euros for Division Example in Week A.
Next step	Resolve the entities and execute the comparison.

The continuation combined the original question with the reply. The recorded run then resolved the entities, performed comparison calls, and returned an answer. Its account-level bases differed between the entities and were stated in the response. The example shows a completed clarification path, not proof that every comparison uses equivalent populations.

The exchange was selected after trying several candidate questions. It therefore cannot establish how often the system clarifies. Another recorded case exposed a policy mismatch. For “sales,” the answer presented both order intake and revenue instead of asking which reading the user intended. The retained reference expected clarification. Providing useful information did not satisfy that requirement. Chapter 7 applies the recorded expectation when grading the final run.

For RQ2, the distinction is substantive. A system may recognize several meanings yet still choose the wrong action for resolving them. The evaluation must consider the interpretation and outcome, not only whether some returned figures are correct.

6.4.5 Escalation After an Attempted Tool Step

A separate quantity question first entered the Predefined Tool Step. The selector chose `drill_down`, but the answering step reported that it could not complete the requested analysis before executing a tool call. That outcome caused the orchestrator to escalate.

Request	Customer contributions measured in pieces
Selected tool	<code>drill_down</code>
Step outcome	<code>cannot_complete</code>
Next step	Open Analysis Step
Recorded result	Answer from a query over the raw data

Unlike Section 6.4.2, the catalog did not reject the tool path at entry. The attempted step identified the mismatch and returned an explicit outcome. The orchestrator followed that outcome rule without a separate model review.

The record shows recovery from an unsuitable initial selection. It does not establish that such recoveries are frequent, inexpensive, or always successful.

6.4.6 Refusal for a Question Outside the Data

A market-share question illustrates refusal in the final company condition of Section 7.2.1. The catalog found no fitting tool, so the question went directly to the Open Analysis Step. The step read the semantic overview and searched for competitor and market-share terms before returning a refusal. The following block summarizes the recorded path with an adapted question.

Request	Division Example's market share against competitors in the requested month
Catalog	No fitting predefined tool
Open Analysis	Semantic overview and two term lookups, then refusal
Delivered text	Fixed data-scope message

The orchestrator replaced the step's specific explanation with the generic data-scope message. Here, that message correctly identified competitor information as unavailable. The fixed message also omitted the step's offer to report the division's own order intake or revenue. Section 7.5.4 discusses the limitations of this replacement.

6.4.7 Published Summaries and Follow-Up Questions

The conversational system retrieved a published weekly report through `weekly_digest`. It named that source and used the published findings rather than recomputing the summary. This made the reporting output available for a management question through Shared Memory.

However, a later follow-up did not preserve the headline measure. Its analysis switched from inclusive order intake to confirmed orders and used a narrower business-line slice for the customer. The component figures were consistent with their sources, but the answer was not a continuation of the same KPI and scope.

The constructed figures illustrate why that distinction matters. The report's inclusive movement is +61.6M, while the confirmed component is -60.0M. Both can be correct because the unconfirmed component contributes +121.6M. An answer about confirmed orders alone does not explain the inclusive increase unless it connects that component back to the original question.

This is a limitation observed in the recorded conversation, not an error introduced into a fictional agent transcript. Supplying the earlier turn and retrieving a report did not by themselves preserve the intended analytical basis. The follow-up needed continuity of KPI, scope, and period, not only access to earlier text.

6.4.8 Observed Outcomes

The conversational records show that a business question could be translated into a predefined tool request or a query over raw records. The model did more than phrase a result. It selected a tool, resolved a name, applied an interpretation rule, asked for clarification, or refused a question outside the data. The escalation example also shows a selected tool path reporting its limitation so that another step could attempt the question.

For RQ2, these examples make interpretation observable, but they also show why numerical agreement is not enough. The "sales" response recognized alternative meanings without following the expected clarification behavior. The summary follow-up returned a component analysis while changing the original KPI and scope. These outcomes limit what can be claimed from answers that match their own tool observations. These two interpretation failures occurred in the earlier recorded configuration. Chapter 7 assesses the revised system, including a follow-up that preserves the intended KPI.

For RQ3, the traces identify two different execution paths. The predefined path uses an existing calculation after selecting its inputs. The open path must construct the analytical request and execute a query. The examples establish that both paths operate, but their different questions do not provide a controlled comparison of correctness or model usage. Chapter 7 makes that comparison on matched cases.

Together with the weekly results, the records show how the same semantic foundation supports reporting and subsequent questions. The synthetic drill decision illustrates a permitted departure from the rule. The broader comparisons for RQ1 follow in Section 7.4.3 and Section 7.4.4. These records do not establish improved management decisions or general applicability across organizations.

7 Evaluation

Chapter 6 illustrated the system through constructed examples and recorded execution paths. This chapter assesses numerical consistency, agent decisions, answer correctness, and model usage on the saved experiments. Execution and correctness are assessed separately. A completed run can still answer the wrong question, and a permitted reporting choice need not be the most useful one.

7.1 Evaluation Scope

The evaluation follows the three research questions in the Introduction. It examines the information produced for top-level management, not changes in management decisions.

Table 7.1 Research questions and the evidence used to assess them.

Research question	Evidence	What the evidence can establish
RQ1, agent-guided reporting	Controlled drill decisions, repeated scenarios, complete hierarchy traversals, and paired story selections	Permitted choices, changes to collected and presented findings, and constraint compliance
RQ2, business-language interpretation	Reference queries, whole-answer assessment, outcome rules, and conversation chains	Whether the delivered response satisfies the requested measure, scope, period, and outcome
RQ3, analytical tools and model usage	Common questions answered by the ladder, direct ReAct method, and retained SOMA-inspired baseline	Correctness and resource differences under the recorded configurations

Semantic-context construction supports these investigations. Its evidence concerns the origin and checking of selected interpretation rules. RQ1 concerns reporting choices within defined constraints, including differences in collected findings, story selection, and the remaining movement. Expert assessment of management usefulness is outside the measured scope.

7.2 Data, Configurations, and Measures

7.2.1 Experimental Conditions

Company records support the numerical checks and analytical questions. Constructed inputs expose rare threshold and failure conditions. Models were accessed through the configured model application programming interface (API). Unlike Chapter 6's illustrative figures, the counts and resource measurements here are observed experimental results. Company identities, financial values, and complete reports remain in the private evidence.

Table 7.2 distinguishes the configured strong summary model from the models selected for the experiments. Model names use a consistent hyphenated form.

Table 7.2 Configured strong summary model and experimental model settings. The configuration-only row is not an additional experiment.

Configuration or experiment	Data and connection	Model	Purpose
Strong summary role (configuration only)	Application configuration	claude-opus-5	Story selection and optional phrasing
Controlled drill and repeat	Constructed observations, model API	gpt-5.6-luna	Boundary decisions and paired agreement
Complete real-data drills	Retained warehouse data, company model API	gpt-5.6-luna	Traversal and candidate selection
Three paired story runs	Fixed drill traces, company model API	claude-sonnet-5	Story selection with unchanged phrasing
Final ladder and direct agent	Retained DuckDB warehouse, company model API	claude-sonnet-5	Execution, correctness, and usage
Retained SOMA baseline	Earlier run, retained DuckDB warehouse and company model API	claude-sonnet-5	Sampling-and-selection comparison
Value-lookup diagnostic	Read-only lookup, separate model API	claude-sonnet-5	Name matching and delivered outcomes
Additional summary comparison	Retained warehouse data, company model API	gpt-5.6-luna drill, claude-opus-5 story selection	Live drill and story selection with template phrasing

The three paired story runs selected `claude-sonnet-5` instead of the configured `claude-opus-5` and retained template phrasing. Company runs record the Sonnet alias as `claudesonnet5`, shown as `claude-sonnet-5` in the table for consistency. The lookup diagnostic records the hyphenated spelling directly. These gateway aliases do not establish immutable provider versions.

The additional summary comparison used live drill and story-selection agents with template phrasing. The reference used the same analytical tools, accepted the drill recommendations without model calls, and used deterministic story selection. Both configurations used the same analytical thresholds, budgets, and localization settings. The live drill additionally received four-week reference statistics to help interpret its evidence. Template phrasing means that no phrasing pass is included in this experiment.

The company connection identifies the model endpoint. It does not make the conversational measurements an Impala performance test, because those runs used the retained DuckDB warehouse.

The final conversational condition is `company-final-20260913T141712Z`. It records source-file hashes, semantic-context version `ef76d76e3daa`, lookup and summary hashes, and three concurrent workers. The retained SOMA condition is `company-revised-20260912-r4`. Its earlier prompts and execution code are not relabeled as the final system. Common questions, reference requirements, grading, and token-accounting rules support the comparison, while this version difference remains explicit.

The conversational evaluation used conversation history and published summaries, without retrieving approaches from previously completed analyses.

7.2.2 Development and Final Assessment

No model weights were trained. Development changed tools, prompts, semantic guidance, and response handling. Targeted validation runs checked discovered failures and previously successful cases. The final enterprise run then reassessed the declared question set. Earlier attempts remain preserved and are not replaced by successful retries from another connection.

The recurring question forms were known during development, and some questions directly informed corrections. The final run is therefore a revised-system assessment, not an untouched test of unseen

questions. Earlier small holdout and API recovery runs remain development evidence and are not added to its denominator. Repeated questions, related scenario families, and historical weeks also share structure.

7.2.3 Measures

Execution reliability concerns whether an attempt reaches an answer, clarification, or refusal without a terminal technical error or an exhausted limit. Correctness concerns whether that outcome satisfies the request. A successful clarification or refusal can be correct, while a delivered answer can be incorrect.

Usage comprises input tokens, output tokens, model calls, and elapsed time, including recorded retries. Cached input and reasoning output are not added a second time to their respective token totals. Missing usage is not treated as zero. Monetary charges are secondary because model prices and cache accounting differ. The conversational comparison has usage records for every included call. Timings reflect the recorded three-worker execution rather than an isolated latency benchmark.

7.3 Evidence for the Semantic Context

The time investigation compared analytics values with raw-record windows across four divisions, five KPIs, and 35 weeks. All 700 calendar-week sums matched their analytics values. Restricting the windows to weekdays changed 46 comparisons. This supports the stored weekly-label interpretation on the inspected data, not an assumption that weekend dates always have or never have bookings.

The time agent’s investigation suggested the fiscal calendar, which expert review confirmed. The approved context also defined how quarter expressions should be interpreted. A question exposing competing meanings of orders led to a glossary rule. A proposed correction that confused booking time with requested delivery was rejected using the time evidence. Section 6.2 illustrates these distinct sources of knowledge.

The construction records contain one batch round each for question-level and schema-level challenges. The construction class has a separate four-question interactive record, not a completed batch.

Table 7.3 Recorded construction challenge batches. Handled verdicts are model assessments, not evaluation accuracy scores.

Class	Recorded attempts	Recorded rounds	Marked handled	New lessons
Question-level	101	1	34	22
Schema-level	100	1	9	46

The handled verdicts came from the construction-time model assessment with the July 2026 engine. New lessons were proposals, not necessarily approved corrections. These counts describe construction activity, not answer accuracy.

The review log records eight glossary promotions and six further approved changes, including interpretation rules and a generator-prompt fix. It also records one rejected note and nine column-meaning edits held back pending a rerun. No comparable before-and-after assessment establishes a quantified benefit from context construction as a whole.

7.4 Executive-Summary Evaluation

The numerical checks assess the reporting procedure first. The two agent roles are then examined separately through controlled drill decisions, complete real-data walks, and story selection on fixed candidates.

7.4.1 Numerical Consistency and Analytical Routes

Ten earlier summaries across three divisions and four reporting dates passed all 14 implemented numerical and reporting checks. Their drill accepted tool recommendations without model calls, and story selection used the deterministic procedure. These results assess the shared reporting calculations and remain attached to those saved reports.

The additional summary comparison of Section 7.4.5 passed all 110 fast numerical and reporting checks across its ten saved reports. Nine reports had completed live story selection; the remaining saved report used fallback placement. All nine completed live-agent reports therefore passed the eleven applied checks. These covered headline and component values against independent warehouse queries, accounting reconciliation, recorded story amounts, lead percentages, and per-card rendering.

Table 7.4 Numerical and reporting checks on the ten earlier reference summaries, generated with deterministic drill and story selection.

Check group	Evidence checked	Recorded result
Headline and components	Separately written queries of analytics root and component rows	All ten summaries passed
Ownership and reconciliation	Production cell assignments, separate owner sums, and unassigned remainder	All ten summaries passed
Explanation percentages	Amounts divided by their recorded parent deviations	All 62 lead percentages passed
Rendering	Names and amounts within their own HTML story cards	All ten summaries passed

Ownership checks inspect assignments exported by a fresh production execution. Each cell must belong to the population, match its owner, and not qualify for an earlier story in the ownership order. Separate sums check each published story and the remainder within one base currency unit. This is a consistency check using the production allocation functions, not an independent reimplement or validation of the estimated normal values assigned to raw slices.

Route replay checked 76 story records. The 59 applicable records passed, while 17 had no applicable route to replay. The checks compared published lines with replayed filters and parent-slice measurements. They extend the percentage checks to the analytical paths, but do not assess whether those paths are useful to a manager.

The time-window audit found that raw attribution excludes the Sunday reporting date, although the analytics week includes it. No Sunday postings affected the ten checked summaries, so their amounts were unchanged. The retained time investigation records month-end Sunday postings elsewhere. The current cutoff can therefore omit records belonging to the analytics week.

7.4.2 Controlled Drill Decisions and Repetition

The controlled experiment contains 100 node–dimension scenarios in ten families. It includes clear leaders, threshold boundaries, opposing movements, flat parents, unavailable reference statistics, and prohibited targets. These are single decisions, not complete hierarchy traversals. Each scenario passes through the production session, driver, and execution guards.

Hard constraints were recorded before inference. Near a discretionary boundary, retaining the recommendation and making a permitted exception can both be acceptable. The test does not define every override as a success.

Table 7.5 Two live attempts on each of 100 controlled drill scenarios. Retry work is included in usage.

Measure	First run	Repeat
Completed attempts	100	100
Override proposals	36	36
Proposals changed by execution guards	2	2
Executed choices differing from the rule	34	34
Final driver fallback	0	0
Schema-rejected responses, subsequently retried	1	10
Provider inference requests	302	310
Input tokens	365,491	376,625
Output tokens	31,789	33,488

All 200 final executions satisfied the checked hard constraints. Two below-floor inclusion proposals occurred in each run. Their displayed amounts rounded to the floor, although their unrounded amounts were lower. The guards removed those targets. This demonstrates successful enforcement, not correct initial eligibility judgment.

The repeat used identical inputs, reference evidence, implementation, and model alias. Actions, target sets, and counter sets agreed in 98 of the 100 pairs. The two differences occurred in high-surprise boundary scenarios where both choices were permitted. The equal totals of 34 departures therefore do not imply identical decisions.

The agent sometimes changed its selection when stronger supporting signals accompanied a boundary case, while clear leaders retained the rule. This demonstrates use of the available discretion. It neither establishes improved reporting nor isolates the effect of reference statistics, since this experiment did not remove those statistics in a matched condition.

Some recorded reasons still confused surprise measures or interpreted a flat-parent share incorrectly. Decision validity and explanation quality are distinct. No expert-quality score is assigned to all 200 reasons.

7.4.3 Real-Data Hierarchy Traversals

The real-data experiment selected 100 division-week inputs, 25 for each of four divisions. The current semantic context was applied retrospectively to the retained warehouse. An initial local batch stopped at its evaluation spending allowance. A separately recorded enterprise run subsequently completed all 100 selected walks.

Table 7.6 Observed real-data drill results in two separate run conditions.

Measure	Initial local batch	Enterprise run
Completed walks	79	100
Evaluation-budget interruptions	3	0
Selected walks not started	18	0
Decisions in completed walks	2,844	3,710
Executed choices differing from the rule	10	14
Walks with a changed candidate set	9	13

The enterprise record contains 258 boundary decisions, no final driver fallback, no recorded guard intervention, and no production-limit stop. All 100 traversal audits passed. The 14 departures changed candidate sets in 13 walks. Most real-data decisions retained the recommendation, unlike the deliberately boundary-heavy constructed set.

The complete enterprise batch used 15,477,106 input tokens and 1,062,929 output tokens across 11,464 provider requests. One drill decision can require several model calls. The conditions are not

pooled as 179 distinct inputs, and local counts are not scaled to 100. Neither batch ran the entire summary pipeline for each walk. Changed candidates establish an effect on collection, not better final summaries.

7.4.4 Paired Story Selection

Three company-gateway experiments isolated story selection. Each pair reused the same deterministic drill trace and measured clusters. Template phrasing remained unchanged, so wording could not conceal the effect of selection.

Table 7.7 Final story counts and planning outcomes for three fixed-input pairs. A–C are anonymous experiment identifiers.

Paired input	Deterministic stories	Agent-selected stories	Refused planning proposals	Final fallback
A	7	4	0	No
B	13	7	9	No
C	8	6	9	No

The agent completed all three plans and produced fewer final cards in each. In A, regional views replaced several customer stories, and the absolute net movement outside the cards decreased. In B and C, fewer cards left a larger absolute net movement in context and remainder combined. All six artifacts reconciled to their scope movement within the accounting tolerance.

These changes show where the agent affected the report, but not a uniform improvement. Signed story shares can exceed 100 percent or become negative when opposing movements offset one another. Closeness to 100 percent is not a quality score. The positive and negative remainder and the roles of omitted findings also matter.

Rejected proposals in B and C included unsupported refinement relations and unselected parents. The agent continued from the rejection observations. Final ownership could also change the proposed plan. In A, a selected remainder movement was demoted after its owned amount fell below the floor. The table counts final cards, not intermediate selections.

The runs required 6, 86, and 24 model calls respectively. Their input-token counts were 14,198, 1,665,223, and 158,390, with 1,226, 21,889, and 4,911 output tokens. A completed plan need not be inexpensive. The variation also supports workload-dependent allowances rather than assuming the same number of actions for every cluster set.

7.4.5 Agent Choices in Complete Summary Runs

The additional experiment compares nine completed live-agent reports with the preserved deterministic reports for the same division-week inputs. All nine completed live drill and live story selection, with template phrasing. This comparison examines changes in collection and report organization together. A tenth input is not included, because its story selection did not complete and its report used deterministic placement.

The deterministic procedure applies fixed placement rules in a score-based order, with refinements placed after their parent. The agent chooses reporting actions using the measured evidence and the drill's notes, subject to the same placement guards. Both procedures make grouping decisions; the difference is how those choices are made.

The live drill made 233 decisions across these nine walks. It recorded no overrides or drill failures. Candidate views and deviations matched their deterministic references, as did the recorded candidate, merge, and cluster counts. Accepting the recommendation was still a live agent decision: the drill used 699 recorded model calls.

The drill supplied different carry-forward notes in seven of the nine cases. These notes can inform later selection, so the experiment is not an isolated test of the story-selection model on completely identical

inputs. It compares the two configurations over matching measured candidates. The three fixed-input pairs of Section 7.4.4 remain the separate experiment that isolates story selection.

Main stories and opposing-movement counters are counted separately. Both appear as cards.

Table 7.8 Report composition for nine completed live-agent summaries against their deterministic references. Each label identifies one division-week, compared using the deterministic and agent configurations; Summary 8 did not complete live selection.

Input	Main stories, reference → agent	Counter cards, reference → agent	All cards, reference → agent	Refused planning proposals
Summary 1	4 → 3	0 → 1	4 → 4	0
Summary 2	2 → 2	3 → 2	5 → 4	0
Summary 3	6 → 7	2 → 1	8 → 8	4
Summary 4	5 → 5	1 → 1	6 → 6	2
Summary 5	4 → 3	4 → 5	8 → 8	1
Summary 6	2 → 2	7 → 6	9 → 8	0
Summary 7	5 → 2	2 → 2	7 → 4	0
Summary 9	7 → 6	1 → 2	8 → 8	6
Summary 10	6 → 5	2 → 1	8 → 6	1

Four reports contained fewer cards and five contained the same number. The balance between main stories and counters also changed, including in reports whose length stayed the same. In Summary 7, regional views became part of the report, while several customer stories and a product-family story no longer appeared as separate cards. The report decreased from seven cards to four. This shows a change in presentation, not evidence that the shorter report was more useful.

Across the nine plans, 190 of 204 proposals were accepted and 14 were refused. Every included plan completed without fallback. A refused proposal was an intermediate correction, not a failed report. Accepted proposals are not identical to final reporting roles, because ownership and amount-floor checks also affect the published result.

The remainder is movement not assigned to the selected stories or context. The table reports directions of change rather than confidential financial amounts. Negative remainder refers to its absolute magnitude.

Table 7.9 Direction of change of the positive and negative remainder magnitudes for the same nine inputs.

Input	Positive remainder magnitude	Negative remainder magnitude
Summary 1	Increased	Increased
Summary 2	Increased	Increased
Summary 3	Decreased	Decreased
Summary 4	Decreased	Decreased
Summary 5	Decreased	Decreased
Summary 6	Increased	Increased
Summary 7	Decreased	Decreased
Summary 9	Increased	Increased
Summary 10	Decreased	Decreased

Both remainder magnitudes decreased in five cases and increased in four. Summary 7 and Summary 10 became shorter while leaving less positive and negative movement in the remainder. Summary 2 and Summary 6 also became shorter, but both remainder magnitudes increased. Report length and remaining movement therefore describe different effects of selection.

Neither explained share nor net remainder is a quality score. Positive and negative movements can offset each other, so a small net remainder can hide larger movements of both signs. Changes in context and opposing-movement visibility also need consideration. These comparisons show what the agent changed, but do not establish which report a manager would prefer.

For identical ordered inputs and parameters, the deterministic decision rules prescribe the same choices. The parity tests check that the mock-driver paths agree with the deterministic baselines on the tested drill and story-selection inputs. Separately, the repeated controlled drill experiment of Section 7.4.2 produced the same executed actions, targets, and counters in 98 of 100 scenario pairs. That result measures repeat agreement for individual drill decisions, not for complete summaries.

The recorded model usage covers the nine completed executions, including their intermediate proposals. It is not the total resource use of every attempt made during the experiment.

Table 7.10 Recorded model usage of the nine completed live-agent summaries, including intermediate proposals.

Stage	Configured model	Input tokens	Output tokens	Recorded work
Drill	gpt-5.6-luna	940,499	64,199	699 model calls; 233 decisions
Story selection	claude-opus-5	1,695,659	52,871	204 planning turns; 14 refused proposals
Combined	Both models	2,636,158	117,070	Nine completed reports

The completed reports used an average of about 292,906 input tokens and 13,008 output tokens each. Usage varied across reports: Summary 9 alone accounted for 54.2 percent of the story-selection input tokens. Repeated context and intermediate proposals contribute to these totals; they do not represent distinct records read.

The deterministic decision configuration had no model-token cost for these stages, but database queries, pair measurement, ownership, and publishing still required computation. Agent decisions added model work, even where the drill retained the same candidates. Input and output tokens are reported separately by model because their totals alone do not establish comparable monetary cost.

Durations below are measured from the start to the finish of each run, not as the sum of stage timers. Both rows cover the same nine division-week inputs.

Table 7.11 Observed start-to-finish duration per report for the same nine inputs under both configurations.

Configuration	Observed duration per report	Median
Deterministic reference	0.3–2.7 minutes	0.4 minutes
Live drill and story selection	3.6–18.1 minutes	5.9 minutes

The live-agent runs took longer in the recorded executions. The configurations ran under different execution conditions, including different degrees of parallelism and shared gateway use. These figures are observed runtimes, not an isolated measurement of agent overhead. The clustering algorithm is shared, but its measured runtime also depends on the execution environment.

For scheduled weekly reporting, the practical timing question is whether generation finishes before publication. This experiment records execution time but does not establish a publication deadline or a service-level guarantee.

7.4.6 Readability and Management Usefulness

Informal supervisor feedback requested clearer analytical routes and a more visible agent contribution. It informed report revisions, but was not a rating study of the revised outputs. No structured usefulness ratings are available. Numerical checks, route replay, and paired selection records cannot replace that assessment.

7.5 Conversational Evaluation

7.5.1 Questions and References

Each method answered the same 33 common questions once. These comprise 23 original questions and ten supplementary questions covering lookups, trends, comparisons, rankings, complete populations, and business-language interpretation. Six common questions received two further repetitions. The ladder also received ten control or conversation attempts, including four two-turn chains. These are excluded from the three-method comparison because SOMA does not conduct clarification exchanges.

References specify the KPI, unit, scope, period, grouping, and required outcome. Separately written read-only SQL provides the numerical results. Full-response review checks entity–value associations and volunteered claims against these references and recorded observations. The final review also reran reference and supplementary queries where required. Reference requirements were not changed to accept the observed response.

Automatic matching screens for discrepancies rather than assigning final correctness. A date written in words can fail a literal match, while a matching headline can conceal omitted rows. Clarification and refusal follow their recorded expectations. This evidence-based review is not independent domain-expert validation. Assistance used in preparing and checking it is disclosed in the front matter.

7.5.2 Execution Reliability

The final enterprise condition completed 100 attempts containing 104 user turns. None ended in a technical error or an exhausted limit. This establishes operational completion on these inputs, not correctness.

Table 7.12 Conversational execution reliability. Completion does not establish correctness.

Method	Attempts	Completed	Execution errors	Limit reached
Full ladder, final version	55	55	0	0
Direct ReAct, final version	45	45	0	0
SOMA, retained version	45	44	1	0

The ladder total includes its additional control and conversation tests. SOMA's retained failure was a database execution failure and remains in its denominator. It is not removed because it was unrelated to SQL interpretation.

All 630 recorded requests in the final enterprise condition returned HTTP 200. However, 98 invalid final-tool submissions occurred across 92 user turns before completion. Missing required fields caused recorded rejections. The loops recovered within their allowances, so these are internal protocol failures rather than terminal errors. Of 111 answering-step outcomes, 110 used the final-answer tool and one used the recorded unstructured-text fallback. Zero terminal errors does not mean every first response satisfied the schema.

7.5.3 Answer Correctness

A fully correct response satisfies the requested content and outcome without a material unsupported addition. A partial response provides useful correct content but omits required information or adds a misleading claim. An incorrect response gives the wrong requested result or fails the required business outcome. Unresolved remains available when the evidence or recorded interpretation allowance does not settle the case.

Unsupported causal explanations are substantive errors even around correct figures. A disclosed pre-view remains partial when the question requires a complete result. A dropped sign or wrong entity is a correctness issue, not merely wording. Presentation defects such as stray tags are recorded separately unless they change the meaning.

Table 7.13 Full-response correctness on the common questions. Unassessable execution failures remain in the end-to-end denominator.

Method, 33 common questions	Fully correct	Partial	Incorrect	Unresolved	No assessable response
Full ladder, final version	25	7	1	0	0
Direct ReAct, final version	22	11	0	0	0
SOMA, retained version	24	5	2	1	1

End-to-end full correctness was 25 of 33 for the ladder, 22 of 33 for direct ReAct, and 24 of 33 for the retained SOMA condition. Unresolved and failed cases remain in the denominator. The SOMA fiscal-quarter answer is unresolved because it discloses a shifted weekly window, while the existing weekly-reading allowance does not settle that specific boundary choice. The table preserves that uncertainty rather than selecting a favorable interpretation.

On the original 23 questions, the final ladder produced 20 correct, two partial, and one incorrect response. Direct ReAct produced 15 correct and eight partial responses. The ladder's incorrect answer omitted the leading customer although the tool had returned it. Other failures concerned incomplete breakdowns, unsupported causes for negative values, and confusion between stored outlier flags and control-band membership. Correct queries and correct headline amounts did not prevent these errors.

The ten ladder-only attempts produced six fully correct, one partial, and three incorrect outcomes when a conversation was graded by its weakest turn. All four unavailable-information cases ended in the required refusal. Remaining errors concerned required clarification and a customer total assembled from incomplete business-line coverage. A published-summary follow-up preserved the requested KPI in the final run, showing that the earlier interpretation failure did not persist in that case.

7.5.4 Value-Lookup Diagnostic

A separate model-API experiment tested eight name-identification questions against the read-only lookup. It used Sonnet, isolated memory, and recorded prompts and lookup versions. The endpoint was checked before inference. Its lookup version differs from the final enterprise condition, so the results are not pooled with that condition.

Table 7.14 Eight single-pass value-lookup diagnostics. Company names are withheld.

Input condition	Lookup and agent behavior	Delivered response
Exact stored name	Correct match, related account distinguished	Acceptable
Misspelling 1	Correct stored spelling selected	Acceptable, correction implicit
Misspelling 2	Correct stored spelling selected	Acceptable, correction explicit
Case-normalized name	Correct stored spelling selected	Acceptable
Ambiguous name 1	One of several close accounts chosen without support	Not acceptable
Ambiguous name 2	Alternatives listed and user asked to choose	Acceptable
Invented name 1	No confident match, no substitution	Generic refusal lost the specific reason
Invented name 2	No confident match, no substitution	Generic refusal lost the specific reason

The lookup returned the expected candidate evidence in all eight cases. Seven agent decisions were supported, but only five delivered responses were acceptable. For the invented names, the Open Analysis Step retained the relevant reason, while the orchestrator replaced it with an unrelated generic data-scope refusal. This was a delivery defect, not an incorrect name match.

The ambiguous inputs produced different decisions. One prompted a user-facing question, though its typed status was answer, while the other selected a close account definitively. Similarity scores are spelling evidence, not calibrated probabilities of identity. These eight single-pass cases diagnose failure modes rather than estimate general accuracy. All bypassed the Predefined Tool Step, so they do not establish its live resolver behavior. The experiment used 27 calls, 187,757 input tokens, and 7,016 output tokens, with one recovered invalid final submission and no gateway error.

7.6 Model Usage and Method Comparison

7.6.1 Comparison Conditions

The ladder and direct ReAct method share the final company-gateway condition. Their difference is the tool-first selection and dispatch path. SOMA supplies the retained sampling-and-selection comparison, using three candidates and at most five probes. Its earlier implementation is identified in the same tables, not treated as an unchanged-system control.

All comparisons below use the first attempt on each of the 33 common questions. Extra repetitions and ladder-only exchanges are excluded. The recorded Sonnet alias and warehouse are shared, but implementation and prompt differences limit causal claims about SOMA. No targeted recovery replaces a failed attempt.

7.6.2 Tokens, Calls, and Time

Table 7.15 Model usage and elapsed time on the common questions. The SOMA-inspired row retains the earlier enterprise baseline.

Method	Attempts	Median input tokens	Median output tokens	Median calls	Median time, s
Full ladder	33	78,169	2,127	6	33.4
Direct ReAct	33	100,043	1,907	5	30.9
Retained SOMA	33	169,240	14,375	13	163.3

Input-token interquartile ranges were 39,350–132,721 for the ladder, 97,555–125,303 for direct ReAct, and 168,205–170,547 for SOMA. The ladder used 3,153,055 input tokens in total, compared with 3,939,175 for direct ReAct, a reduction of 20.0 percent on this question mix. It used more output tokens, 100,964 against 80,627, and more calls, 214 against 192. Lower input use did not imply fewer calls or lower median elapsed time.

The retained SOMA attempts used 5,633,841 input tokens, 467,184 output tokens, and 431 calls. Sampling and selection required more recorded work, without a higher fully correct count than the revised ladder on this set. This describes the retained configuration, not a general ranking of SOMA-SQL against ReAct.

The path split explains the ladder's input usage. Each row compares the same questions across the ladder and direct ReAct. Groups follow the ladder's observed path, not randomized tool coverage.

Table 7.16 Input usage on matched questions, grouped by the ladder's observed path.

Observed ladder path	Questions	Ladder input tokens	Direct ReAct input tokens
Predefined Tool Step only	16	589,802	1,672,495
Direct entry to Open Analysis	13	1,869,515	1,799,143
Tool step followed by escalation	4	693,738	467,537

Tool-only cases used 64.7 percent fewer input tokens. Direct-entry cases used 3.9 percent more, while trying tools before escalation used 48.4 percent more. Tool suitability determines whether the ladder saves work or adds it.

A secondary comparison retained only the 20 common questions answered fully correctly by both final methods. The ladder used 1,522,688 input tokens against 2,192,256 for direct ReAct, 30.5 percent fewer. Output tokens were 49,828 against 40,682. This subset supports savings for supported answers, but does not replace the all-attempt result or count partial answers as successes. Progressive disclosure is part of the ladder and was not isolated in an ablation.

7.6.3 Repeated Conversational Attempts

Six questions received three attempts per method. The additional repetitions are not new questions. Requested numerical results agreed within tolerance across all three attempts for each question under both final methods. Whole-answer grades were less stable because supporting statements varied. Four of the six ladder questions and two of the six direct-agent questions were fully correct in all three attempts.

In the retained SOMA condition, three questions were fully correct across all three attempts. Two others contained a partial response. On the remaining intersection question, one repetition stopped at an absent analytics route even though raw records could answer it. Small within-condition checks demonstrate variability, not a general repeatability rate for future questions.

7.7 Research-Question Assessment and Limitations

For RQ1, some walks in the earlier drill experiment collected different candidates, while the nine complete summary comparisons retained the same measured candidates. Report composition still changed: four reports contained fewer cards, and both remainder magnitudes decreased in five cases but increased in four. Every completed live-agent report passed the eleven applied numerical and reporting checks. Agent selection changed how the measured findings were organized into reports, with additional model usage and longer observed execution times. Shorter reports did not consistently leave less movement in the remainder. These results show the consequences of the reporting choices, but do not establish which configuration produces more useful reports for management.

For RQ2, the final system completed its attempts but did not always satisfy the business request. Remaining problems concerned incomplete populations, unsupported explanations, outcome policy, and interpretation of tool evidence. The lookup diagnostic further separated candidate retrieval, entity choice, and delivery wording. RQ2 cannot be answered by execution success alone.

For RQ3, the tool-first ladder reduced input tokens on this set, especially when a predefined calculation completed the question. Selection and unsuccessful tool attempts could instead add work. Correctness and separate input/output measurements qualify this finding. The retained SOMA baseline broadens the comparison, but its older code condition prevents a controlled claim that the method alone caused every difference.

The study concerns one organization and many question forms known during development. Constructed scenarios emphasize boundaries, historical weeks share structure, and model aliases can change behind the gateway. These limits preclude a general success rate for unseen organizations or management questions.

Numerical checks establish consistency of the evaluated artifacts, not causal explanations or the accuracy of allocated normal values. No independent study validates the allocation model, no end-to-end sensitivity sweep establishes optimal clustering or score parameters, and no structured expert assessment establishes report quality. These questions remain open despite observed constraint compliance and completed executions.

8 Conclusion and Future Work

8.1 Conclusion

This thesis developed an agentic LLM system that connects weekly KPI reporting with business-language analysis for top-level management. Both tasks use the same semantic definitions and underlying data. The reporting procedure identifies recorded changes and their concentrations, while conversation supports further questions about the reports and the data. Tools calculate the evidence and enforce constraints. Agents interpret requests and propose reporting choices within those constraints. Ten summaries generated with deterministic decisions passed all 14 implemented numerical and reporting checks. The system therefore addresses what changed and where the movement occurred, but does not establish its business causes.

RQ1. Agent-guided reporting. Agents affected collection and report organization in different experiments. In the completed enterprise batch reported in Section 7.4.3, 14 departures from the drill recommendation changed candidate sets in 13 of 100 walks. All 200 controlled boundary executions satisfied the checked hard constraints, including after guards corrected below-floor proposals. The three fixed-input story comparisons showed changes in grouping. In the additional complete-summary comparison, the nine live walks retained the same measured candidates, but four reports contained fewer cards after agent selection. All nine passed the eleven applied numerical and reporting checks.

The agent's contribution was choosing among permitted actions while tools retained control of measurements and constraints. Agent selection changed how the measured findings were organized into reports, with additional model usage and longer observed execution times. Shorter reports did not consistently leave less movement in the remainder. These results show the consequences of the reporting choices, but do not establish which configuration produces more useful reports for management.

RQ2. Business-language interpretation. The system answered questions through predefined tools and generated SQL, including a follow-up that preserved the published summary's requested KPI. Every attempt in the final enterprise configuration described in Section 7.2 completed without a terminal execution failure. However, the ladder fully satisfied 25 of the 33 common questions, with seven partial and one incorrect response. Remaining problems included incomplete answers, unsupported explanations around correct figures, and clarification failures in the separate conversation tests. A small diagnostic of eight single-pass cases on a separate model API further indicated weaknesses in entity choice and refusal delivery. The distinction matters for executive use because successful data access does not ensure that the delivered answer satisfies the business request. During semantic-context construction, review also rejected a proposed rule that confused booking time with requested delivery.

RQ3. Analytical tools and model usage. The ladder used 20.0 percent fewer input tokens than direct ReAct analysis across the same 33 questions. It produced more fully correct responses, 25 against 22, but used more output tokens and model calls. Tools saved work when they completed the request. An unsuccessful tool attempt before escalation instead added work. Tool coverage and selection therefore determined the observed benefit, rather than tool use alone.

The retained SOMA-inspired implementation produced 24 fully correct responses and required more recorded input, output, and calls. It generates and assesses several SQL candidates and can issue diagnostic probes. Passing extensive semantic context through these stages also adds repeated input. This explains where work occurs, but the experiment did not isolate each source of overhead. The earlier implementation and prompt differences also limit a method-only comparison. Progressive disclosure was not evaluated separately from the ladder.

A practical lesson from development was to make execution and validation feedback available to the agent. In the final conversational run, the loops recovered from invalid final-tool submissions within their execution limits. During development, the same feedback path proved useful when new failure types appeared. A rejected submission could therefore become another observation rather than ending the attempt. This supported recovery from protocol failures, but did not establish the correctness of the delivered answer.

The main contribution is the connection between traceable reporting and business-language analysis over shared semantic definitions. The experiments show where agent decisions changed the collected findings, the story selection, and the analytical path. Evidence from one organization and development-informed questions does not establish general applicability. Within that scope, the thesis provides an implemented and evaluated division of work between deterministic measurement and constrained interpretation.

8.2 Future Work

Analytical reliability requires an explicit contract between a tool result and the interpretation it supports. For example, results should state population coverage, aggregation, periods, units, and truncation. Outcome handling should preserve clarification and refusal reasons, reduce invalid final-tool submissions, and distinguish any unstructured-text fallback. These changes need regression tests because clearer interfaces alone cannot ensure correct interpretation.

The allocation of expected values to raw slices also needs validation, including its behavior when historical records are missing. Sensitivity studies should examine how the working thresholds and scoring factors affect the selected findings.

Future work could test whether lower-cost models make the SOMA-inspired approach more economical while preserving answer correctness. Generating and comparing several candidates may compensate for a less capable model, but this requires evaluation. Matched experiments should hold the questions, candidate and probe limits, and semantic information fixed while varying the model. Correctness, input and output tokens, elapsed time, and monetary cost should be reported separately.

Semantic search could be compared with the current spelling-based value lookup for name grounding, and retrieved context with directly supplied definitions and reports. Matched comparisons should assess answer correctness and model usage while keeping essential interpretation rules available. The existing lookup-refresh procedure provides a starting point, but an additional semantic index introduces maintenance costs and freshness requirements. Database queries would still provide current numerical answers.

Expert comparisons of reports generated from matched inputs should examine whether the selected stories preserve important developments and make their analytical routes easier to follow. Main stories, supporting detail, counters, and the remainder should be assessed together. The remainder is the movement not assigned to stories, counters, or context; it may still contain important deviations that deserve further analysis. Expert review could assess whether these deviations should be reported separately to top-level management.

Drill decisions and their explanations should be assessed separately, with reference statistics varied while current observations remain fixed. Further experiments could compare calling the drill agent at every decision with calling it only near flagged boundaries, measuring changes in candidate collection, the notes passed to story selection, and model usage. Matched experiments could also compare lower-cost and stronger story-selection models on the same measured clusters and supplied notes, assessing report composition and model usage together without assuming that the stronger model produces the more useful report. Evaluation should also extend to unseen question forms, later periods, and another organization. Recurring uncovered questions could motivate new predefined tools, followed by coverage and regression tests. A separate comparison could isolate the contribution of progressive disclosure.

User integration should be evaluated through tasks that connect the summary to further analysis. Participants could locate a movement, inspect its supporting route, and ask a follow-up question. Task completion, interpretation errors, and clarification outcomes would provide evidence beyond interface screen-

shots. A longitudinal study with management users could examine whether reusing previous analytical approaches helps with recurring reporting questions. It could also identify repeated requests that justify new predefined tools or revisions to the semantic context. Broader deployment also requires authenticated access, authorization, isolated user controls, and concurrent-use tests. This would assess whether the implemented analysis can be used reliably in the intended management workflow.

A Tool Inventory

The 20 predefined analytical tools of Section 4.3.3. The selector reads the one-line descriptions of the 17 question-specific tools; the three core tools are added automatically to every selection. The wording is prompt-facing and is retained from the implementations, not rewritten as thesis prose. The final-answer tool is a response interface, not one of these analytical tools.

Table A.1 The predefined analytical tools. The one-line definitions are generated from the tool implementations.

Tool	What it answers	Core
describe_semantics	Describe the available KPIs, axes (perspectives), routes and the measure.	yes
resolve_entity	Resolve a business name to the qualified series names the data uses.	yes
list_entities	List stored analytics series for a KPI/axis, not all raw-data entities.	yes
get_value	Look up a single series' value for a KPI (point lookup).	
get_outliers	Return flagged outliers, most severe first (by outlier_score).	
get_bound_breaches	Find control-bound breaches, NOT values below their expected mean; filter by route/scope.	
rank_series	Rank stored analytics rows by value or deviation, not raw-population or period totals.	
get_trend	Read weekly history; with explicit weeks compute their average and below-average weeks.	
compare	Compare two series, or one series across two dates, for a KPI.	
compare_periods	Compare SUMS for one series across two periods, not window averages.	
drill_down	Concentration-guided drill: find WHERE a KPI's move comes from.	
scan_level	HORIZONTAL (breadth-first) scan: the movers across a whole depth level.	
weekly_digest	Read the published weekly report, optionally for a division; never recompute a substitute.	
kpi_split	Break one slice into ALL its KPIs — how firm is the number?	
flag_history	Flags over TIME: who is flagged often, streaks, direction flips.	
entity_profile	One entity's full state: every KPI, plus WHERE its money sits.	
movers	Big moves REGARDLESS of flags — including ones the flagging missed.	
explain_flag	Explain ONE node's flag in plain words: unusual, or normal wiggle?	
new_series	Entities appearing for the FIRST time in the latest week.	
kpi_relation	Two KPIs side by side — do they move together or against each other?	

B Constructed Executive-Summary Examples

These reports use fictional names and constructed values. Week A continues the Chapter 6 example. Weeks B and C illustrate a decrease and offsetting movements. They are not live-agent results.

Division Example · Week A

Constructed figures · millions of euros
Deviation from the expected level

Order intake is 275.0M against an expected 213.4M, a deviation of +61.6M. Confirmed orders are 60.0M below expectation, while unconfirmed orders are 121.6M above.

61.6M

TOTAL, ABOVE
EXPECTED

-60.0M

CONFIRMED ORDERS

+121.6M

UNCONFIRMED
ORDERS

4 + 2

STORIES + OPPOSING
MOVEMENTS

WHAT DROVE THE WEEK

Customer Pulsar division / customer

32.0M above expected

Reported contribution **+32.0M** · Breakdown for the full group **+32.0M**

- The movement is within Business Line Beta, Product Family Copper and Value Chain C.
 - It is spread evenly across 30 process groups. No single group stands out.

↳ Related context: Business Line Beta. Additional contribution +6.0M, full-group movement +65.0M. Pulsar contributes +32.0M (49.2% of +65.0M).

Customer Vega division / customer

27.0M above expected

Reported contribution **+27.0M** · Breakdown for the full group **+27.0M**

- The movement is within Business Line Beta, Product Family Silver and Value Chain C.

Customer Comet division / customer

25.3M above expected

Reported contribution **+25.3M** · Breakdown for the full group **+25.3M**

- The full movement is within Business Line Alpha, Product Family Amber and Region A.
- Value Chain A contributes +24.0M (94.9% of the +25.3M above).

Customer Meteor division / customer

12.0M above expected

Reported contribution **+12.0M** · Breakdown for the full group **+12.0M**

- The remainder check identified this movement within Business Line Gamma and Product Family Gold.

MOVEMENTS IN THE OPPOSITE DIRECTION

Customer Nova division / customer

Reported contribution **-18.0M** · Breakdown for the full group **-18.0M**

- The decrease is within Business Line Alpha, Product Family Violet and Value Chain B.

18.0M below expected

Customer Lumen division / customer

Reported contribution **-14.0M** · Breakdown for the full group **-14.0M**

- The decrease is within Business Line Gamma, Product Family Jade and Value Chain D.

14.0M below expected

Not picked as a story. Movements outside the stories total -2.7M. This includes +6.0M of additional context shown above and a net remainder of -8.7M (+1.3M of increases and -10.0M of decreases).

Order intake is 160.0M against an expected 216.0M, a deviation of -56.0M. Confirmed orders are 44.0M below expectation, while unconfirmed orders are 12.0M below.

56.0M

TOTAL, BELOW
EXPECTED

-44.0M

CONFIRMED ORDERS

-12.0M

UNCONFIRMED
ORDERS

2 + 2

STORIES + OPPOSING
MOVEMENTS

WHAT DROVE THE WEEK

Business Line Beta division / business line

46.0M below expected

Reported contribution **-46.0M** · Breakdown for the full group **-36.0M**

- The breakdown uses the full business line before the separately reported movements are removed.
- Product Family Copper contributes -40.0M (111.1% of the -36.0M above).
 - Region A contributes -28.0M (70.0% of the -40.0M above).
 - Value Chain A contributes -24.0M (85.7% of the -28.0M above).

↳ Already counted in Region C: -4.0M.

↳ Already counted in Customer Vega: +14.0M.

Business Line Gamma · Product Family Gold

24.0M below expected

division / business line / product family

Reported contribution **-24.0M** · Breakdown for the full group **-24.0M**

- The decrease is spread across Customers Meteor and Aster in Region D.
- Customer Meteor contributes -16.0M (66.7% of the -24.0M above).

MOVEMENTS IN THE OPPOSITE DIRECTION

Customer Vega division / customer

14.0M above expected

Reported contribution **+14.0M** · Breakdown for the full group **+14.0M**

- The increase is within Business Line Beta, Product Family Silver and Region B.

Region C division / region

4.0M above expected

Reported contribution **+4.0M** · Breakdown for the full group **+4.0M**

- Customer Nova contributes +8.0M, while Customer Comet contributes -4.0M.

Not picked as a story. Movements outside the stories total -4.0M. This consists of a net remainder of -4.0M (+3.0M of increases and -7.0M of decreases).

Order intake is 209.0M against an expected 205.0M, a deviation of +4.0M. Confirmed orders are 18.9M below expectation, while unconfirmed orders are 22.9M above. The small net increase combines larger movements in both directions.

4.0M

TOTAL, ABOVE
EXPECTED

-18.9M

CONFIRMED ORDERS

+22.9M

UNCONFIRMED
ORDERS

2 + 2

STORIES + OPPOSING
MOVEMENTS

WHAT DROVE THE WEEK

Product Family Silver division / product family

36.0M above expected

Reported contribution **+36.0M** · Breakdown for the full group **+36.0M**

- The family includes movements from Customers Vega and Comet in different business lines.
- Business Line Beta contributes +24.0M (66.7% of the +36.0M above).

Region A division / region

34.0M above expected

Reported contribution **+34.0M** · Breakdown for the full group **+34.0M**

- The regional increase combines movements from Customers Comet and Orbit.
- Business Line Alpha contributes +28.0M (82.4% of the +34.0M above).
 - Product Family Amber contributes +21.0M (75.0% of the +28.0M above).
 - Value Chain A contributes +18.0M (85.7% of the +21.0M above).

MOVEMENTS IN THE OPPOSITE DIRECTION

Business Line Alpha · Customer Nova

38.0M below expected

division / business line / customer

Reported contribution **-38.0M** · Breakdown for the full group **-38.0M**

- The decrease is within Product Family Violet, Value Chain B and Region B.

Value Chain D · Region E division / value chain / region

20.0M below expected

Reported contribution **-20.0M** · Breakdown for the full group **-20.0M**

- The decrease is within Business Line Gamma, Product Family Jade and Customer Lumen.

Not picked as a story. Movements outside the stories total -8.0M. This consists of a net remainder of -8.0M (+0.0M of increases and -8.0M of decreases).

C Large-Format Figures

This appendix repeats four figures of the main text in landscape format so that their labels can be read at full size. The diagrams are unchanged.

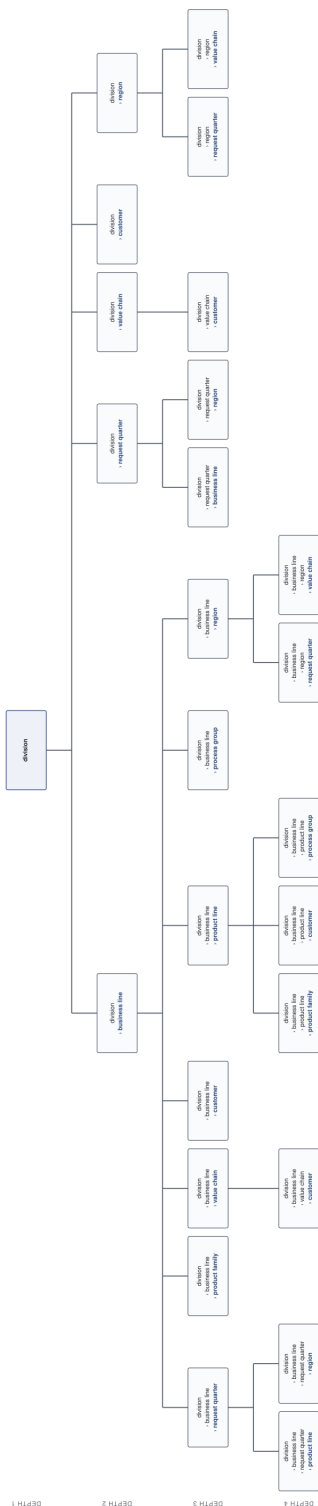


Figure C.1 Larger version of the unified route hierarchy in Figure 4.2.

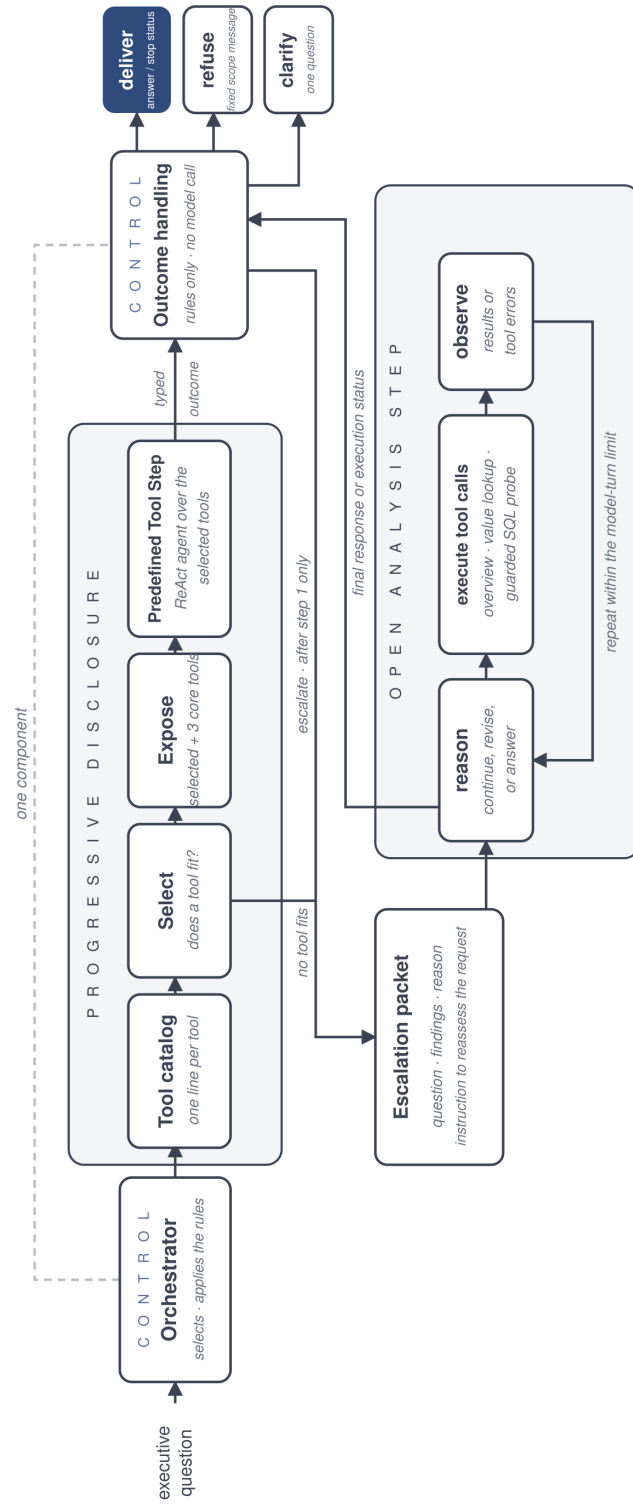


Figure C.2 Larger version of the main ladder path in Figure 4.11.

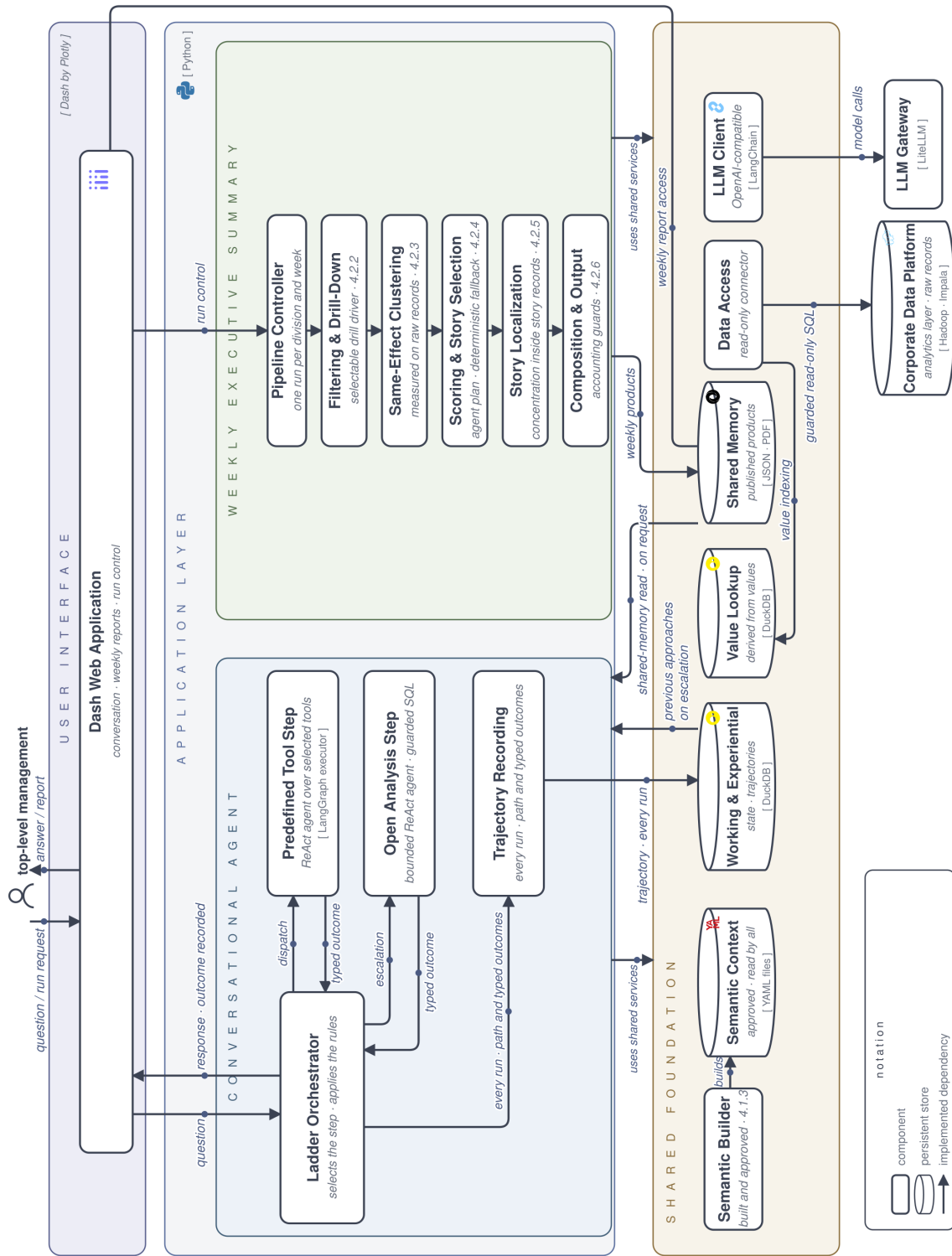


Figure C.3 Larger version of the layered system architecture in Figure 5.1.

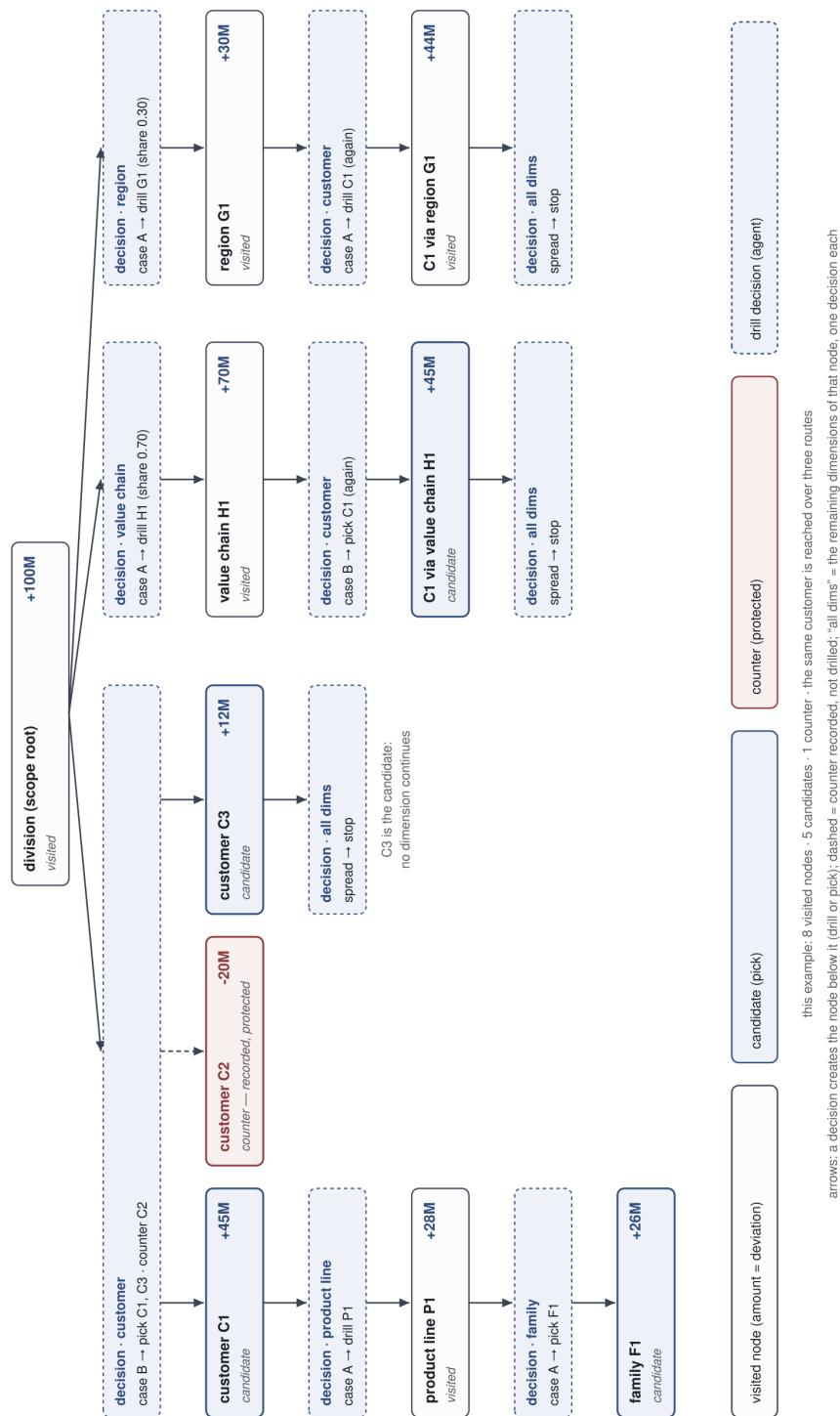


Figure C.4 Larger version of the drill decisions of one walk in Figure 5.3.

D Application Interface

The following screenshots show the existing Dash components with the constructed example of Chapter 6. All business names and financial figures are fictional. Conversation messages, tool observations, and context metadata were supplied for the screenshots rather than obtained from live model runs. The operator view is idle. A visible banner identifies this documentation setup.

The captures distinguish report reading and conversation from developer controls. They document the interface, not answer correctness, usability, or management benefit. Selected developer cards are shown without unrelated configuration panels. The download buttons are retained interface elements, but their callbacks are not connected in this prototype.

INTERFACE DEMONSTRATION · CONSTRUCTED DATA · NO LIVE AGENT RUN

Executive Copilot

Weekly reporting and business questions

Clean

Full trace

...

CE

CEO

Weekly Summaries

Browse published weekly reports.

Week A

Constructed example · not a live run

View

LATEST

EARLIER WEEKS · MEMORY

Week A

EXAMPLE

Illustrative reporting period

Conversation

Weekly summary

Developer

Full ladder

Direct Text-to-SQL

clarify

ask

never

Business context and analytical data

Open the latest published summary

AI

Which measure and reporting period should I compare?

Compare Customer Comet and Customer Orbit.

AI

Total order intake in euros for Week A, Division Example.

For Division Example in Week A, total order intake was **76.0M EUR for Customer Comet** and **8.0M EUR for Customer Orbit**.
These figures combine confirmed and unconfirmed order intake. The comparison uses the same reporting period and customer dimension for both customers.
Staged conversation using the constructed example, not a recorded model answer.

Ask about the data...

forget

Figure D.1 Conversation view with a staged clarification exchange. The same panel presents the question, clarification, and subsequent response. The example illustrates their display, not a measured successful interaction.

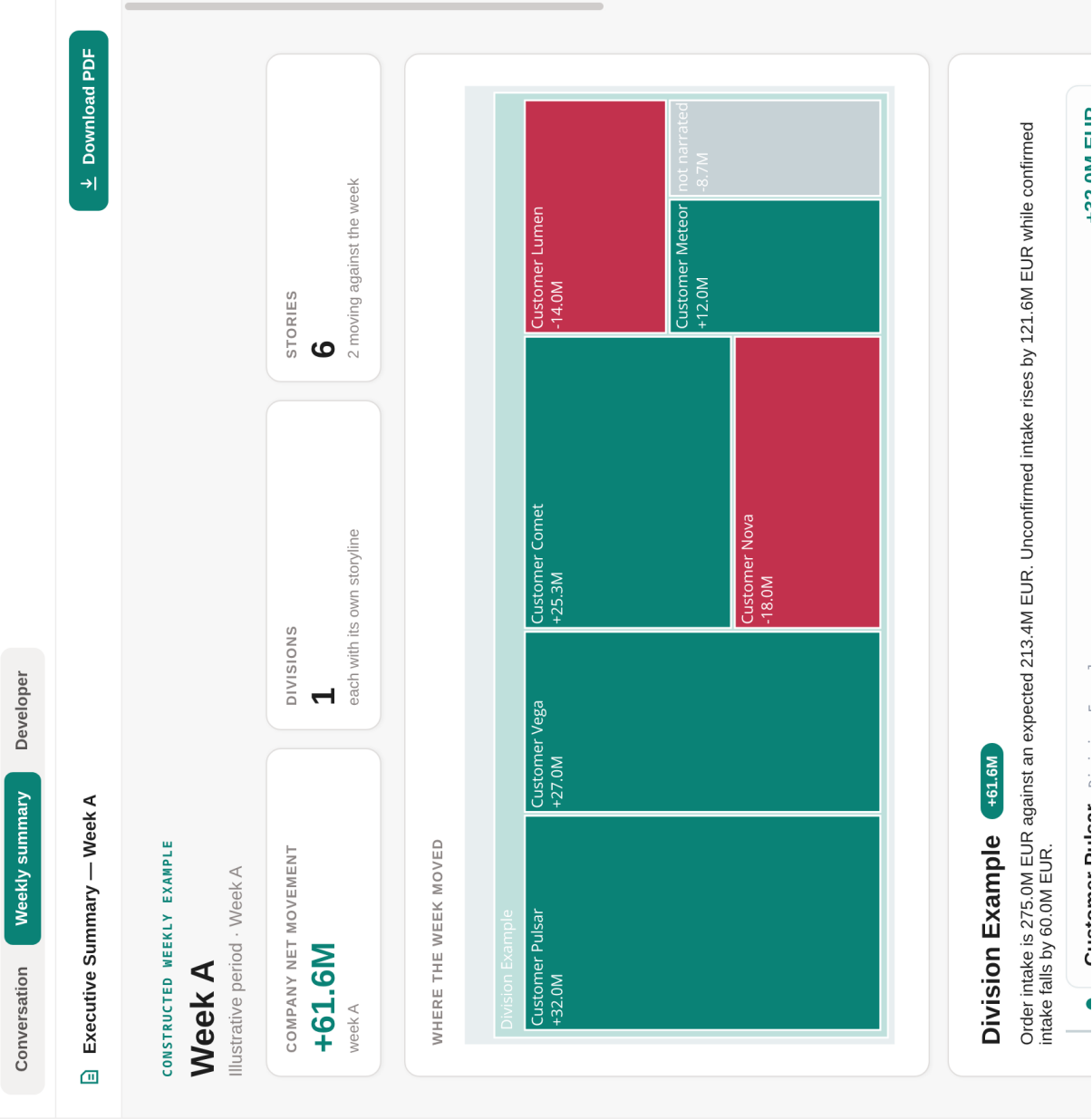


Figure D.2 Weekly-report overview using the constructed example. The library remains visible beside the headline and story overview. Colors distinguish positive and opposing movements. The figure is a rendering of supplied report content, not evidence of agent selection.

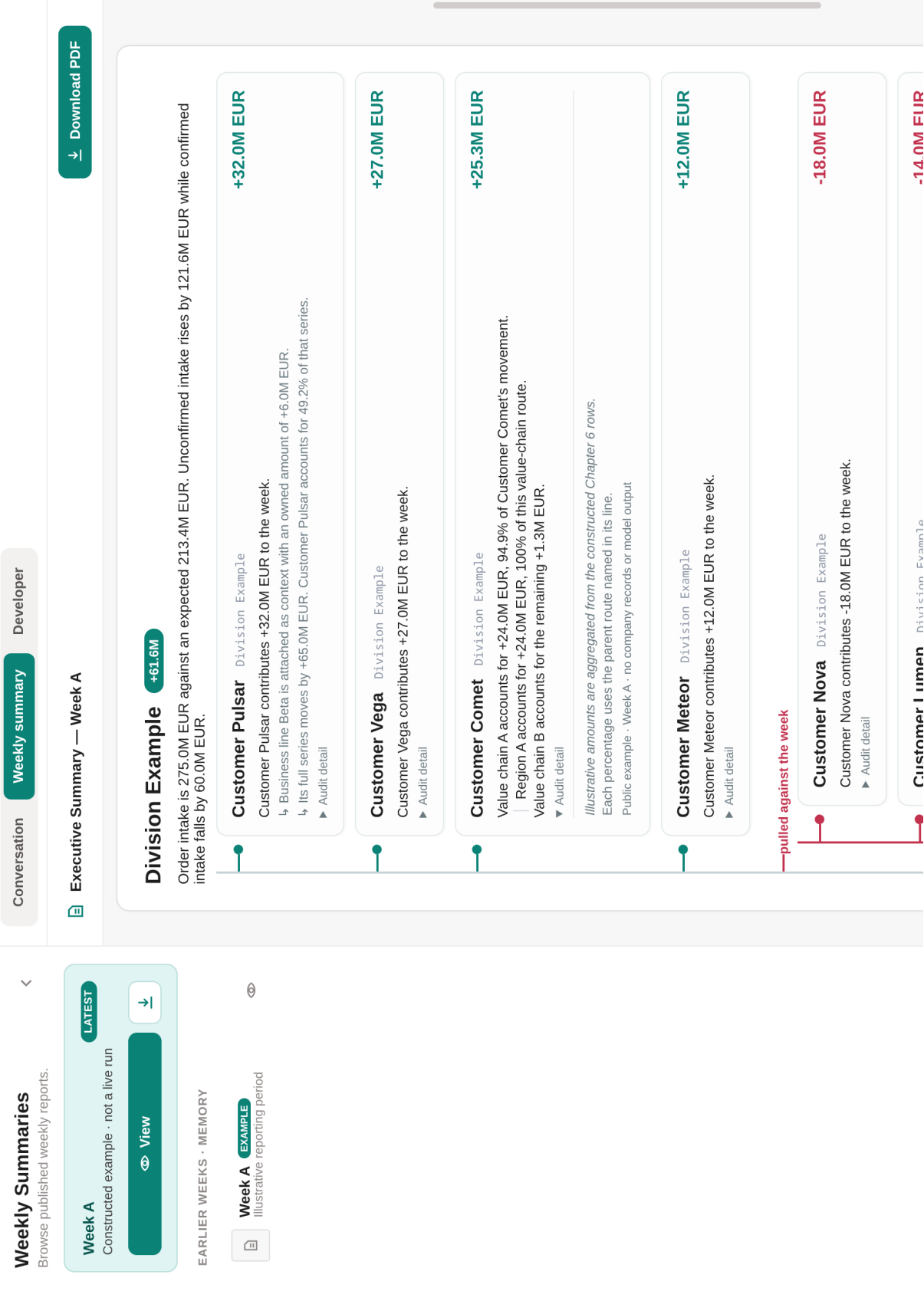


Figure D.3 Report detail after scrolling within the same view. Customer Comet's nested lines name the parent of each percentage. Audit detail is expanded, while opposing movements form a separate branch. Business line Beta's context amount is distinguished from its full-series deviation.

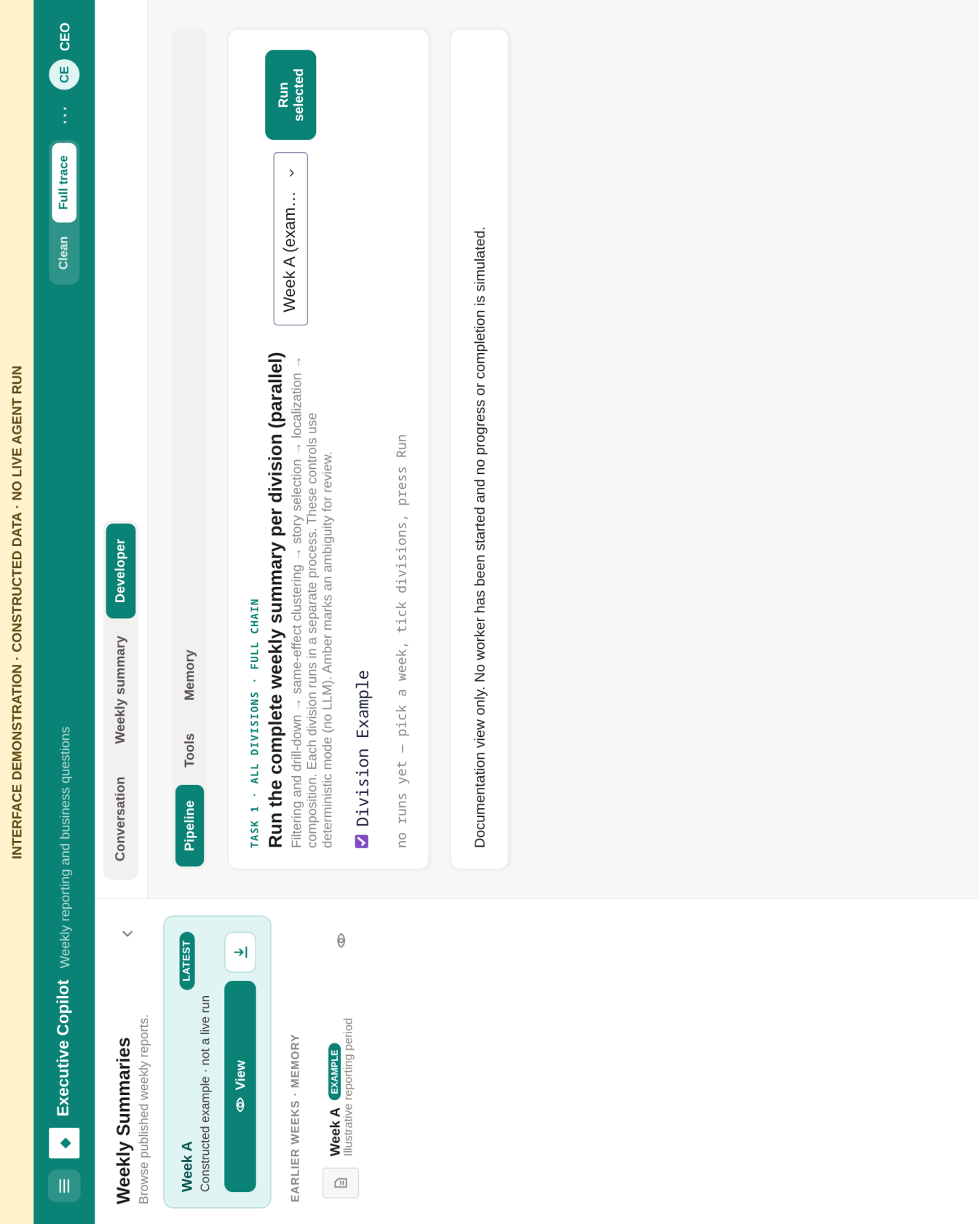


Figure D.4 Complete-summary controls in the Developer area, shown before execution. The operator selects a reporting week and divisions. These controls invoke deterministic mode. No worker was started for this capture, and no completion or timing is simulated.

Weekly Summaries

Browse published weekly reports.

Week A

Constructed example · not a live run

View

Download

EARLIER WEEKS · MEMORY

Week A

EXAMPLE

Illustrative reporting period

Conversation

Weekly summary

Developer

Pipeline

Tools

Memory

SINGLE-TOOL RUNNER · CONFIGURED BACKEND · NO LLM

Run one deterministic tool

get_value

Run

Illustrative single-value request for the constructed example. The observation below is supplied by the capture fixture, not a live tool call.

kpi (string)

confirmed

name_of_series (string)

division_Example

date (string · default: '')

2030-02-10

```
{
  "basis": "constructed interface fixture",
  "kpi": "confirmed",
  "unit": "EUR",
  "division": "Example",
  "week": "A",
  "value": 7650000.0
}
```

KEYWORD PREVIEW · DEVELOPMENT ONLY

Preview keyword matches to tools

This diagnostic uses keyword matching, not the ladder's model-based catalog selection.

e.g. Which key accounts are flagged as outliers this week?

Preview

Figure D.5 Selected tool-development controls. The parameter form uses the tool's declared schema, while its displayed observation is a constructed fixture. The keyword preview is a development diagnostic, not the progressive-disclosure selector used by the conversational ladder.

Weekly Summaries

Browse published weekly reports.

Week A

Constructed example · not a live run

View

LATEST

↓

EARLIER WEEKS · MEMORY

Week A

EXAMPLE

Illustrative reporting period

Conversation

Weekly summary

Developer

Pipeline

Tools

Memory

MEMORY · WHAT THE SYSTEM KNOWS

Semantic context and memory

semantic (declared · measured) → experiential (delivered answers, unverified) → working (per-run) · taxonomy from the harness paper

SEMANTIC · TENANT-DECLARED

approved semantic YAML · tenant Example (constructed)

approved illustrative period by Example reviewer · no draft waiting

measure: Order intake (EUR)

KPIS

- order intake: Confirmed plus unconfirmed orders
- confirmed: Confirmed orders only
- unconfirmed: Unconfirmed orders only

AXES

- customer: Customer Comet, Customer Orbit, ...
- business_line: Alpha, Beta, Gamma
- value_chain: Value chain A, B, ...
- region: Region A, B, ...

KPI FAMILIES (ONE MOVEMENT, MANY KPIS)

- confirmed + unconfirmed → order_intake-family

AXIS TYPES + SUMMARY PARAMS

- reporting_week: time
- customer: entity

SHARED · PUBLISHED WEEKLY PRODUCTS

registered by the summary pipeline; the conversational agent reads them only when asked about the week

- summary: Example: Week-A (constructed fixture)

WORKING MEMORY

conversation state is maintained per session

This view does not list individual conversation sessions.

Figure D.6 Selected read-only Memory cards with staged context metadata and a constructed summary handle. The cards display semantic definitions, shared products, and a note on session working state. They do not provide browser-based context construction or approval.

E Detailed Algorithms

This appendix contains the detailed procedures referenced in Chapter 5. Their inputs, decision rules, and outputs are explained in the corresponding sections.

Algorithm 4 Pair measurement on shared attribution frames

Require: Normalized candidate predicates and reporting week

Ensure: Pair records with signed ratios and evidence

```
1: procedure MEASURE_ALL_PAIRS(candidates, week)
2:   cache  $\leftarrow$  empty frame cache
3:   dossiers  $\leftarrow$  {}
4:   for each unordered pair (A, B) of candidates do
5:     ancestor  $\leftarrow$  shared part of A and B, then the nearest analytics row above it
6:     if ancestor is missing then
7:       raise missing-ancestor error
8:     end if
9:     dimensions  $\leftarrow$  sorted requested dimensions beyond shared predicate
10:    frame  $\leftarrow$  cache.frame(ancestor, dimensions, week, expected value)
11:    mass_A, mass_B  $\leftarrow$  frame.deviation(A), frame.deviation(B)
12:    mass_AB  $\leftarrow$  0
13:    if A and B have a compatible intersection then
14:      mass_AB  $\leftarrow$  frame.deviation(A and B)
15:    end if
16:    ratio_A  $\leftarrow$  0
17:    ratio_B  $\leftarrow$  0
18:    if mass_A  $\neq$  0 then
19:      ratio_A  $\leftarrow$  mass_AB / mass_A
20:    end if
21:    if mass_B  $\neq$  0 then
22:      ratio_B  $\leftarrow$  mass_AB / mass_B
23:    end if
24:    evidence  $\leftarrow$  tiers, directional coverage, official and frame deviations
25:    ratio_available  $\leftarrow$  mass_A  $\neq$  0 and mass_B  $\neq$  0
26:    dossiers[(A, B)]  $\leftarrow$  signed masses, ratios, ratio_available, evidence
27:  end for
28:  return dossiers
29: end procedure
```

Algorithm 5 Impact evidence of one cluster

Require: Cluster, contained clusters, shared frame, reporting scope, scope deviation, and working parameters

Ensure: Measured deviation, lift, cluster share, depth factor, and unscaled impact score

```
1: procedure IMPACT_EVIDENCE(cluster, frame, parameters)
2:   slice  $\leftarrow$  representative predicate of cluster
3:   deviation  $\leftarrow$  frame.deviation(slice)
4:   scope_current  $\leftarrow$  |frame.current(scope)|
5:   if scope_current = 0 then
6:     scope_current  $\leftarrow$  1
7:   end if
8:   deviation_share  $\leftarrow$  0
9:   if scope_deviation  $\neq$  0 then
10:    deviation_share  $\leftarrow$  |deviation| / |scope_deviation|
11:   end if
12:   value_share  $\leftarrow$  |frame.current(slice)| / scope_current
13:   lift  $\leftarrow$  1
14:   if not flat_scope and value_share  $\neq$  0 then
15:     lift  $\leftarrow$  deviation_share / value_share
16:   end if
17:   shares  $\leftarrow$  []
18:   for each other in contained_clusters do
19:     if other is a refinement of cluster then
20:       continue
21:     end if
22:     if directional_coverage(other, cluster) < strict_merge_min then
23:       continue
24:     end if
25:     if depth(other) > depth(cluster) then
26:       continue
27:     end if
28:     share  $\leftarrow$  0
29:     if deviation  $\neq$  0 then
30:       other_mass  $\leftarrow$  frame.deviation(representative(other))
31:       share  $\leftarrow$  |other_mass| / |deviation|
32:     end if
33:     append share to shares
34:   end for
35:   sort shares in descending order
36:   cluster_share  $\leftarrow$  1
37:   if shares is not empty then
38:     if first(shares)  $\geq$  leader_threshold then
39:       cluster_share  $\leftarrow$  max(0, 1 - first(shares))
40:     else
41:       selected  $\leftarrow$  first max_case_b_children entries of shares
42:       qualifying  $\leftarrow$  entries of selected at or above share_rule
43:       if qualifying is not empty and sum(qualifying)  $\geq$  group_requirement then
44:         cluster_share  $\leftarrow$  max(0, 1 - sum(qualifying))
45:       end if
46:     end if
47:   end if
48:   depth_discount_factor  $\leftarrow$  depth_discount ^ max(depth(cluster) - 1, 0)
49:   weighted_amount  $\leftarrow$  |deviation| x max(lift, 1e-9) ^ lift_power
50:   score  $\leftarrow$  weighted_amount x cluster_share x depth_discount_factor
51:   return deviation, lift, cluster_share, depth_discount_factor, score
52: end procedure
```

Algorithm 6 Deterministic placement

Require: Clusters, frame, scope, floor, and story budget

Ensure: Placement, evidence, remainder, and iteration trace

```
1: procedure PLACE_DETERMINISTIC(clusters, frame, scope, floor, story_budget)
2:   active  $\leftarrow$  clusters
3:   tried  $\leftarrow$  {}
4:   converged  $\leftarrow$  false
5:   for iteration in 1..MAX_ITERATIONS do
6:     rank active by score, refinements after their parent
7:     for each c in active in rank order do
8:       role(c)  $\leftarrow$  branch if c is a refinement of a placed story
9:       container if c contains a placed distinct story
10:      dependent if  $\text{overlap}(c, \text{some placed story}) \geq \text{context\_overlap\_min}$ 
11:      story otherwise
12:    end for
13:    own records by role priority, then rank
14:    drop  $\leftarrow$  branches, owners below floor, context whose remainder is below floor
15:    if drop is not empty then
16:      remove drop and record it as evidence
17:      continue
18:    end if
19:    movers  $\leftarrow$  unowned cells grouped by each story dimension set,
20:    ranked by  $|\text{net}| \times \text{cluster\_share} \times \text{depth\_discount\_factor}$ 
21:    candidate  $\leftarrow$  first mover with  $|\text{net}| \geq \text{floor}$  that is not a placeholder,
22:    not in tried, contains no placed story, and either
23:    fits the story budget or is the first opposing mover
24:    if no candidate then
25:      record stopping reason
26:      converged  $\leftarrow$  true
27:      break
28:    end if
29:    add candidate to active and tried with its direction role
30:  end for
31:  if not converged then
32:    raise placement non-convergence error
33:  end if
34:  remainder and accounting as in Algorithm 3
35:  return active with owned amounts, evidence, remainder, iteration trace
36: end procedure
```

▷ as in Algorithm 3

Algorithm 7 Story localization

Require: Slice, expected value, deviation of the slice (at the root the representative's series deviation), dimensions, depth, and the parent word

Ensure: Localization lines, each recording the deviation its shares divide by and the word used for it

```
1: procedure LOCALIZE(slice, expected, deviation, dimensions, depth, parent_word)
2:   if depth  $\geq$  max_depth or |deviation| < floor or dimensions is empty then
3:     return []
4:   end if
5:   for each dimension in dimensions do
6:     rows[dimension]  $\leftarrow$  attribution split of slice by dimension, placeholders removed,
7:     sorted by |deviation|
8:     top_share[dimension]  $\leftarrow$  deviation of the first row / deviation
9:   end for
10:  located  $\leftarrow$  dimensions with top_share  $\geq$  location_share
11:  if located is not empty then
12:    child  $\leftarrow$  slice narrowed to the top value of every located dimension
13:    path, route_deviation, coverage  $\leftarrow$  measured explanatory path through located
14:    emit route, offset, and audit lines using measured coverage,
15:    each with denominator = deviation and parent_word
16:    return lines + localize(child, expected of child, route_deviation,
17:    dimensions without located, depth + 1, "this route")
18:  end if
19:  lead  $\leftarrow$  the dimension with the largest top_share
20:  if top_share[lead]  $\geq$  lead_share then
21:    child  $\leftarrow$  slice narrowed to the top value of lead
22:    emit lead line (amount, value, share of parent_word), denominator = deviation
23:    return lines + localize(child, expected of child, deviation of child,
24:    dimensions without lead, depth + 1, "that slice")
25:  end if
26:  carriers  $\leftarrow$  rows[lead] with share  $\geq$  name_share and |deviation|  $\geq$  floor
27:  if count of carriers  $\geq$  2 then
28:    emit split and up to three leaves with measurement records
29:    emit raw cross-check against the whole story when slice is narrowed
30:    return lines
31:  end if
32:  emit diffuse line
33:  return lines
34: end procedure
```

Algorithm 8 The guarded phrasing pass

Require: Template sentences and protected names

Ensure: Accepted wording or the original template per sentence

```
1: procedure PHRASE(sentences)
2:   text  $\leftarrow$  original template text for every sentence
3:   for each sentence s with its protected names do
4:     masked[s], mapping[s]  $\leftarrow$  protect names, then amounts
5:   end for
6:   proposals  $\leftarrow$  one model call with masked, or {} on any failure
7:   for each sentence s do
8:     if s has no proposal then
9:       continue
10:    end if
11:    p  $\leftarrow$  proposal for s
12:    if fact_sequence(masked[s]) = fact_sequence(p) and p has no digit outside placeholders and  $\text{length}(p) \leq$ 
      max_growth  $\times$  length(masked[s]) + max_extra then
13:      text[s]  $\leftarrow$  restore(p, mapping[s])
14:    else
15:      text[s]  $\leftarrow$  template text of s
16:    end if
17:  end for
18:  return text
19: end procedure
```

Bibliography

- [1] OpenAI. “GPT-4 Technical Report”. In: *CoRR* abs/2303.08774 (2023). DOI: 10.48550/ARXIV.2303.08774. arXiv: 2303.08774. URL: <https://doi.org/10.48550/arXiv.2303.08774>.
- [2] D. Guo et al. “DeepSeek-R1 incentivizes reasoning in LLMs through reinforcement learning”. In: *Nat.* 645.8081 (2025), pp. 633–638. DOI: 10.1038/S41586-025-09422-Z. URL: <https://doi.org/10.1038/s41586-025-09422-z>.
- [3] OpenAI. *GPT-5 System Card*. <https://openai.com/index/gpt-5-system-card/>. Accessed 2026-08-19. 2025.
- [4] X. Ning, K. Tieu, D. Fu, T. Wei, Z. Li, Y. Bei, J. Zou, M. Ai, Z. Liu, T. Li, L. Chen, Y. Zhao, K. Yang, B. Li, C. Qian, G. Li, X. Lin, Z. Zeng, R. Qiu, S. Chen, Y. Sun, X. Yang, R. Wang, R. Pan, C. Yang, D. Zhang, L. Fang, Z. Cui, Y. Cao, P. Chen, D. Sun, R. Chen, M. Srinivasan, N. Mathur, Y. Xia, H. Li, H. Yan, P. Lu, L. Zhang, T. Zhang, H. Tong, and J. He. “Code as Agent Harness”. In: *CoRR* abs/2605.18747 (2026). DOI: 10.48550/ARXIV.2605.18747. arXiv: 2605.18747. URL: <https://doi.org/10.48550/arXiv.2605.18747>.
- [5] J. Gray, S. Chaudhuri, A. Bosworth, A. Layman, D. Reichart, M. Venkatrao, F. Pellow, and H. Pirahesh. “Data Cube: A Relational Aggregation Operator Generalizing Group-by, Cross-Tab, and Sub Totals”. In: *Data Min. Knowl. Discov.* 1.1 (1997), pp. 29–53. DOI: 10.1023/A:1009726021843. URL: <https://doi.org/10.1023/A:1009726021843>.
- [6] R. Agrawal, A. Gupta, and S. Sarawagi. “Modeling Multidimensional Databases”. In: *Proceedings of the Thirteenth International Conference on Data Engineering, April 7-11, 1997, Birmingham, UK*. Ed. by W. A. Gray and P. Larson. IEEE Computer Society, 1997, pp. 232–243. DOI: 10.1109/ICDE.1997.581777. URL: <https://doi.org/10.1109/ICDE.1997.581777>.
- [7] S. Chaudhuri and U. Dayal. “An Overview of Data Warehousing and OLAP Technology”. In: *SIGMOD Rec.* 26.1 (1997), pp. 65–74. DOI: 10.1145/248603.248616. URL: <https://doi.org/10.1145/248603.248616>.
- [8] V. Chandola, A. Banerjee, and V. Kumar. “Anomaly detection: A survey”. In: *ACM Comput. Surv.* 41.3 (2009), 15:1–15:58. DOI: 10.1145/1541880.1541882. URL: <https://doi.org/10.1145/1541880.1541882>.
- [9] L. Wang, C. Ma, X. Feng, Z. Zhang, H. Yang, J. Zhang, Z. Chen, J. Tang, X. Chen, Y. Lin, W. X. Zhao, Z. Wei, and J. Wen. “A survey on large language model based autonomous agents”. In: *Frontiers Comput. Sci.* 18.6 (2024), p. 186345. DOI: 10.1007/S11704-024-40231-1. URL: <https://doi.org/10.1007/s11704-024-40231-1>.
- [10] L. Gao, A. Madaan, S. Zhou, U. Alon, P. Liu, Y. Yang, J. Callan, and G. Neubig. “PAL: Program-aided Language Models”. In: *International Conference on Machine Learning, ICML 2023, 23-29 July 2023, Honolulu, Hawaii, USA*. Ed. by A. Krause, E. Brunskill, K. Cho, B. Engelhardt, S. Sabato, and J. Scarlett. Vol. 202. Proceedings of Machine Learning Research. PMLR, 2023, pp. 10764–10799. URL: <https://proceedings.mlr.press/v202/gao23f.html>.
- [11] T. Schick, J. Dwivedi-Yu, R. Dessì, R. Raileanu, M. Lomeli, E. Hambro, L. Zettlemoyer, N. Cancedda, and T. Scialom. “Toolformer: Language Models Can Teach Themselves to Use Tools”. In: *Advances in Neural Information Processing Systems 36: Annual Conference on Neural Information Processing Systems 2023, NeurIPS 2023, New Orleans, LA, USA, December 10 - 16, 2023*. Ed. by A. Oh, T. Naumann, A. Globerson, K. Saenko, M. Hardt, and S. Levine. 2023. URL: <http://papers>.

nips.cc/paper_files/paper/2023/hash/d842425e4bf79ba039352da0f658a906-Abstract-Conference.html.

- [12] OpenAI. *Function calling — OpenAI API documentation*. <https://developers.openai.com/api/docs/guides/function-calling>. Accessed 2026-08-21. 2023.
- [13] Anthropic. *Model Context Protocol*. <https://modelcontextprotocol.io>. Open standard for connecting AI applications to external systems. Accessed 2026-08-20. 2024.
- [14] Anthropic. *Equipping agents for the real world with Agent Skills*. <https://www.anthropic.com/engineering/equipping-agents-for-the-real-world-with-agent-skills>. Published October 16, 2025. Accessed 2026-08-20. 2025.
- [15] Y. Qin, S. Liang, Y. Ye, K. Zhu, L. Yan, Y. Lu, Y. Lin, X. Cong, X. Tang, B. Qian, S. Zhao, L. Hong, R. Tian, R. Xie, J. Zhou, M. Gerstein, D. Li, Z. Liu, and M. Sun. “ToolLLM: Facilitating Large Language Models to Master 16000+ Real-world APIs”. In: *The Twelfth International Conference on Learning Representations, ICLR 2024, Vienna, Austria, May 7-11, 2024*. OpenReview.net, 2024. URL: <https://openreview.net/forum?id=dHng200Jjr>.
- [16] S. G. Patil, T. Zhang, X. Wang, and J. E. Gonzalez. “Gorilla: Large Language Model Connected with Massive APIs”. In: *Advances in Neural Information Processing Systems 37: Annual Conference on Neural Information Processing Systems 2024, NeurIPS 2024, Vancouver, BC, Canada, December 10 - 15, 2024*. Ed. by A. Globersons, L. Mackey, D. Belgrave, A. Fan, U. Paquet, J. M. Tomczak, and C. Zhang. 2024. URL: http://papers.nips.cc/paper_files/paper/2024/hash/e4c61f578ff07830f5c37378dd3ecb0d-Abstract-Conference.html.
- [17] Y. He, Y. Zhao, J. Wang, and H. Chen. “Is Progressive Disclosure All You Need for Long-Context Agents?” In: *CoRR* abs/2607.17598 (2026). DOI: 10.48550/ARXIV.2607.17598. arXiv: 2607.17598. URL: <https://doi.org/10.48550/arXiv.2607.17598>.
- [18] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. R. Narasimhan, and Y. Cao. “ReAct: Synergizing Reasoning and Acting in Language Models”. In: *The Eleventh International Conference on Learning Representations, ICLR 2023, Kigali, Rwanda, May 1-5, 2023*. OpenReview.net, 2023. URL: https://openreview.net/forum?id=WE_vluYUL-X.
- [19] L. Zheng, W. Chiang, Y. Sheng, S. Zhuang, Z. Wu, Y. Zhuang, Z. Lin, Z. Li, D. Li, E. P. Xing, H. Zhang, J. E. Gonzalez, and I. Stoica. “Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena”. In: *Advances in Neural Information Processing Systems 36: Annual Conference on Neural Information Processing Systems 2023, NeurIPS 2023, New Orleans, LA, USA, December 10 - 16, 2023*. Ed. by A. Oh, T. Naumann, A. Globerson, K. Saenko, M. Hardt, and S. Levine. 2023. URL: http://papers.nips.cc/paper_files/paper/2023/hash/91f18a1287b398d378ef22505bf41832-Abstract-Datasets_and_Benchmarks.html.
- [20] G. Katsogiannis-Meimarakis and G. Koutrika. “A survey on deep learning approaches for text-to-SQL”. In: *VLDB J.* 32.4 (2023), pp. 905–936. DOI: 10.1007/S00778-022-00776-8. URL: <https://doi.org/10.1007/s00778-022-00776-8>.
- [21] S. A. Somayajula, M. M. Liu, C. Lei, F. Parllaku, D. Garcia, R. Wang, S. F. A. Shah, A. Bansal, S. Bharadwaj, T. Sheng, S. Ravi, and D. Roth. “SOMA-SQL: Resolving Multi-Source Ambiguity in NL-to-SQL via Synthetic Log and Execution Probing”. In: *CoRR* abs/2606.11424 (2026). DOI: 10.48550/ARXIV.2606.11424. arXiv: 2606.11424. URL: <https://doi.org/10.48550/arXiv.2606.11424>.
- [22] T. Yu, R. Zhang, K. Yang, M. Yasunaga, D. Wang, Z. Li, J. Ma, I. Li, Q. Yao, S. Roman, Z. Zhang, and D. R. Radev. “Spider: A Large-Scale Human-Labeled Dataset for Complex and Cross-Domain Semantic Parsing and Text-to-SQL Task”. In: *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing, Brussels, Belgium, October 31 - November 4, 2018*. Ed. by E. Riloff, D. Chiang, J. Hockenmaier, and J. Tsujii. Association for Computational Linguistics, 2018, pp. 3911–3921. DOI: 10.18653/V1/D18-1425. URL: <https://doi.org/10.18653/v1/d18-1425>.

- [23] J. Li, B. Hui, G. Qu, J. Yang, B. Li, B. Li, B. Wang, B. Qin, R. Geng, N. Huo, X. Zhou, C. Ma, G. Li, K. C. Chang, F. Huang, R. Cheng, and Y. Li. “Can LLM Already Serve as A Database Interface? A Blg Bench for Large-Scale Database Grounded Text-to-SQLs”. In: *Advances in Neural Information Processing Systems 36: Annual Conference on Neural Information Processing Systems 2023, NeurIPS 2023, New Orleans, LA, USA, December 10 - 16, 2023*. Ed. by A. Oh, T. Naumann, A. Globerson, K. Saenko, M. Hardt, and S. Levine. 2023. URL: http://papers.nips.cc/paper_files/paper/2023/hash/83fc8fab1710363050bbd1d4b8cc0021-Abstract-Datasets_and_Benchmarks.html.
- [24] Y. Gao, Y. Liu, X. Li, X. Shi, Y. Zhu, Y. Wang, S. Li, W. Li, Y. Hong, Z. Luo, J. Gao, L. Mou, and Y. Li. “A Preview of XiYan-SQL: A Multi-Generator Ensemble Framework for Text-to-SQL”. In: *CoRR* abs/2411.08599 (2024). DOI: 10.48550/ARXIV.2411.08599. arXiv: 2411.08599. URL: <https://doi.org/10.48550/arXiv.2411.08599>.
- [25] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W. Yih, T. Rocktäschel, S. Riedel, and D. Kiela. “Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks”. In: *Advances in Neural Information Processing Systems 33: Annual Conference on Neural Information Processing Systems 2020, NeurIPS 2020, December 6-12, 2020, virtual*. Ed. by H. Larochelle, M. Ranzato, R. Hadsell, M. Balcan, and H. Lin. 2020. URL: <https://proceedings.neurips.cc/paper/2020/hash/6b493230205f780e1bc26945df7481e5-Abstract.html>.
- [26] A. K. Elmagarmid, P. G. Ipeirotis, and V. S. Verykios. “Duplicate Record Detection: A Survey”. In: *IEEE Trans. Knowl. Data Eng.* 19.1 (2007), pp. 1–16. DOI: 10.1109/TKDE.2007.250581. URL: <https://doi.org/10.1109/TKDE.2007.250581>.
- [27] Y. Gao and Z. Luo. “Automatic database description generation for Text-to-SQL”. In: *CoRR* abs/2502.20657 (2025). DOI: 10.48550/ARXIV.2502.20657. arXiv: 2502.20657. URL: <https://doi.org/10.48550/arXiv.2502.20657>.
- [28] XGenerationLab. *XiYan-DBDescGen: Automatic Database Description Generation for Text-to-SQL*. <https://github.com/XGenerationLab/XiYan-DBDescGen>. Open-source software, Apache License 2.0. Accessed 2026-08-31. 2025.
- [29] Y. Liu, Y. Zhu, Y. Gao, Z. Luo, X. Li, X. Shi, Y. Hong, J. Gao, Y. Li, B. Ding, and J. Zhou. “XiYan-SQL: A Novel Multi-Generator Framework for Text-to-SQL”. In: *IEEE Trans. Knowl. Data Eng.* 38.4 (2026), pp. 2474–2487. DOI: 10.1109/TKDE.2026.3657851. URL: <https://doi.org/10.1109/TKDE.2026.3657851>.
- [30] Kaelio. *ktx: The context layer for data agents*. <https://github.com/Kaelio/ktx>. Open-source software, Apache License 2.0. Version 0.16.0 examined. Accessed 2026-08-31. 2025.
- [31] S. Sarawagi, R. Agrawal, and N. Megiddo. “Discovery-Driven Exploration of OLAP Data Cubes”. In: *Advances in Database Technology - EDBT’98, 6th International Conference on Extending Database Technology, Valencia, Spain, March 23-27, 1998, Proceedings*. Ed. by H. Schek, F. Saltor, I. Ramos, and G. Alonso. Vol. 1377. Lecture Notes in Computer Science. Springer, 1998, pp. 168–182. DOI: 10.1007/BFB0100984. URL: <https://doi.org/10.1007/BFB0100984>.
- [32] G. Sathe and S. Sarawagi. “Intelligent Rollups in Multidimensional OLAP Data”. In: *VLDB 2001, Proceedings of 27th International Conference on Very Large Data Bases, September 11-14, 2001, Roma, Italy*. Ed. by P. M. G. Apers, P. Atzeni, S. Ceri, S. Paraboschi, K. Ramamohanarao, and R. T. Snodgrass. Morgan Kaufmann, 2001, pp. 531–540. URL: <http://www.vldb.org/conf/2001/P531.pdf>.
- [33] R. Bhagwan, R. Kumar, R. Ramjee, G. Varghese, S. Mohapatra, H. Manoharan, and P. Shah. “Adtributor: Revenue Debugging in Advertising Systems”. In: *Proceedings of the 11th USENIX Symposium on Networked Systems Design and Implementation, NSDI 2014, Seattle, WA, USA, April 2-4, 2014*. Ed. by R. Mahajan and I. Stoica. USENIX Association, 2014, pp. 43–55. URL: <https://www.usenix.org/conference/nsdi14/technical-sessions/presentation/bhagwan>.

- [34] Y. Sun, Y. Zhao, Y. Su, D. Liu, X. Nie, Y. Meng, S. Cheng, D. Pei, S. Zhang, X. Qu, and X. Guo. “HotSpot: Anomaly Localization for Additive KPIs With Multi-Dimensional Attributes”. In: *IEEE Access* 6 (2018), pp. 10909–10923. DOI: 10.1109/ACCESS.2018.2804764. URL: <https://doi.org/10.1109/ACCESS.2018.2804764>.
- [35] Z. Li, D. Pei, C. Luo, Y. Zhao, Y. Sun, K. Sui, X. Wang, D. Liu, X. Jin, and Q. Wang. “Generic and Robust Localization of Multi-dimensional Root Causes”. In: *30th IEEE International Symposium on Software Reliability Engineering, ISSRE 2019, Berlin, Germany, October 28-31, 2019*. Ed. by K. Wolter, I. Schieferdecker, B. Gallina, M. Cukier, R. Natella, N. R. Ivaki, and N. Laranjeiro. IEEE, 2019, pp. 47–57. DOI: 10.1109/ISSRE.2019.00015. URL: <https://doi.org/10.1109/ISSRE.2019.00015>.
- [36] E. Wu and S. Madden. “Scorpion: Explaining Away Outliers in Aggregate Queries”. In: *Proc. VLDB Endow.* 6.8 (2013), pp. 553–564. DOI: 10.14778/2536354.2536356. URL: <http://www.vldb.org/pvldb/vol6/p553-wu.pdf>.
- [37] F. Abuzaid, P. Kraft, S. Suri, E. Gan, E. Xu, A. Shenoy, A. Anathanaraya, J. Sheu, E. Meijer, X. Wu, J. F. Naughton, P. Bailis, and M. Zaharia. “DIFF: A Relational Interface for Large-Scale Data Explanation”. In: *Proc. VLDB Endow.* 12.4 (2018), pp. 419–432. DOI: 10.14778/3297753.3297761. URL: <http://www.vldb.org/pvldb/vol12/p419-abuzaid.pdf>.
- [38] R. Ding, S. Han, Y. Xu, H. Zhang, and D. Zhang. “QuickInsights: Quick and Automatic Discovery of Insights from Multi-Dimensional Data”. In: *Proceedings of the 2019 International Conference on Management of Data, SIGMOD Conference 2019, Amsterdam, The Netherlands, June 30 - July 5, 2019*. Ed. by P. Boncz, S. Manegold, A. Ailamaki, A. Deshpande, and T. Kraska. ACM, 2019, pp. 317–332. DOI: 10.1145/3299869.3314037. URL: <https://doi.org/10.1145/3299869.3314037>.
- [39] P. Ma, R. Ding, S. Han, and D. Zhang. “MetaInsight: Automatic Discovery of Structured Knowledge for Exploratory Data Analysis”. In: *SIGMOD ’21: International Conference on Management of Data, Virtual Event, China, June 20-25, 2021*. Ed. by G. Li, Z. Li, S. Idreos, and D. Srivastava. ACM, 2021, pp. 1262–1274. DOI: 10.1145/3448016.3457267. URL: <https://doi.org/10.1145/3448016.3457267>.
- [40] Y. Wang, Z. Sun, H. Zhang, W. Cui, K. Xu, X. Ma, and D. Zhang. “DataShot: Automatic Generation of Fact Sheets from Tabular Data”. In: *IEEE Transactions on Visualization and Computer Graphics* 26.1 (Jan. 2020), pp. 895–905. ISSN: 2160-9306. DOI: 10.1109/TVCG.2019.2934398. URL: <https://doi.org/10.1109/TVCG.2019.2934398>.
- [41] D. Shi, X. Xu, F. Sun, Y. Shi, and N. Cao. “Calliope: Automatic Visual Data Story Generation from a Spreadsheet”. In: *IEEE Trans. Vis. Comput. Graph.* 27.2 (2021), pp. 453–463. DOI: 10.1109/TVCG.2020.3030403. URL: <https://doi.org/10.1109/TVCG.2020.3030403>.
- [42] Z. Tang, W. Wang, Z. Zhou, Y. Jiao, B. Xu, B. Niu, X. Zhou, G. Li, Y. He, W. Zhou, Y. Song, C. Tan, B. Wang, C. He, X. Wang, and F. Wu. “LLM/Agent-as-Data-Analyst: A Survey”. In: *CoRR* abs/2509.23988 (2025). DOI: 10.48550/ARXIV.2509.23988. arXiv: 2509.23988. URL: <https://doi.org/10.48550/arXiv.2509.23988>.
- [43] P. Ma, R. Ding, S. Wang, S. Han, and D. Zhang. “InsightPilot: An LLM-Empowered Automated Data Exploration System”. In: *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*. Ed. by Y. Feng and E. Lefever. Singapore: Association for Computational Linguistics, Dec. 2023, pp. 346–352. DOI: 10.18653/v1/2023.emnlp-demo.31. URL: <https://aclanthology.org/2023.emnlp-demo.31/>.
- [44] W. Zhang, Y. Shen, W. Lu, and Y. Zhuang. “Data-Copilot: Bridging Billions of Data and Humans with Autonomous Workflow”. In: *CoRR* abs/2306.07209 (2023). DOI: 10.48550/ARXIV.2306.07209. arXiv: 2306.07209. URL: <https://doi.org/10.48550/arXiv.2306.07209>.

- [45] T. Cai, X. Wang, T. Ma, X. Chen, and D. Zhou. “Large Language Models as Tool Makers”. In: *The Twelfth International Conference on Learning Representations, ICLR 2024, Vienna, Austria, May 7-11, 2024*. OpenReview.net, 2024. URL: <https://openreview.net/forum?id=qV83K9d5WB>.
- [46] Spider 2.0 Team. *Spider 2.0-Lite Leaderboard*. <https://spider2-sql.github.io/>. Official benchmark leaderboard. Accessed 2026-08-31. 2026.
- [47] F. Lei, J. Chen, Y. Ye, R. Cao, D. Shin, H. Su, Z. Suo, H. Gao, W. Hu, P. Yin, V. Zhong, C. Xiong, R. Sun, Q. Liu, S. Wang, and T. Yu. “Spider 2.0: Evaluating Language Models on Real-World Enterprise Text-to-SQL Workflows”. In: *The Thirteenth International Conference on Learning Representations, ICLR 2025, Singapore, April 24-28, 2025*. OpenReview.net, 2025. URL: <https://openreview.net/forum?id=XmProj9cPs>.
- [48] A. Bhaskar, T. Tomar, A. Sathe, and S. Sarawagi. “Benchmarking and Improving Text-to-SQL Generation under Ambiguity”. In: *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing, EMNLP 2023, Singapore, December 6-10, 2023*. Ed. by H. Bouamor, J. Pino, and K. Bali. Association for Computational Linguistics, 2023, pp. 7053–7074. DOI: 10.18653/V1/2023.EMNLP-MAIN.436. URL: <https://doi.org/10.18653/v1/2023.emnlp-main.436>.
- [49] I. Saparina and M. Lapata. “AMBROSIA: A Benchmark for Parsing Ambiguous Questions into Database Queries”. In: *Advances in Neural Information Processing Systems 37: Annual Conference on Neural Information Processing Systems 2024, NeurIPS 2024, Vancouver, BC, Canada, December 10 - 15, 2024*. Ed. by A. Globersons, L. Mackey, D. Belgrave, A. Fan, U. Paquet, J. M. Tomczak, and C. Zhang. 2024. URL: http://papers.nips.cc/paper_files/paper/2024/hash/a4c942a8405cc910f0a833d28d2573cc-Abstract-Datasets_and_Benchmarks_Track.html.
- [50] M. Dong, N. A. Kumar, Y. Hu, A. Chauhan, C. Hang, S. Chang, L. Pan, W. Lan, H. Zhu, J. Jiang, P. Ng, and Z. Wang. “PRACTIQ: A Practical Conversational Text-to-SQL dataset with Ambiguous and Unanswerable Queries”. In: *Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies, NAACL 2025 - Volume 1: Long Papers, Albuquerque, New Mexico, USA, April 29 - May 4, 2025*. Ed. by L. Chiruzzo, A. Ritter, and L. Wang. Association for Computational Linguistics, 2025, pp. 255–273. DOI: 10.18653/V1/2025.NAACL-LONG.13. URL: <https://doi.org/10.18653/v1/2025.naacl-long.13>.
- [51] Z. Ding, Y. Lin, and T. Zeng. “AmbiSQL: Interactive Ambiguity Detection and Resolution for Text-to-SQL”. In: *CoRR* abs/2508.15276 (2025). DOI: 10.48550/ARXIV.2508.15276. arXiv: 2508.15276. URL: <https://doi.org/10.48550/arXiv.2508.15276>.
- [52] Plotly. *Dash Documentation*. URL: <https://dash.plotly.com/> (visited on 09/11/2026).
- [53] Apache Software Foundation. *Apache Hadoop*. URL: <https://hadoop.apache.org/> (visited on 09/11/2026).
- [54] Apache Software Foundation. *Apache Impala*. URL: <https://impala.apache.org/> (visited on 09/11/2026).
- [55] DuckDB. *Connect*. URL: <https://duckdb.org/docs/current/connect/overview> (visited on 09/11/2026).
- [56] LangChain. *LangChain Overview*. URL: <https://docs.langchain.com/oss/python/langchain/overview> (visited on 09/11/2026).
- [57] LangChain. *LangGraph Overview*. URL: <https://docs.langchain.com/oss/python/langgraph/overview> (visited on 09/11/2026).